# 4장

# 분할 정복 기법

DIVIDE AND CONQUER

큰 문제를 잘게 쪼개어 정복하고, 다시 하나로 합친다

# 이번 장에서 배울 내용

4.1 분할 정복의 기본

4.2 병합 정렬 (Merge Sort)

4.3 퀵 정렬 (Quick Sort)

4.4 최대 구간 합

4.5 쿼드 트리

4.6 마스터 정리

4.7 코딩 테스트 예제

4.1

# 분할 정복이란?

"나누어라, 그리고 정복하라"
큰 문제를 작은 문제로 쪼개어 해결하는 알고리즘 설계 전략

# 나누어라, 그리고 정복하라

큰 문제를 같은 유형의 작은 문제 여러 개로 나누어 각각 해결합니다. 작은 문제의 답을 결합해 원래 문제의 답을 만듭니다.

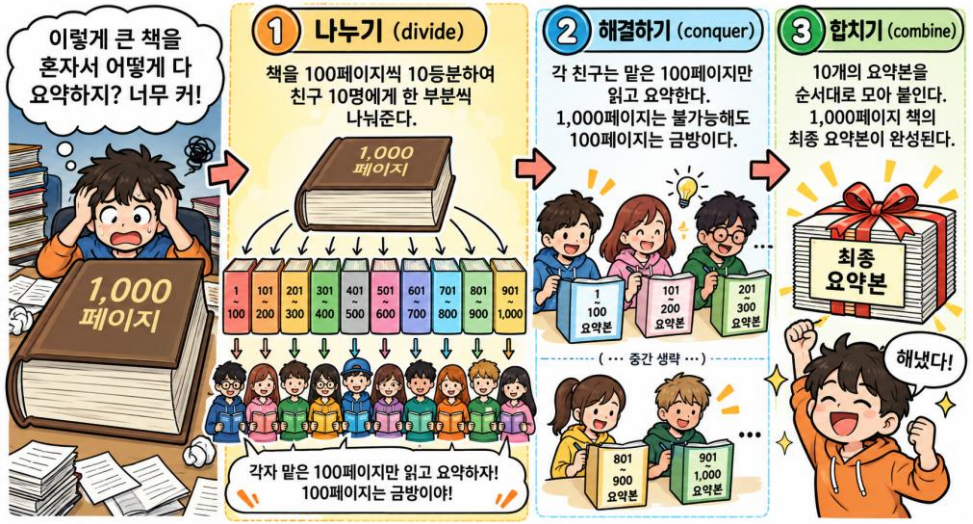
분할, 정복, 결합의 구조를 재귀적으로 적용합니다.

**Divide → Conquer → Combine**



# 직관적 이해 — 거대한 책 요약하기

1,000페이지짜리 책을 혼자 요약하려면 막막하다. 하지만 친구 10명이 있다면 이야기가 달라진다.




### 나누기 (divide)

책을 100페이지씩 10등분하여 친구 10명에게 한 부분씩 나눠준다.

### 해결하기 (conquer)

각 친구는 맡은 100페이지만 읽고 요약한다. 1,000페이지는 불가능해도 100페이지는 금방이다.

### 합치기 (combine)

10개의 요약본을 순서대로 모아 붙인다. 1,000페이지 책의 최종 요약본이 완성된다.

큰 문제를 그대로 상대하면 어렵지만, 작게 쪼개면 각각은 쉽다 — 이것이 분할 정복의 핵심 통찰이다.

# 분할 정복의 3단계

### 분할 (Divide)

문제를 같은 유형의 더 작은 하위 문제 여러 개로 나눈다.

### 정복 (Conquer)

하위 문제를 재귀적으로 해결한다. 충분히 작아지면 직접 푼다.

### 결합 (Combine)

하위 문제의 해답을 조합해 원래 문제의 해답을 만든다.

**분할 정복은 한 번만 나누는 알고리즘이 아니다**

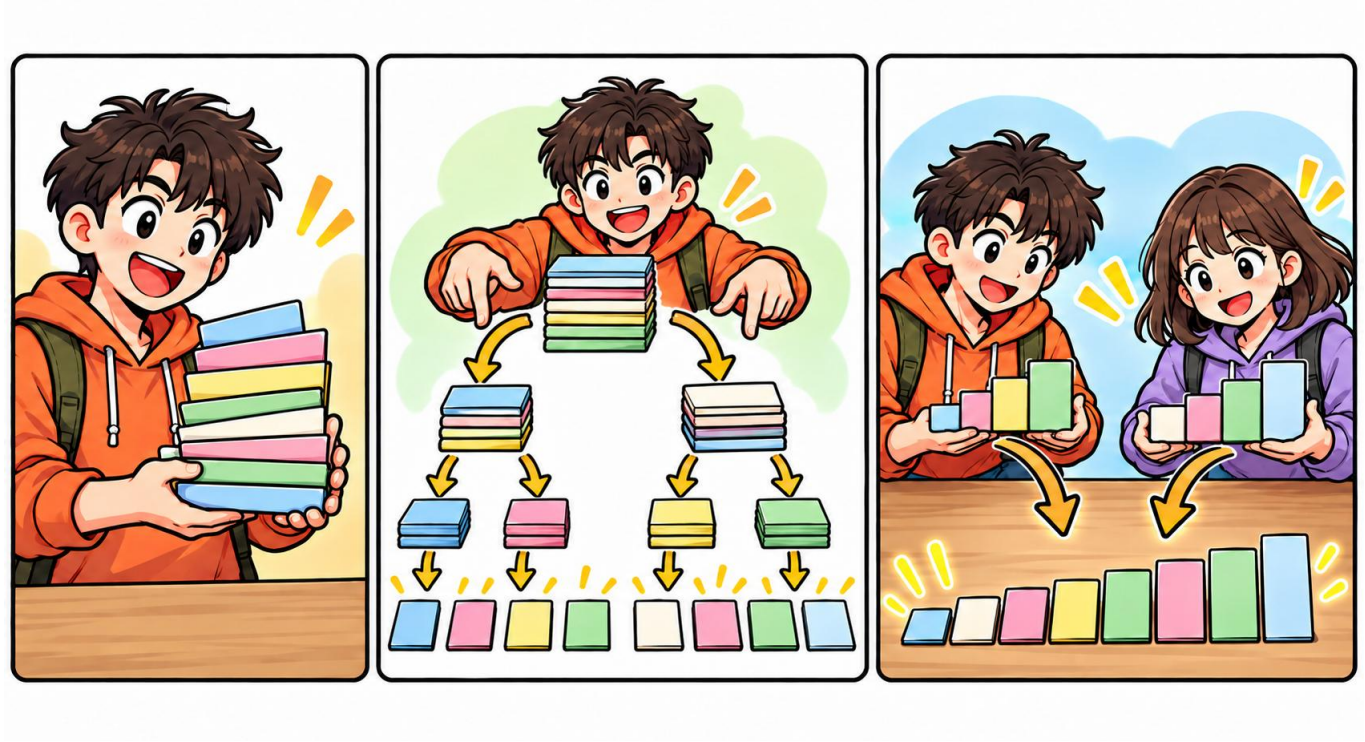
분할된 문제가 아직 커서 즉시 해결할 수 없다면 다시 분할한다. 아주 쉽게 풀릴 때까지 계속 분할한다.

divide_and_conquer.pseudo

```
DIVIDE_AND_CONQUER(P):
    if P의 크기가 충분히 작으면 then return SolveDirectly(P)
    P1, P2, ..., Pk <- Divide(P)                  // 분할
    Si <- DIVIDE_AND_CONQUER(Pi)   for i = 1..k   // 정복
    return Combine(S1, S2, ..., Sk)               // 결합
```

# 병합 정렬: 한 장까지 나누고 합치기

입력: [8, 3, 6, 2, 7, 1, 5, 4] → 출력: [1, 2, 3, 4, 5, 6, 7, 8]
작은 문제의 정렬 결과를 받아, 두 줄의 맨 앞에서 작은 값을 골라 붙입니다.



한 장은 이미 정렬된 상태입니다. 정렬된 두 결과를 병합해 더 큰 결과를 만듭니다.

# 병합 정렬 (Merge Sort)

반으로 나누고, 정렬된 두 배열을 합친다
원소가 하나이면 이미 정렬된 상태다.

정렬 전

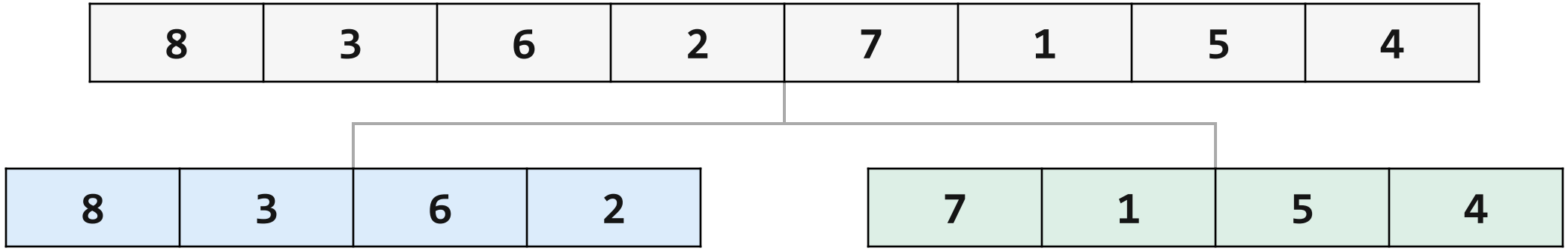
| 8 | 3 | 6 | 2 | 7 | 1 | 5 | 4 |
|---|---|---|---|---|---|---|---|

정렬 후

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|

# 나누기 ① · 8개를 4개씩

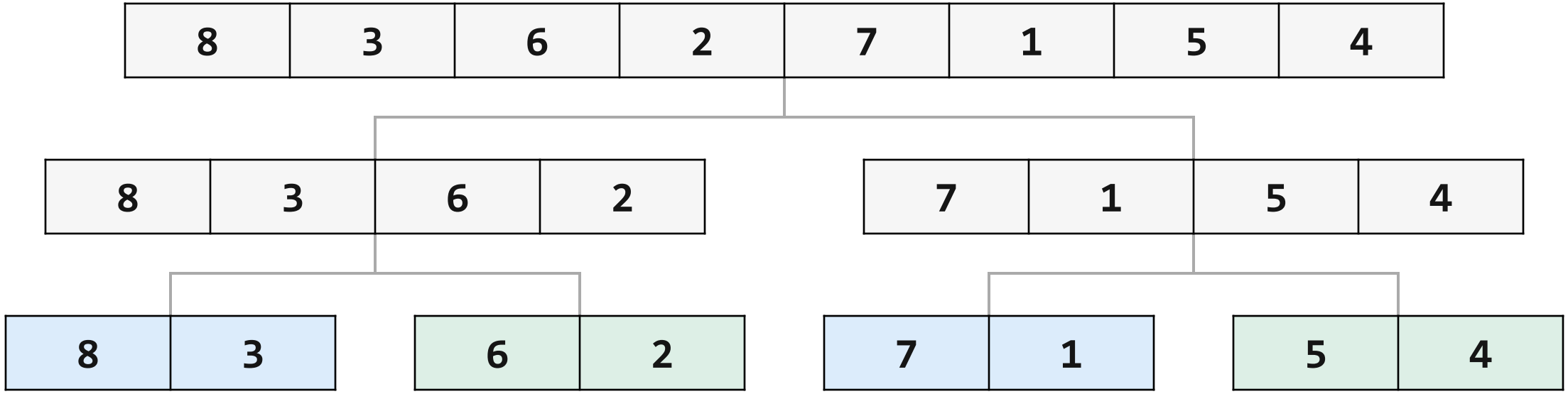
가운데 위치에서 나눕니다. 숫자의 크기는 아직 비교하지 않습니다.
왼쪽 [8,3,6,2], 오른쪽 [7,1,5,4]의 원래 순서를 유지합니다.




다음: 네 개짜리 구간도 더 작은 문제로 나눕니다.

# 나누기 ② · 4개를 2개씩

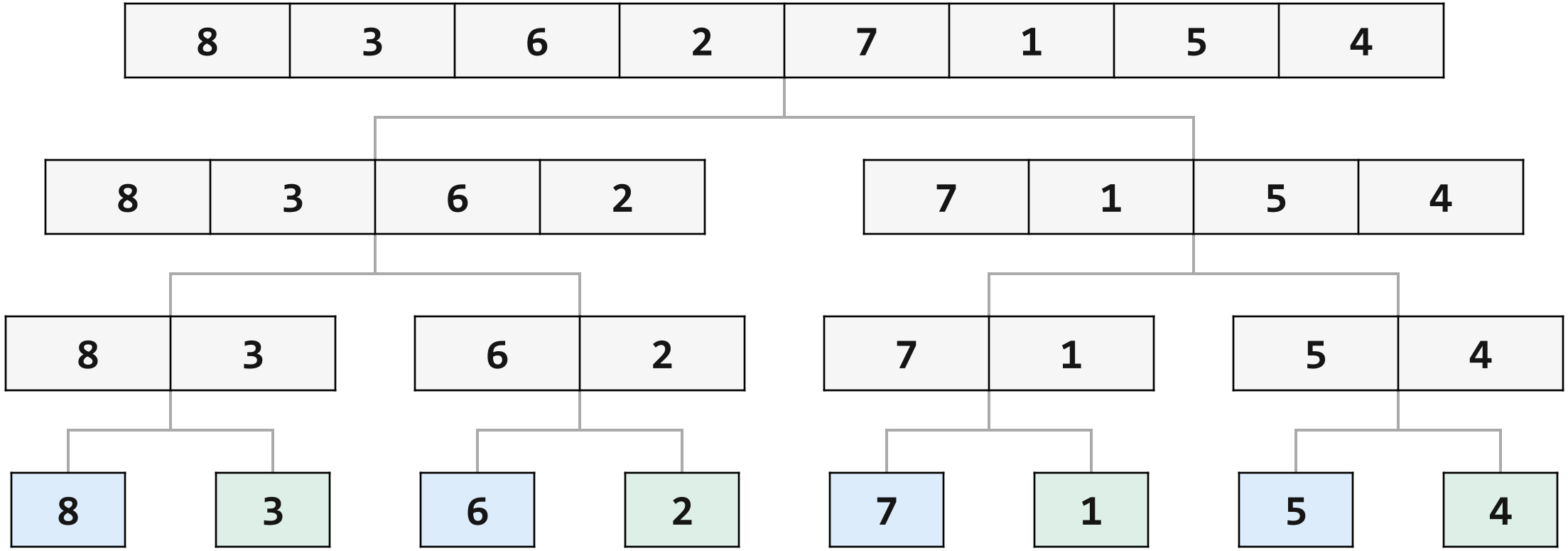
네 개짜리 구간도 각각 가운데에서 나눕니다.
두 개짜리 네 묶음이 생겼지만, 아직 정렬된 결과는 아닙니다.




다음: 두 개짜리 구간을 원소 하나씩 나눕니다.

# 나누기 ③ · 2개를 1개씩

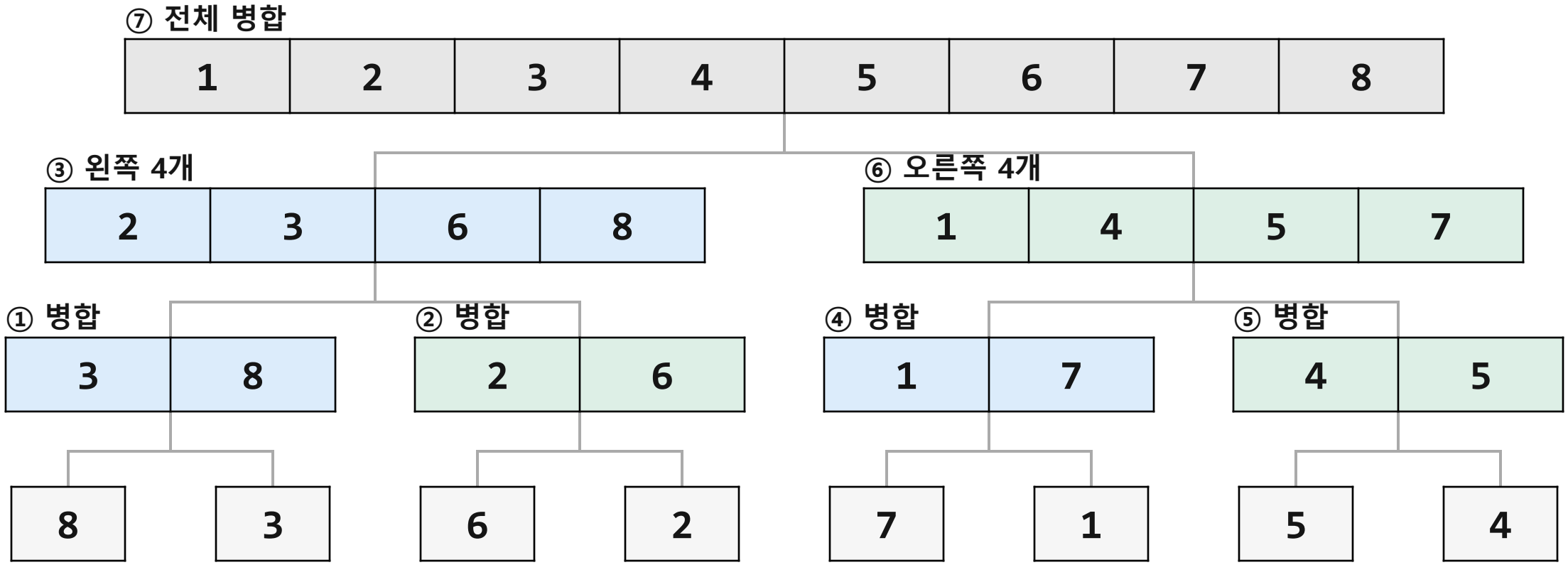
원소가 하나이면 더 정렬할 것이 없습니다. 여기서 재귀 호출을 멈춥니다.
이제 한 원소짜리 결과 두 개를 받아 병합을 시작합니다.




한 원소 = 이미 정렬됨 → 반환 → 두 결과를 병합

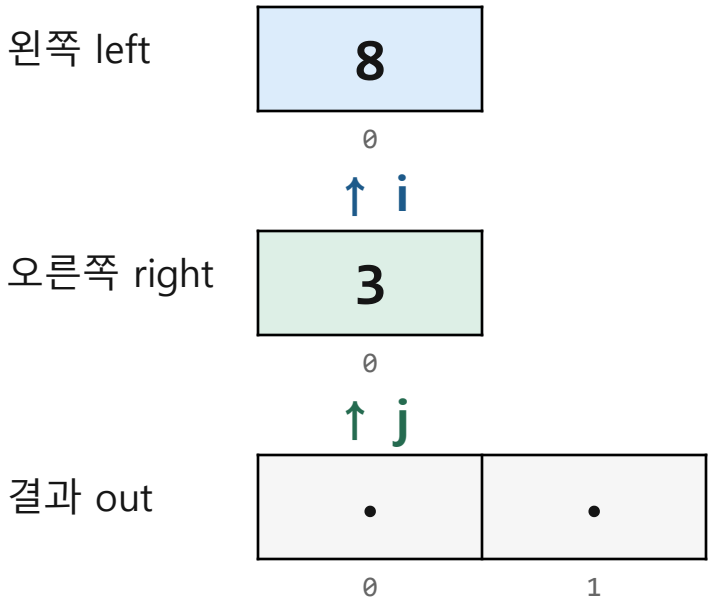
# 실제 병합 순서 · 왼쪽부터 끝낸다

① → ② → ③: 왼쪽 4개 완료 ④ → ⑤ → ⑥: 오른쪽 4개 완료
⑦: 두 결과를 합쳐 전체 8개 정렬

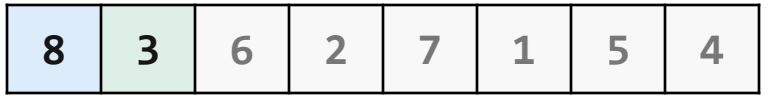


# ① [8]과 [3] 병합 · 시작

i = 0, j = 0. 왼쪽 8과 오른쪽 3을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.




처음 배열에서 이번에 합치는 부분




**왼쪽 4개의 앞쪽 두 개**

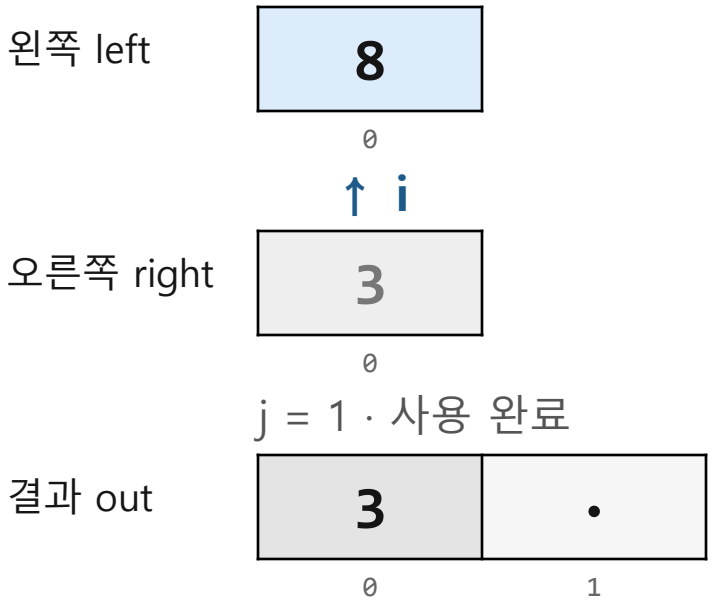
한 원소짜리 두 결과를 받습니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

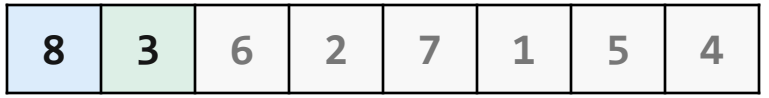
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ① 병합 · 1번째 값 3 추가

왼쪽 8 > 오른쪽 3 → 오른쪽의 3을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.




처음 배열에서 이번에 합치는 부분




**왼쪽 4개의 앞쪽 두 개**

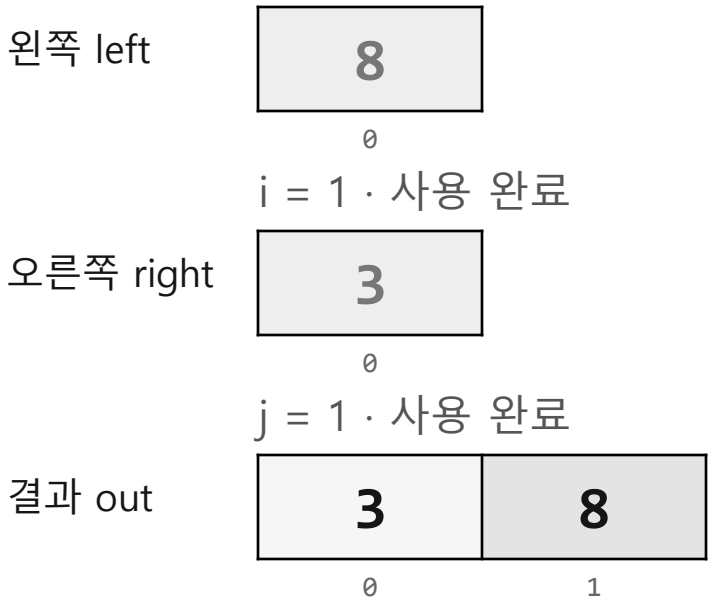
한 원소짜리 두 결과를 받습니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

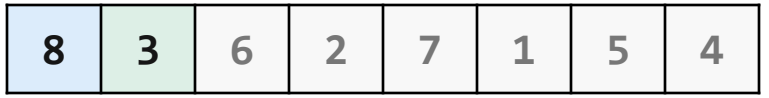
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ① 병합 · 2번째 값 8 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 8을 붙입니다.
i는 0 → 1. 이번 병합의 정렬 결과가 완성되었습니다.




처음 배열에서 이번에 합치는 부분




**왼쪽 4개의 앞쪽 두 개**

이번 병합의 정렬 결과

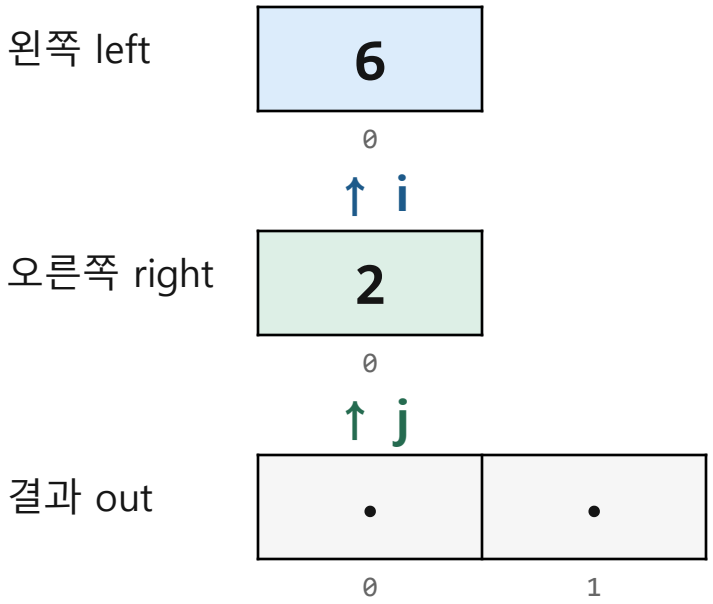
**[3, 8]**

③에서 왼쪽 입력으로 사용

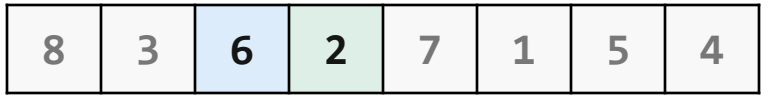
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ② [6]과 [2] 병합 · 시작

i = 0, j = 0. 왼쪽 6과 오른쪽 2을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.




처음 배열에서 이번에 합치는 부분




**왼쪽 4개의 뒤쪽 두 개**

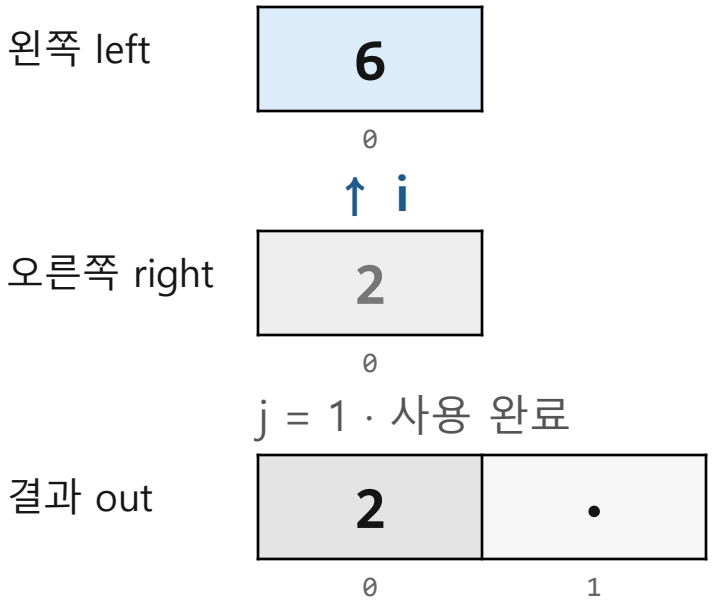
①의 [3,8]은 준비된 채 기다립니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

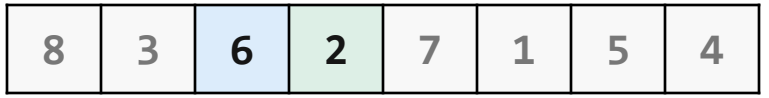
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ② 병합 · 1번째 값 2 추가

왼쪽 6 > 오른쪽 2 → 오른쪽의 2을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.




처음 배열에서 이번에 합치는 부분




**왼쪽 4개의 뒤쪽 두 개**

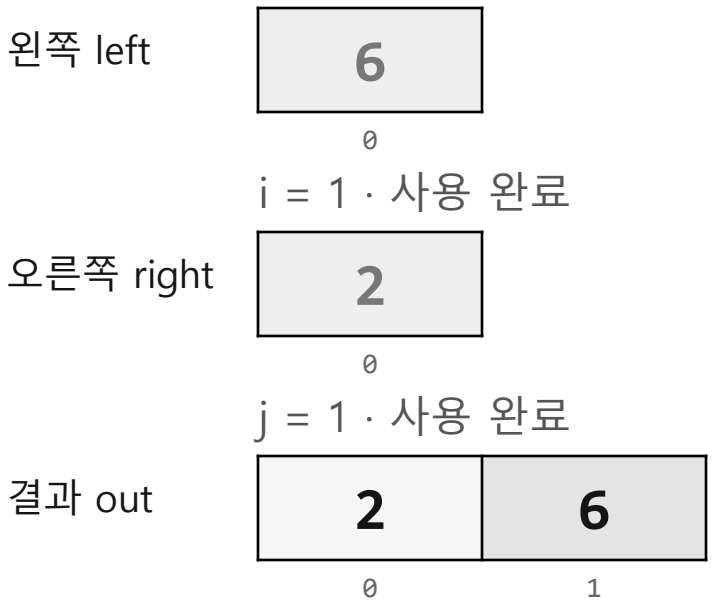
①의 [3,8]은 준비된 채 기다립니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

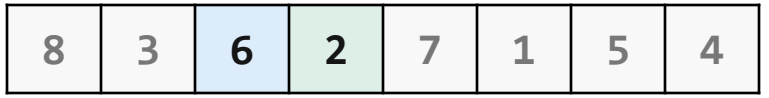
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ② 병합 · 2번째 값 6 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 6을 붙입니다.
i는 0 → 1. 이번 병합의 정렬 결과가 완성되었습니다.




처음 배열에서 이번에 합치는 부분




**왼쪽 4개의 뒤쪽 두 개**

이번 병합의 정렬 결과

**[2, 6]**

③에서 오른쪽 입력으로 사용

파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# 병합의 비교: 고른 쪽의 포인터만 이동

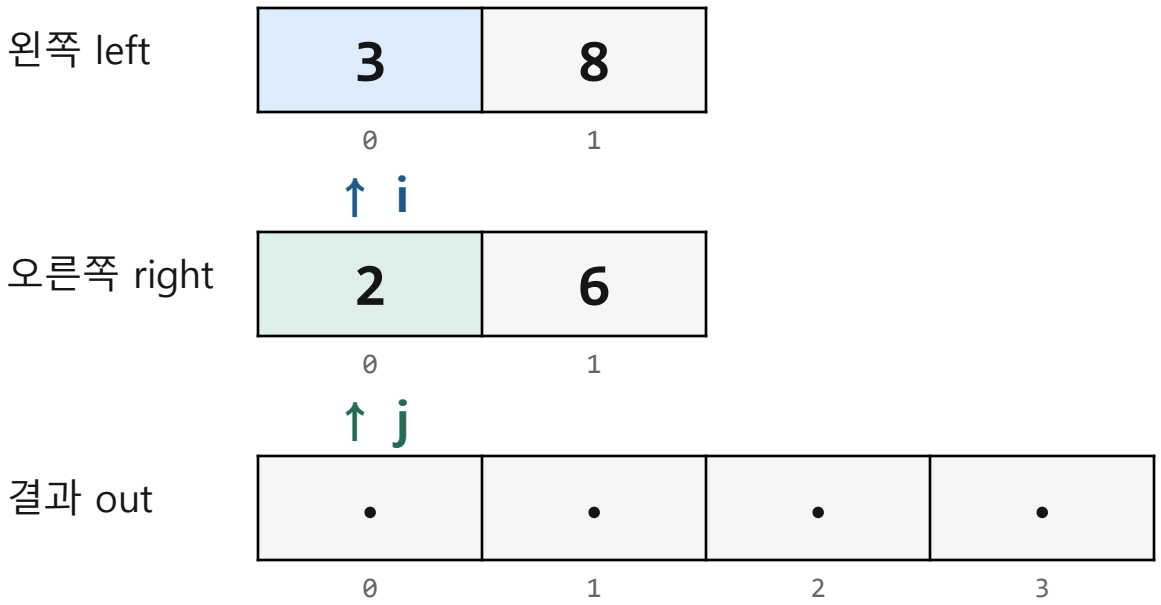
왼쪽 [3, 8]과 오른쪽 [2, 6]은 이미 정렬되어 있습니다. 맨 앞의 3과 2를 비교합니다.




2를 꺼내면 j만 앞으로 이동합니다. i는 아직 사용하지 않은 3을 계속 가리킵니다.

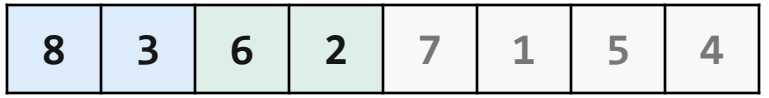
# ③ [3, 8]과 [2, 6] 병합 · 시작

i = 0, j = 0. 왼쪽 3과 오른쪽 2을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.




파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



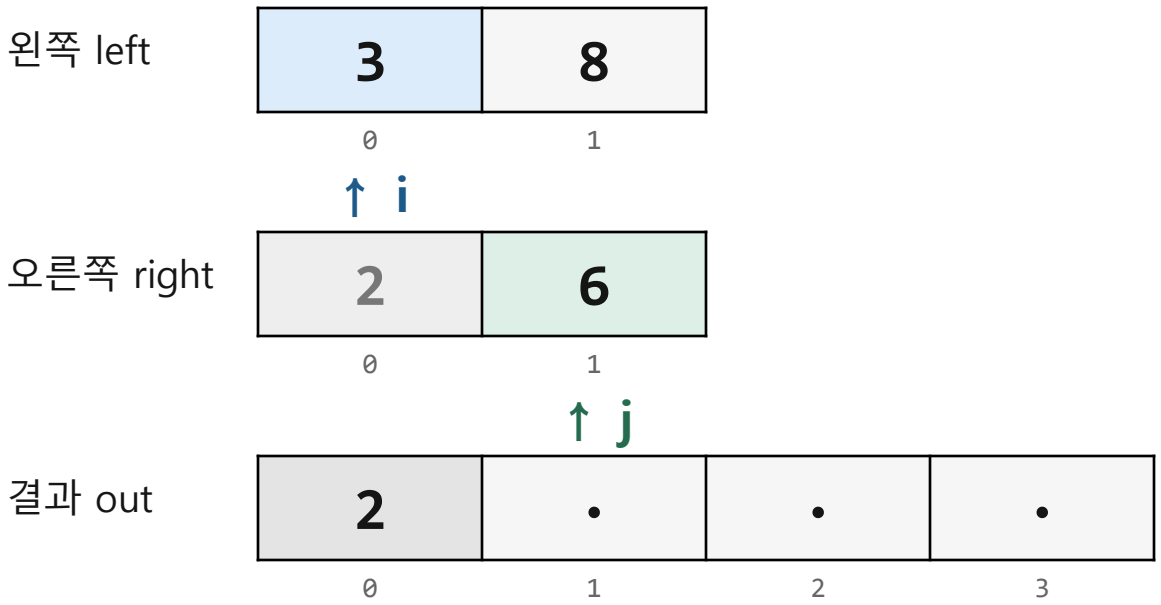

**왼쪽 4개 전체**

왼쪽: ①의 결과 오른쪽: ②의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

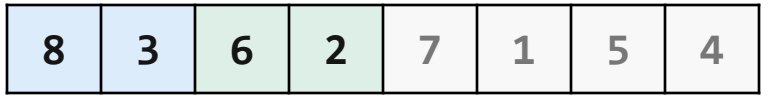
# ③ 병합 · 1번째 값 2 추가

왼쪽 3 > 오른쪽 2 → 오른쪽의 2을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.




파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



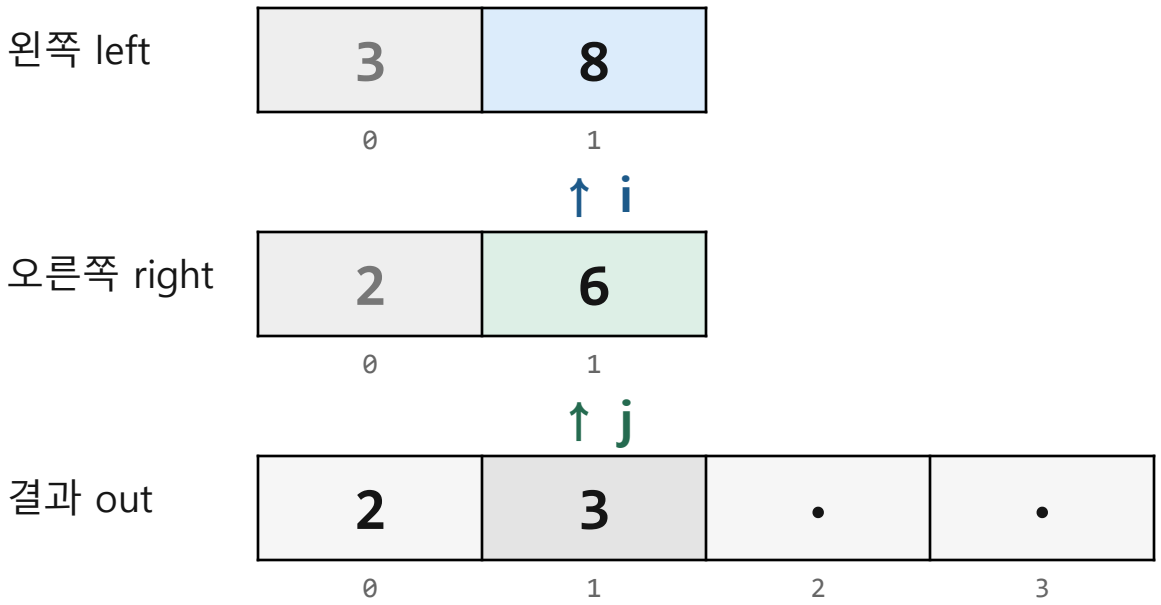

**왼쪽 4개 전체**

왼쪽: ①의 결과 오른쪽: ②의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

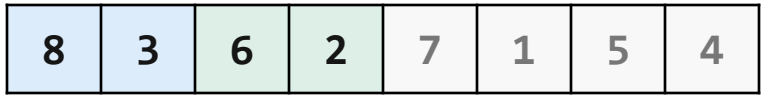
# ③ 병합 · 2번째 값 3 추가

왼쪽 3 ≤ 오른쪽 6 → 왼쪽의 3을 붙입니다.
i는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.




파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



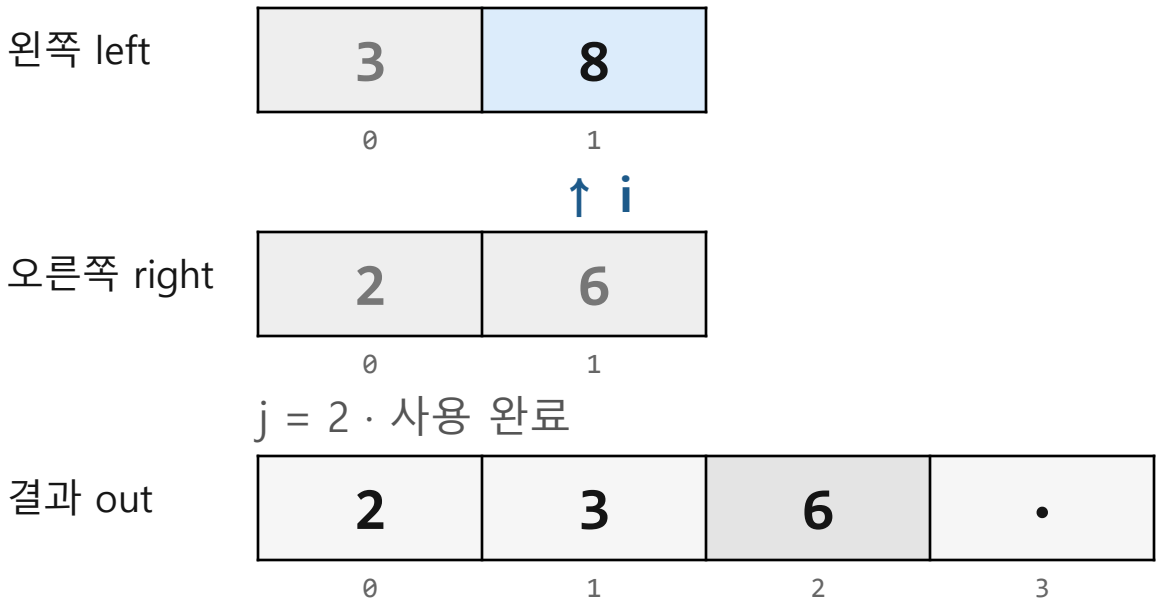

**왼쪽 4개 전체**

왼쪽: ①의 결과 오른쪽: ②의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

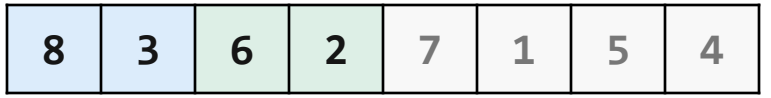
# ③ 병합 · 3번째 값 6 추가

왼쪽 8 > 오른쪽 6 → 오른쪽의 6을 붙입니다.
j는 1 → 2. 사용하지 않은 쪽의 인덱스는 그대로입니다.




파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분




**왼쪽 4개 전체**

왼쪽: ①의 결과 오른쪽: ②의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

# 한쪽을 다 쓰면 남은 값을 붙입니다

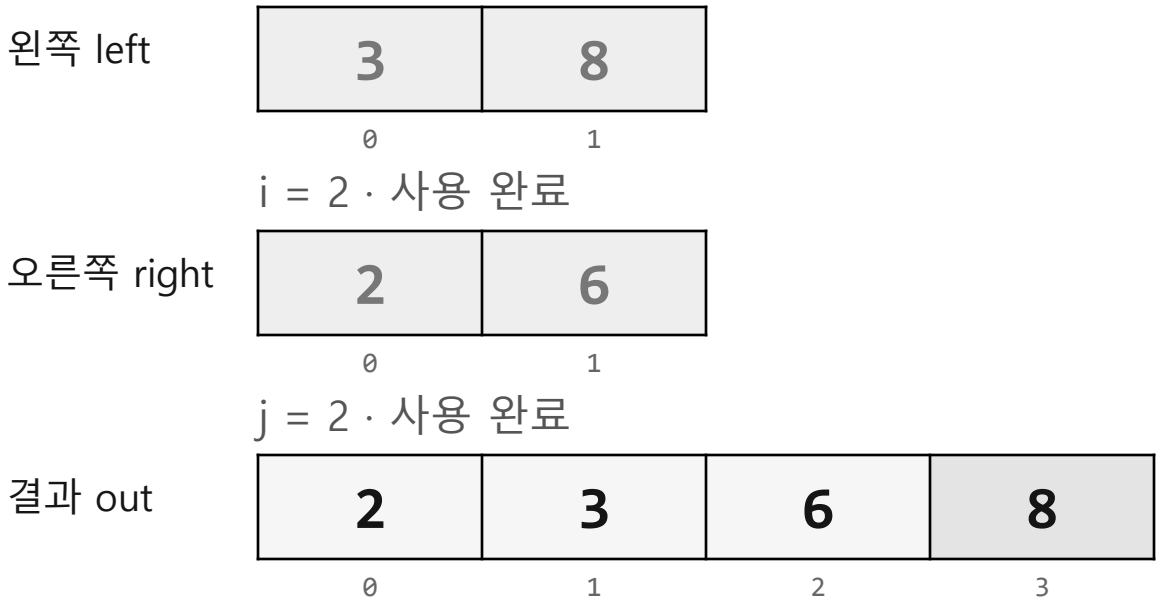
현재 결과는 [2, 3, 6]입니다. 오른쪽은 비었고, 왼쪽에는 8만 남았습니다.



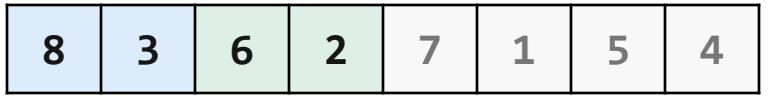

비어 있는 쪽에서는 값을 읽지 않습니다. 남은 쪽의 값을 순서대로 결과에 붙입니다.

# ③ 병합 · 4번째 값 8 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 8을 붙입니다.
i는 1 → 2. 이번 병합의 정렬 결과가 완성되었습니다.




처음 배열에서 이번에 합치는 부분




**왼쪽 4개 전체**

이번 병합의 정렬 결과

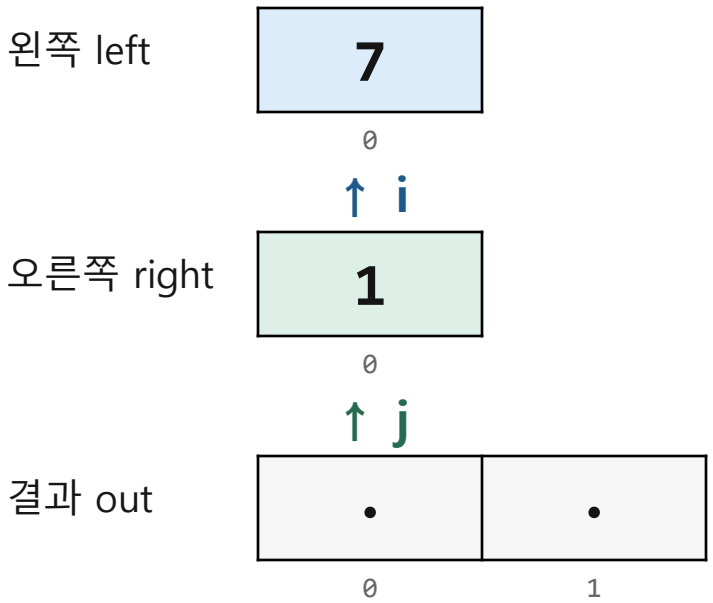
**[2, 3, 6, 8]**

왼쪽 4개 완료 → 오른쪽으로

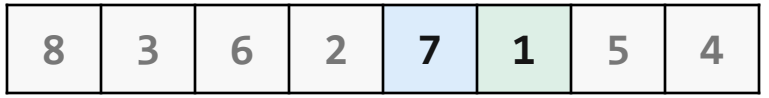
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ④ [7]과 [1] 병합 · 시작

i = 0, j = 0. 왼쪽 7과 오른쪽 1을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.




처음 배열에서 이번에 합치는 부분




**오른쪽 4개의 앞쪽 두 개**

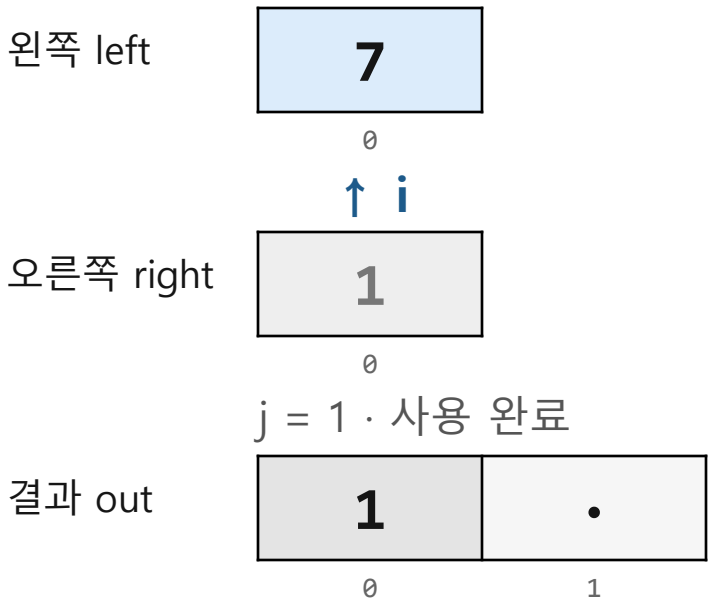
왼쪽 4개 [2,3,6,8]은 정렬 완료

선택한 쪽의 인덱스만
한 칸 움직입니다.

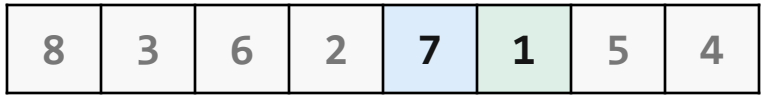
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ④ 병합 · 1번째 값 1 추가

왼쪽 7 > 오른쪽 1 → 오른쪽의 1을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.




처음 배열에서 이번에 합치는 부분




**오른쪽 4개의 앞쪽 두 개**

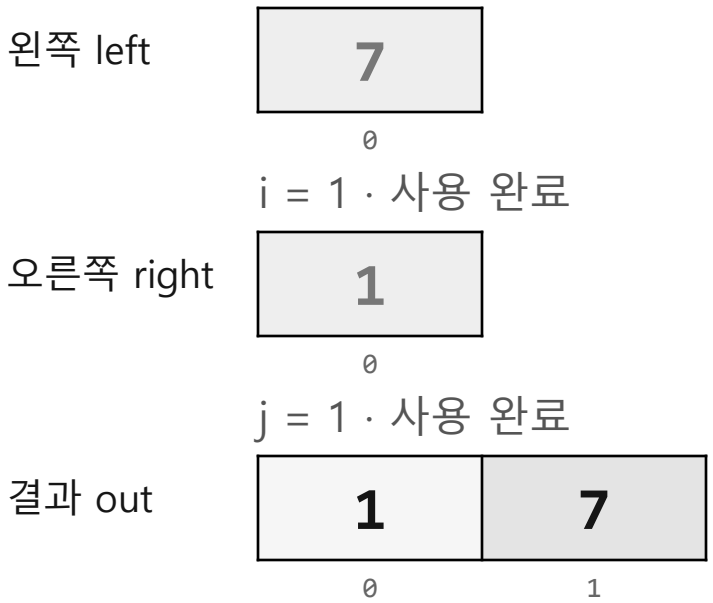
왼쪽 4개 [2,3,6,8]은 정렬 완료

선택한 쪽의 인덱스만
한 칸 움직입니다.

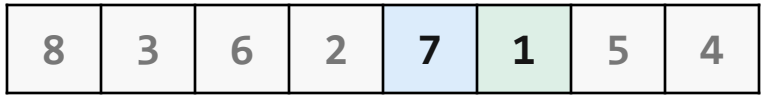
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ④ 병합 · 2번째 값 7 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 7을 붙입니다.
i는 0 → 1. 이번 병합의 정렬 결과가 완성되었습니다.




처음 배열에서 이번에 합치는 부분




**오른쪽 4개의 앞쪽 두 개**

이번 병합의 정렬 결과

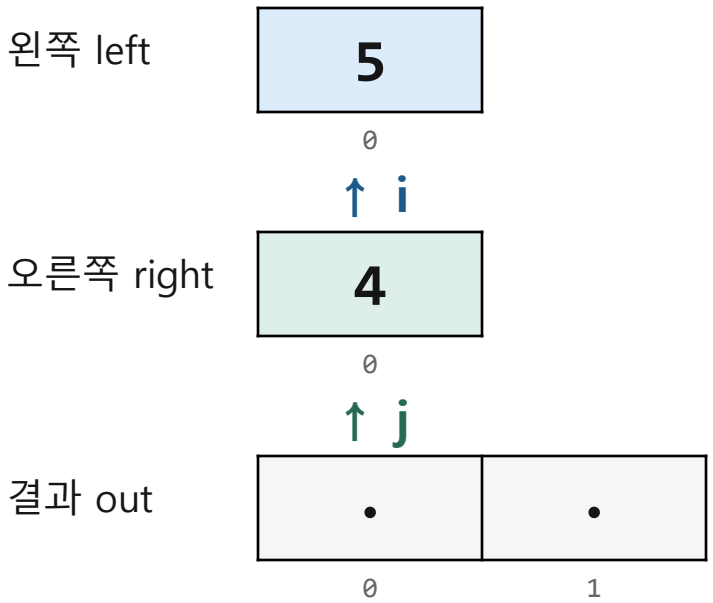
**[1, 7]**

⑥에서 왼쪽 입력으로 사용

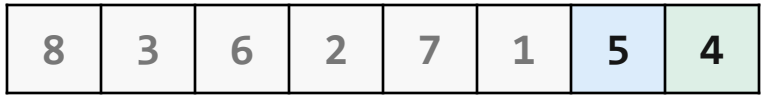
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ⑤ [5]과 [4] 병합 · 시작

i = 0, j = 0. 왼쪽 5과 오른쪽 4을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.




처음 배열에서 이번에 합치는 부분




**오른쪽 4개의 뒤쪽 두 개**

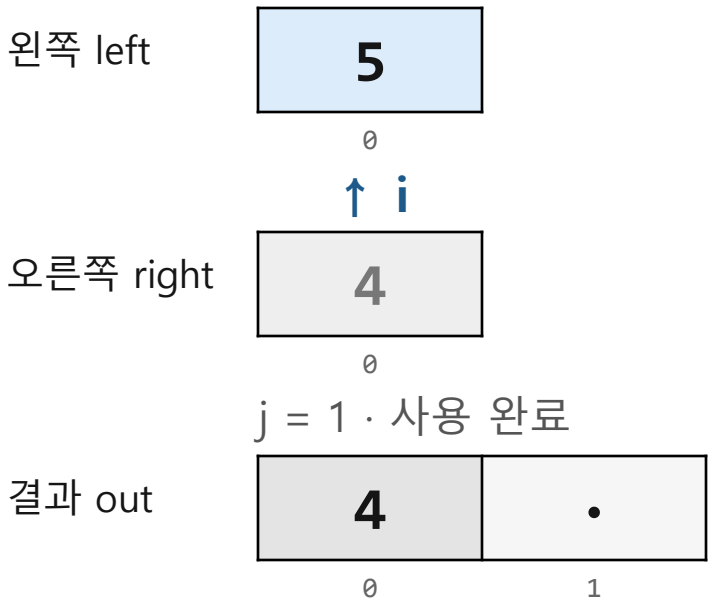
④의 [1,7]은 준비된 채 기다립니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

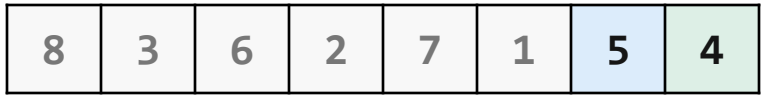
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ⑤ 병합 · 1번째 값 4 추가

왼쪽 5 > 오른쪽 4 → 오른쪽의 4을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.




처음 배열에서 이번에 합치는 부분




**오른쪽 4개의 뒤쪽 두 개**

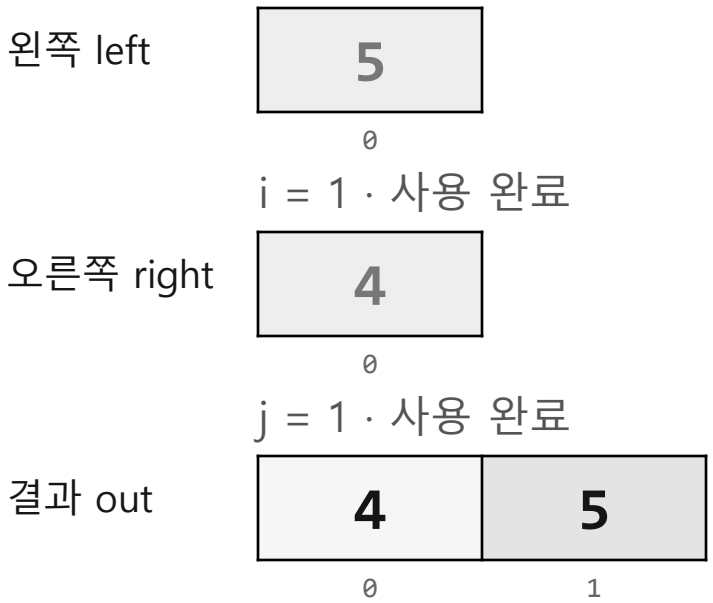
④의 [1,7]은 준비된 채 기다립니다.

선택한 쪽의 인덱스만
한 칸 움직입니다.

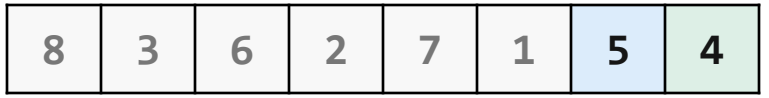
파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

# ⑤ 병합 · 2번째 값 5 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 5을 붙입니다.
i는 0 → 1. 이번 병합의 정렬 결과가 완성되었습니다.




처음 배열에서 이번에 합치는 부분




**오른쪽 4개의 뒤쪽 두 개**

이번 병합의 정렬 결과

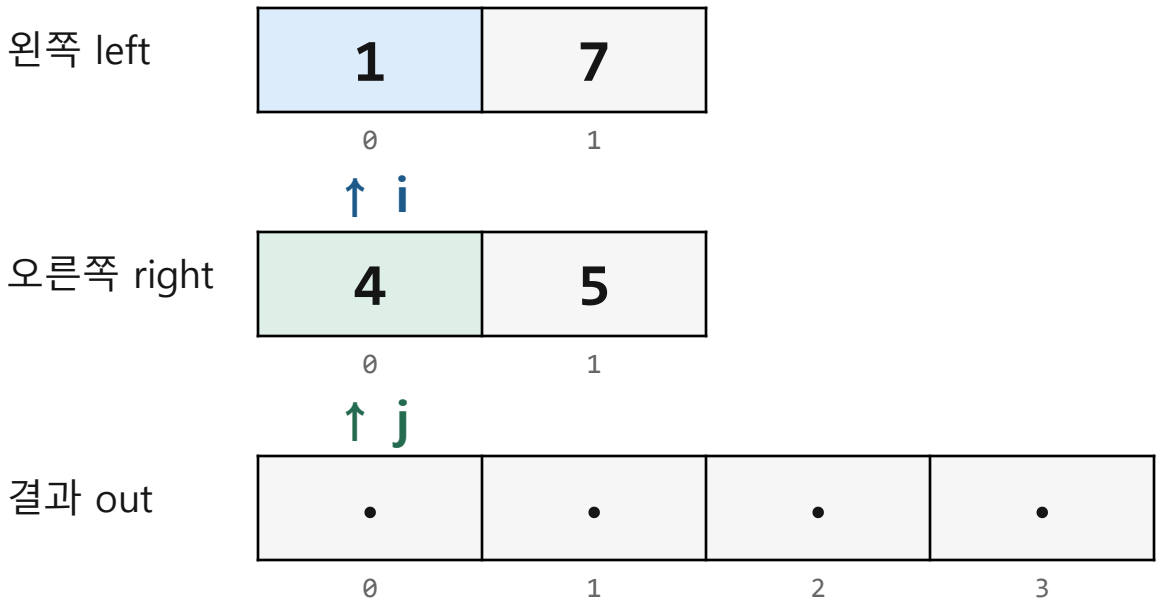
**[4, 5]**

⑥에서 오른쪽 입력으로 사용

파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

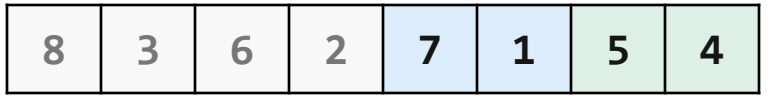
# ⑥ [1, 7]과 [4, 5] 병합 · 시작

i = 0, j = 0. 왼쪽 1과 오른쪽 4을 비교합니다.
정렬된 두 결과를 받아 작은 값부터 새 결과에 넣습니다.




파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



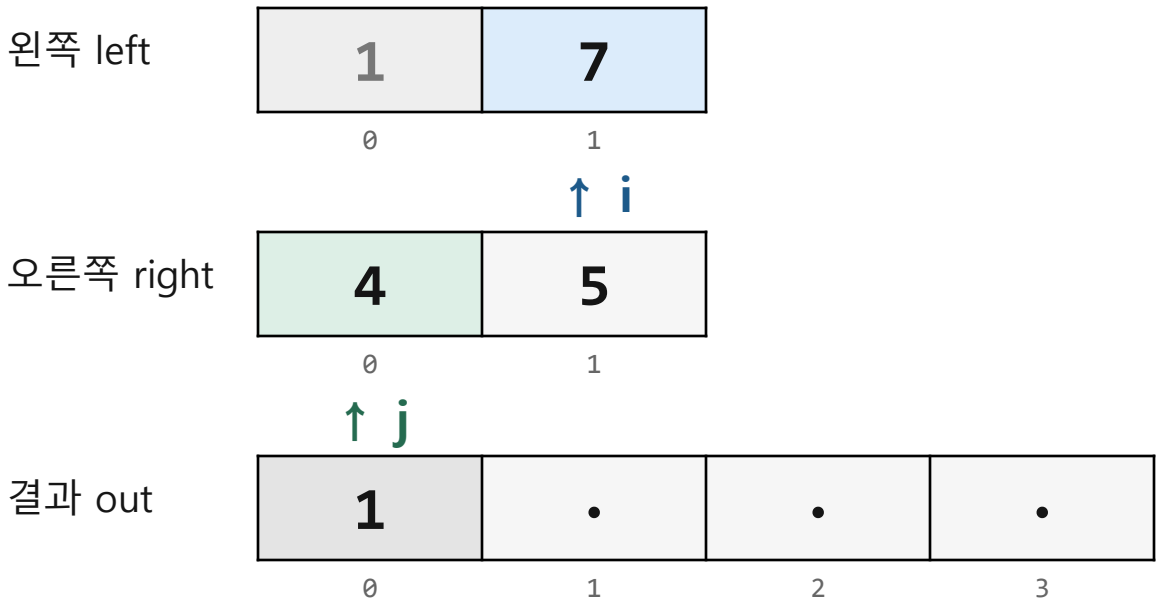

**오른쪽 4개 전체**

왼쪽: ④의 결과 오른쪽: ⑤의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

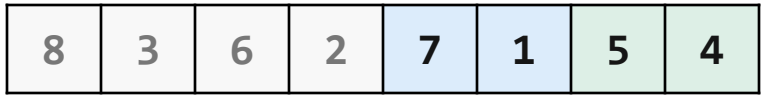
# ⑥ 병합 · 1번째 값 1 추가

왼쪽 1 ≤ 오른쪽 4 → 왼쪽의 1을 붙입니다.
i는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.




파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



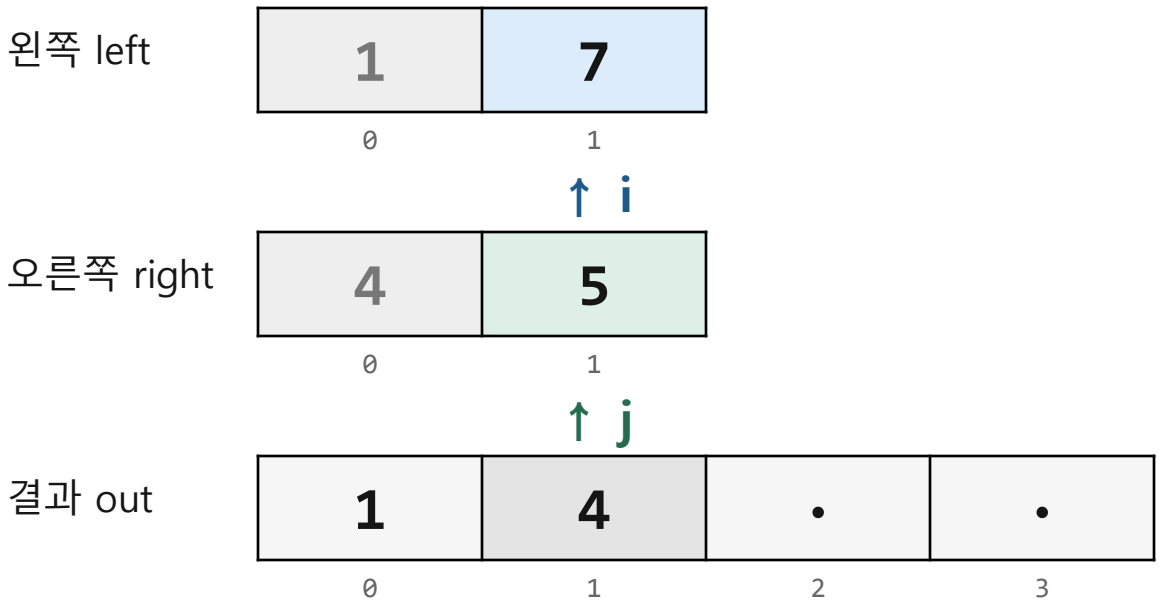

**오른쪽 4개 전체**

왼쪽: ④의 결과 오른쪽: ⑤의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

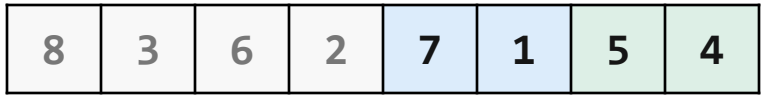
# ⑥ 병합 · 2번째 값 4 추가

왼쪽 7 > 오른쪽 4 → 오른쪽의 4을 붙입니다.
j는 0 → 1. 사용하지 않은 쪽의 인덱스는 그대로입니다.




파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



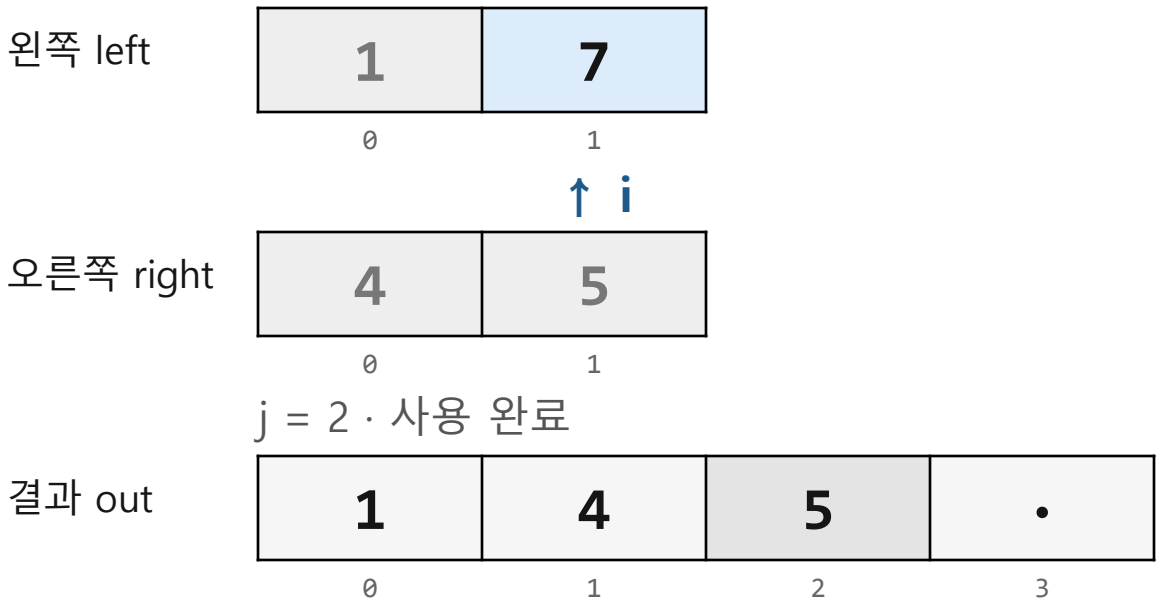

**오른쪽 4개 전체**

왼쪽: ④의 결과 오른쪽: ⑤의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

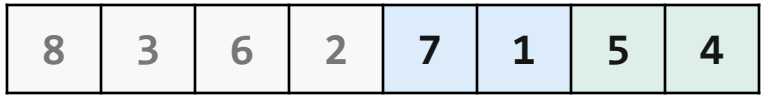
# ⑥ 병합 · 3번째 값 5 추가

왼쪽 7 > 오른쪽 5 → 오른쪽의 5을 붙입니다.
j는 1 → 2. 사용하지 않은 쪽의 인덱스는 그대로입니다.




파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분



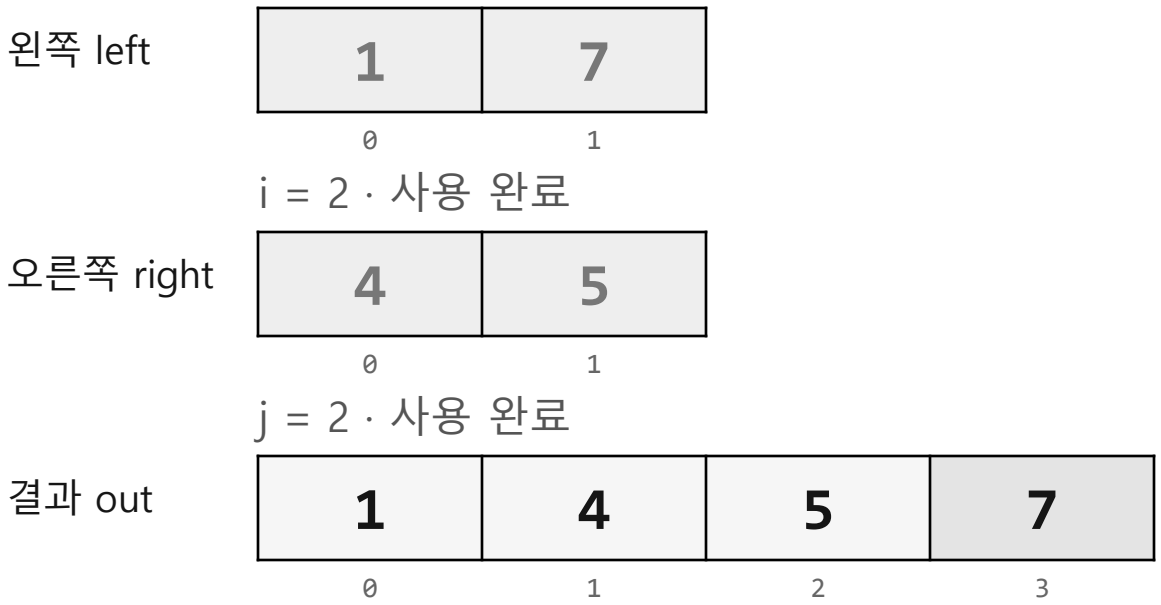

**오른쪽 4개 전체**

왼쪽: ④의 결과 오른쪽: ⑤의 결과

선택한 쪽의 인덱스만
한 칸 움직입니다.

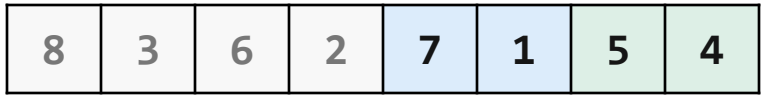
# ⑥ 병합 · 4번째 값 7 추가

오른쪽을 다 사용했습니다. 남은 왼쪽의 7을 붙입니다.
i는 1 → 2. 이번 병합의 정렬 결과가 완성되었습니다.




파랑 i: 왼쪽의 다음 값 초록 j: 오른쪽의 다음 값 회색: 사용한 값

처음 배열에서 이번에 합치는 부분




**오른쪽 4개 전체**

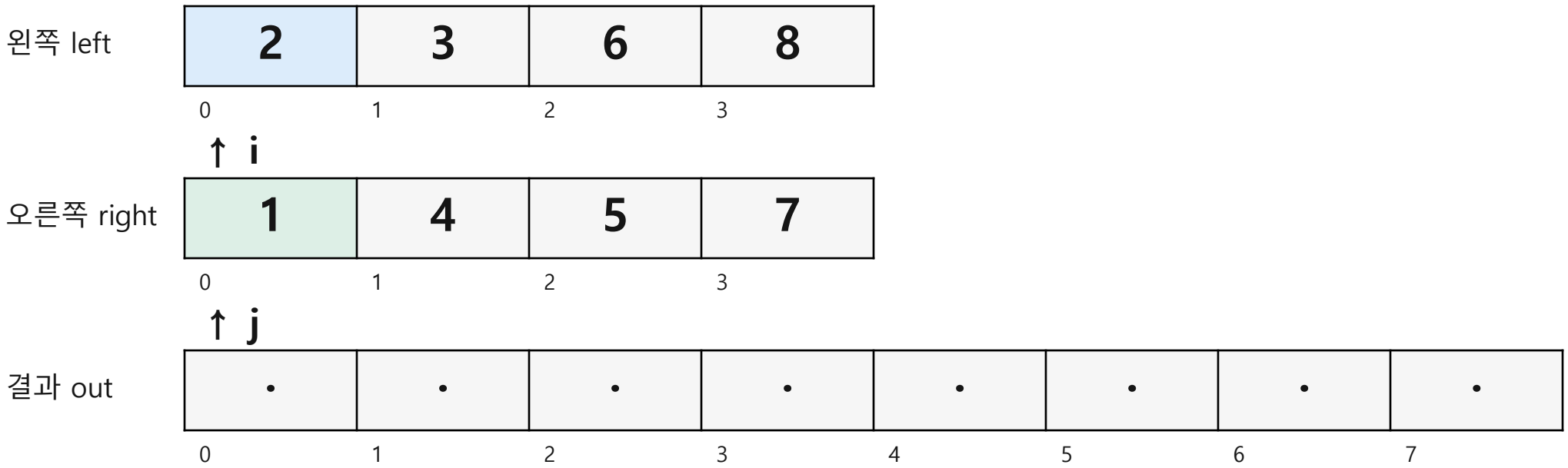
이번 병합의 정렬 결과

**[1, 4, 5, 7]**

⑦에서 오른쪽 입력으로 사용

# 병합 시작 · 두 화살표가 첫 칸

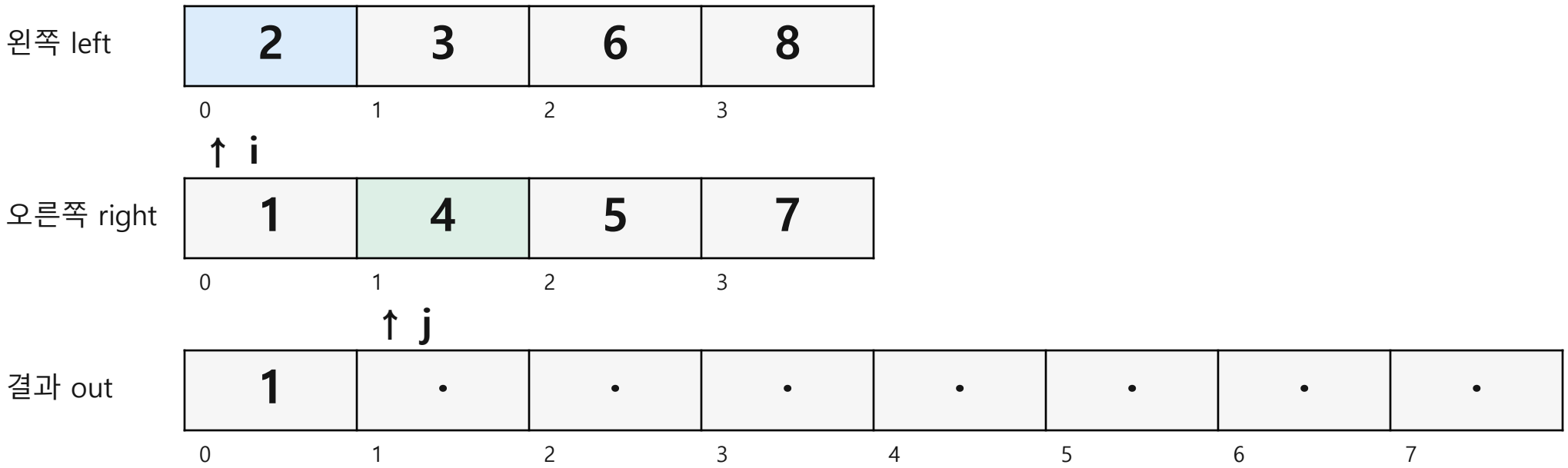
i = 0, j = 0입니다. 왼쪽 2와 오른쪽 1을 비교합니다.
결과 배열은 아직 비어 있습니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# 병합 1단계 · 1을 붙인 뒤

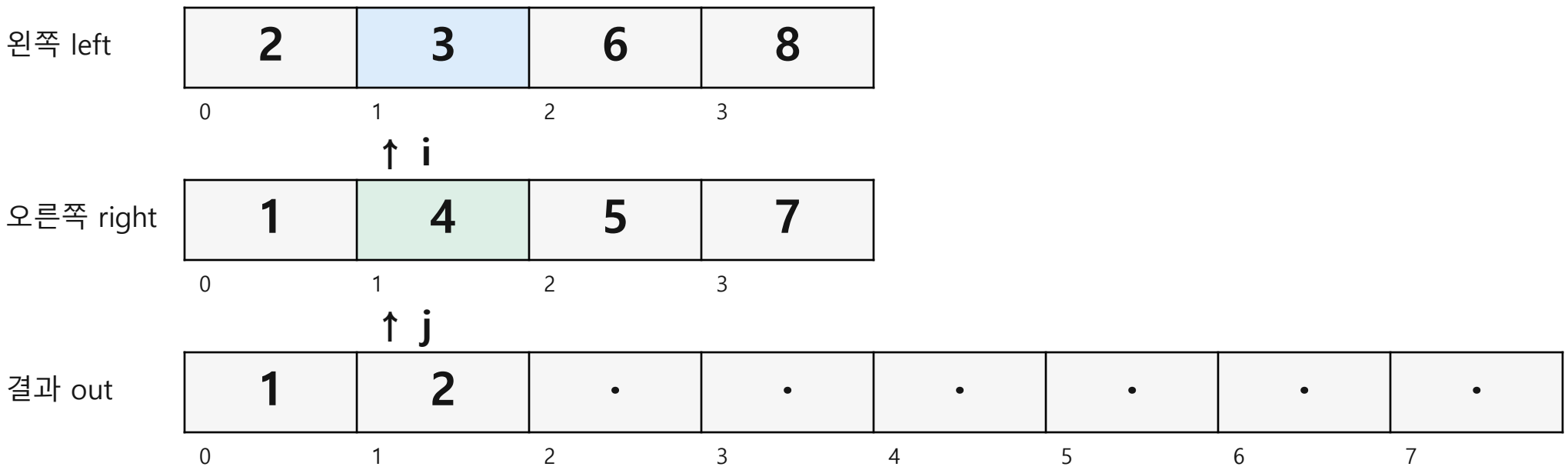
왼쪽 2 > 오른쪽 1. 오른쪽 값을 결과 끝에 붙입니다.
j를 0에서 1로 옮깁니다. i = 0는 그대로입니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# 병합 2단계 · 2을 붙인 뒤

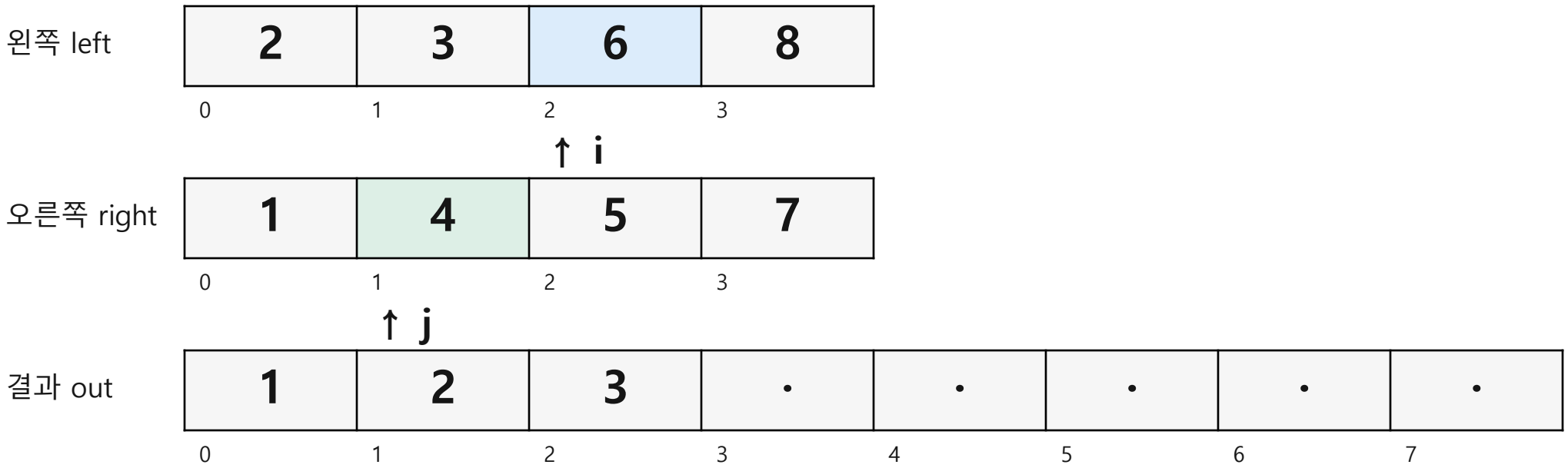
왼쪽 2 ≤ 오른쪽 4. 왼쪽 값을 결과 끝에 붙입니다.
i를 0에서 1로 옮깁니다. j = 1는 그대로입니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# 병합 3단계 · 3을 붙인 뒤

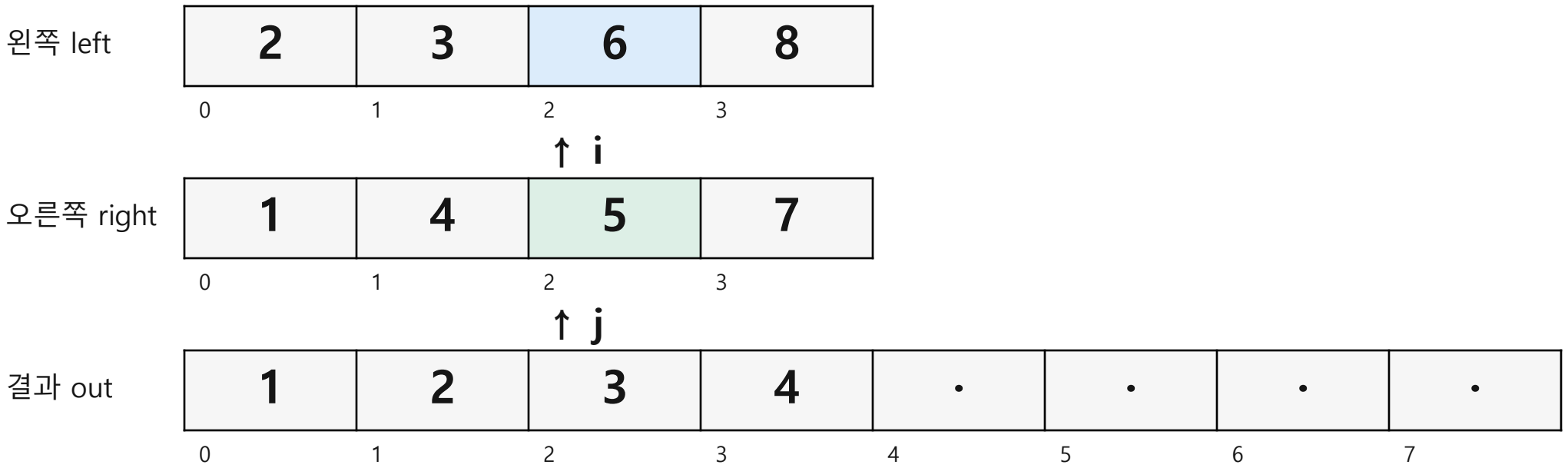
왼쪽 3 ≤ 오른쪽 4. 왼쪽 값을 결과 끝에 붙입니다.
i를 1에서 2로 옮깁니다. j = 1는 그대로입니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# 병합 4단계 · 4을 붙인 뒤

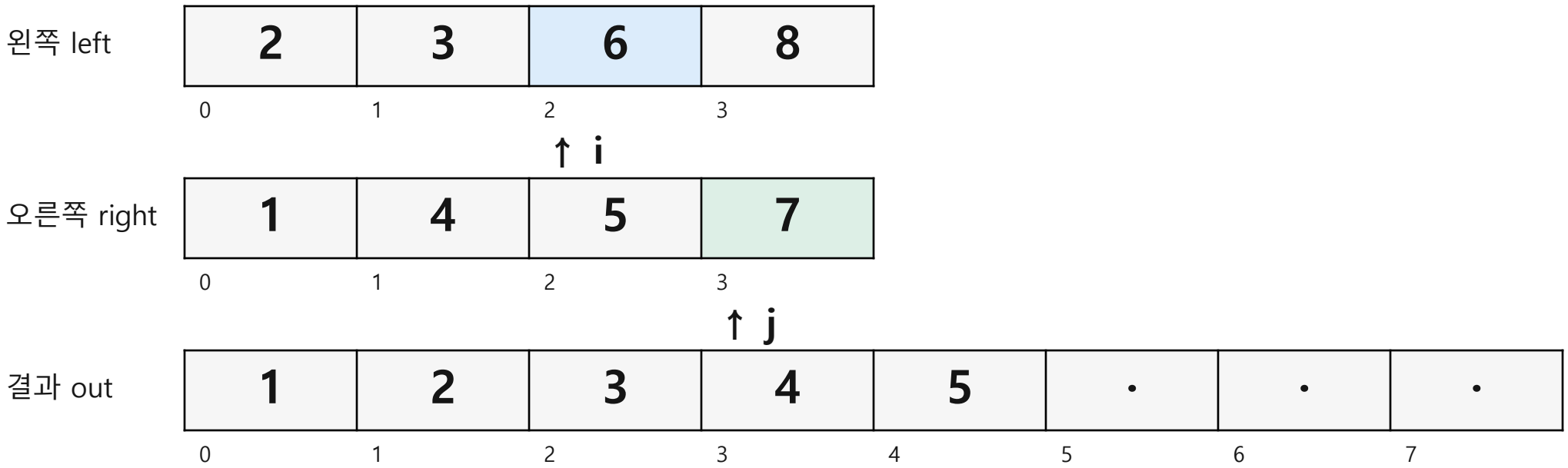
왼쪽 6 > 오른쪽 4. 오른쪽 값을 결과 끝에 붙입니다.
j를 1에서 2로 옮깁니다. i = 2는 그대로입니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# 병합 5단계 · 5을 붙인 뒤

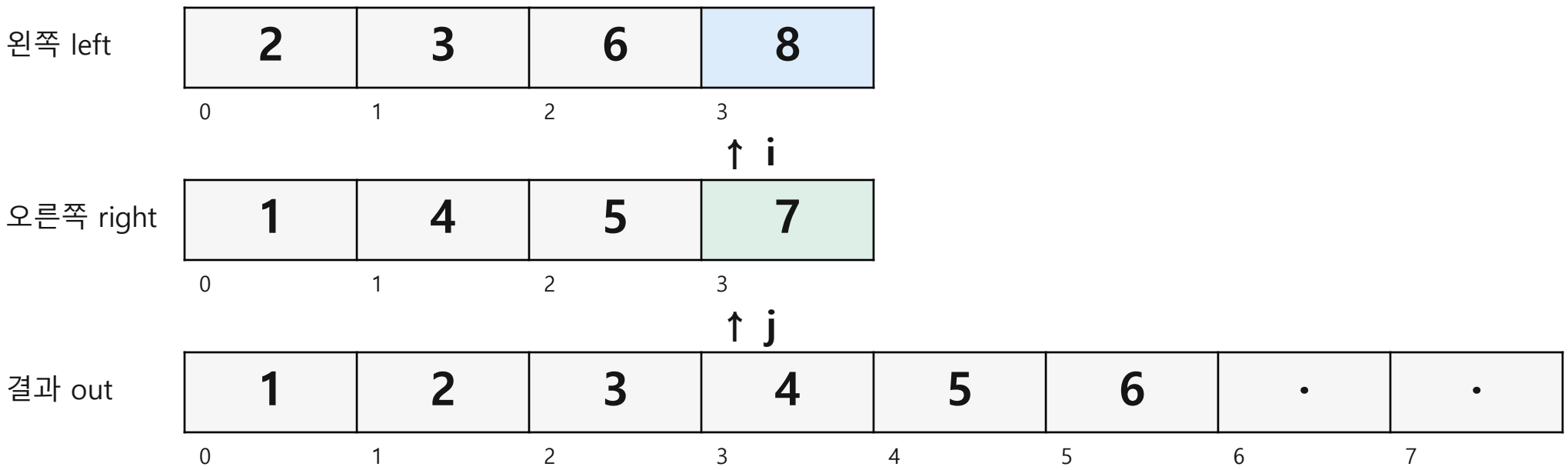
왼쪽 6 > 오른쪽 5. 오른쪽 값을 결과 끝에 붙입니다.
j를 2에서 3로 옮깁니다. i = 2는 그대로입니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# 병합 6단계 · 6을 붙인 뒤

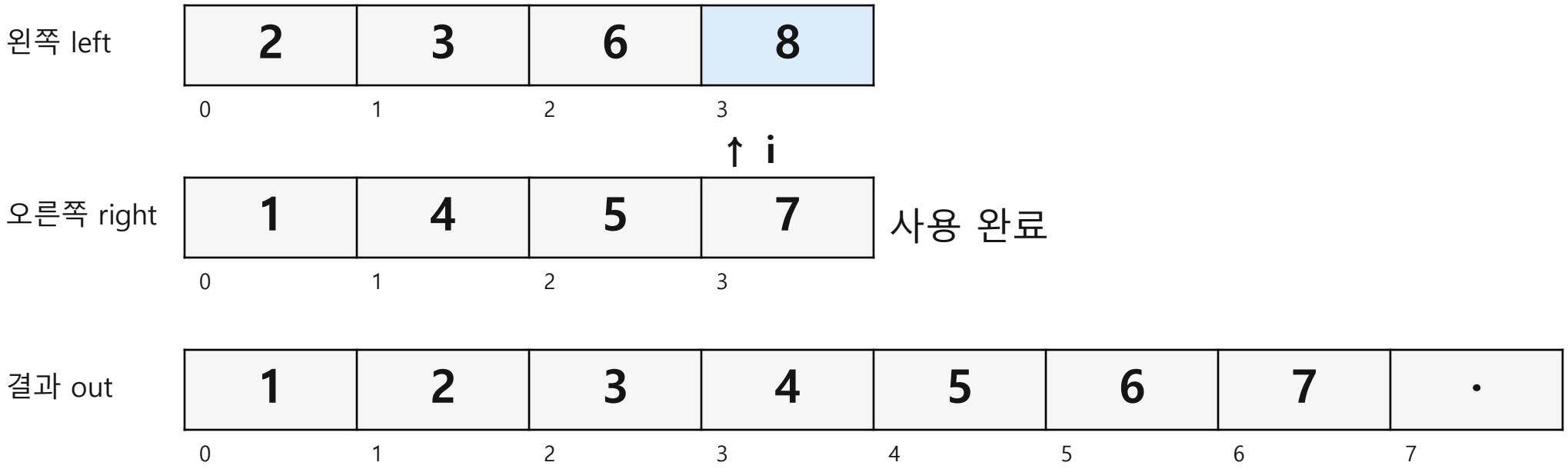
왼쪽 6 ≤ 오른쪽 7. 왼쪽 값을 결과 끝에 붙입니다.
i를 2에서 3로 옮깁니다. j = 3는 그대로입니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# 병합 7단계 · 7을 붙인 뒤

왼쪽 8 > 오른쪽 7. 오른쪽 값을 결과 끝에 붙입니다.
j를 3에서 4로 옮깁니다. i = 3는 그대로입니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# 병합 8단계 · 8을 붙인 뒤

오른쪽을 다 사용했습니다. 왼쪽의 8을 결과 끝에 붙입니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# Python: 병합 함수

```
def merge(left, right):
    out = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    while i < len(left):
        out.append(left[i])
        i += 1
    while j < len(right):
        out.append(right[j])
        j += 1
    return out
```

# Python: 재귀 호출 순서

```
def merge_sort(a):
    if len(a) <= 1:
        return a[:]
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

a = [8, 3, 6, 2, 7, 1, 5, 4]
print(merge_sort(a))
```

# C: 정렬된 두 구간의 병합

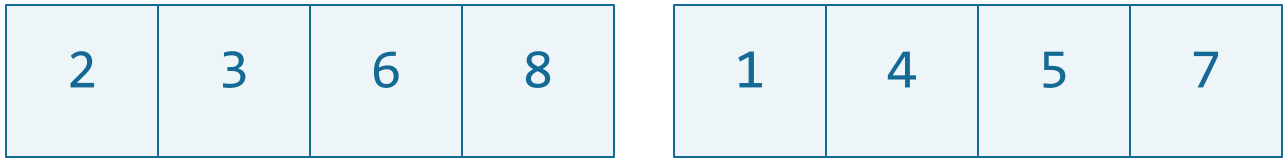
```
void merge(int a[], int tmp[], int lo, int mid, int hi) {
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else tmp[k++] = a[j++];
    }
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= hi) tmp[k++] = a[j++];
    for (int t = lo; t <= hi; t++) a[t] = tmp[t];
}
```

# C: 재귀 호출과 실행

```
/* 앞 슬라이드의 merge 함수를 함께 넣는다. */
void merge_sort(int a[], int tmp[], int lo, int hi) {
    if (lo >= hi) return;
    int mid = lo + (hi - lo) / 2;
    merge_sort(a, tmp, lo, mid);
    merge_sort(a, tmp, mid + 1, hi);
    merge(a, tmp, lo, mid, hi);
}
/* 실행 예: main 안에서 */
int a[] = {8, 3, 6, 2, 7, 1, 5, 4};
int tmp[8];
merge_sort(a, tmp, 0, 7);
/* a는 {1, 2, 3, 4, 5, 6, 7, 8}이 된다. */
```

# 병합 한 번: 값 8개를 결과에 8번 넣습니다

## 이미 정렬된 두 입력




두 줄에서 작은 값부터 하나씩 선택

## 결과




**4개 + 4개 = 8개를 한 번씩 처리**

병합 한 번의 작업량은 전체 길이에 비례합니다. 길이가 n이면 O(n)입니다.

# 병합 한 번: 세 while문의 합은 선형 시간

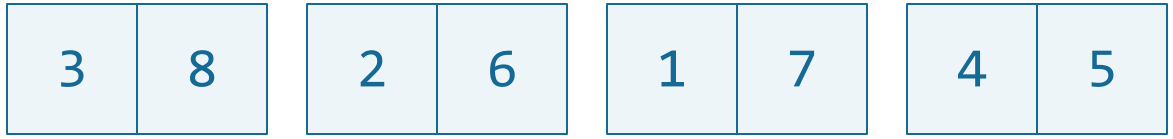
| 반복문 | 하는 일 | 진행하는 인덱스 |
|---|---|---|
| 양쪽을 비교하는 while | 작은 값 하나를 결과에 넣음 | i 또는 j가 1 증가 |
| 왼쪽이 남은 while | 남은 왼쪽 값을 순서대로 넣음 | i가 1 증가 |
| 오른쪽이 남은 while | 남은 오른쪽 값을 순서대로 넣음 | j가 1 증가 |

## 세 반복문의 출력 횟수 합 = p + q = n

값 하나는 결과에 한 번만 들어갑니다. 병합 한 번의 시간은 O(n)입니다.

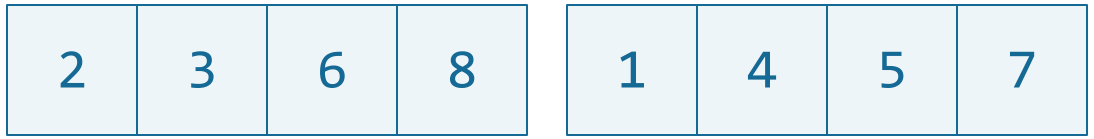
# 전체 병합: 한 층에서 다루는 값은 8개

**첫 병합 층**




**2 × 4 = 8**

**두 번째 층**




**4 × 2 = 8**

**마지막 층**

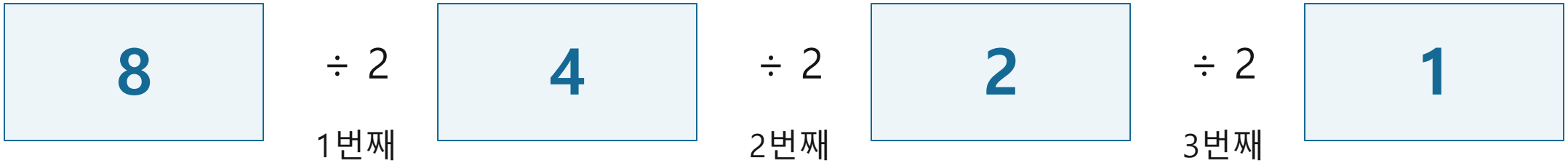



**8 × 1 = 8**

**8 + 8 + 8 = 24번**

각 값이 결과에 들어가는 횟수를 셉니다. 한 원소에서 바로 반환하는 비용은 별도로 있습니다.

# $\log_2 8 = 3$: 반으로 나누는 횟수가 3번




## 8에서 1까지 줄이는 횟수: 3번

$$\log_2 8 = 3$$

$\log_2 16 = 4$, $\log_2 32 = 5$. 원소 수를 두 배로 늘리면 병합 층이 하나 늘어납니다.

# n log n: 한 층의 작업량에 층 수를 곱합니다

| 원소 수 | 병합 층 수 | 결과에 넣는 횟수 |
|---|---|---|
| 8개 | 3층 | 8 × 3 = 24 |
| 16개 | 4층 | 16 × 4 = 64 |
| 32개 | 5층 | 32 × 5 = 160 |

## 원소 수 n × 층 수 $\log_2 n$

병합 정렬의 전체 시간은 O(n log n). O는 입력이 커질 때 작업량이 늘어나는 정도를 나타냅니다.

# T(n): 원소 n개를 정렬하는 데 드는 시간

**8개 정렬의 시간 = 왼쪽 4개 + 오른쪽 4개 + 병합**

| T(8) | = | T(4) | + | T(4) | + | 병합 비용 |
|---|---|---|---|---|---|---|
| 전체 시간 | | 왼쪽 절반의 시간 | | 오른쪽 절반의 시간 | | 현재 구간의 작업 |

원소 수를 n으로 바꾸면

$$T(n) = 2T(n/2) + O(n)$$

2T(n/2)는 절반 크기의 문제 두 개를 풀 시간입니다. O(n)은 그 두 결과를 합칠 시간입니다.

# 병합 정렬 확인

왼쪽 [2, 5], 오른쪽 [1, 3, 7]을 병합하자.
① 첫 번째로 선택하는 값과 움직이는 포인터는?
② 왼쪽이 먼저 끝났을 때 어떤 값을 붙이는가?
③ 재귀 코드에서 merge를 두 재귀 호출 뒤에 쓰는 이유는?

왼쪽

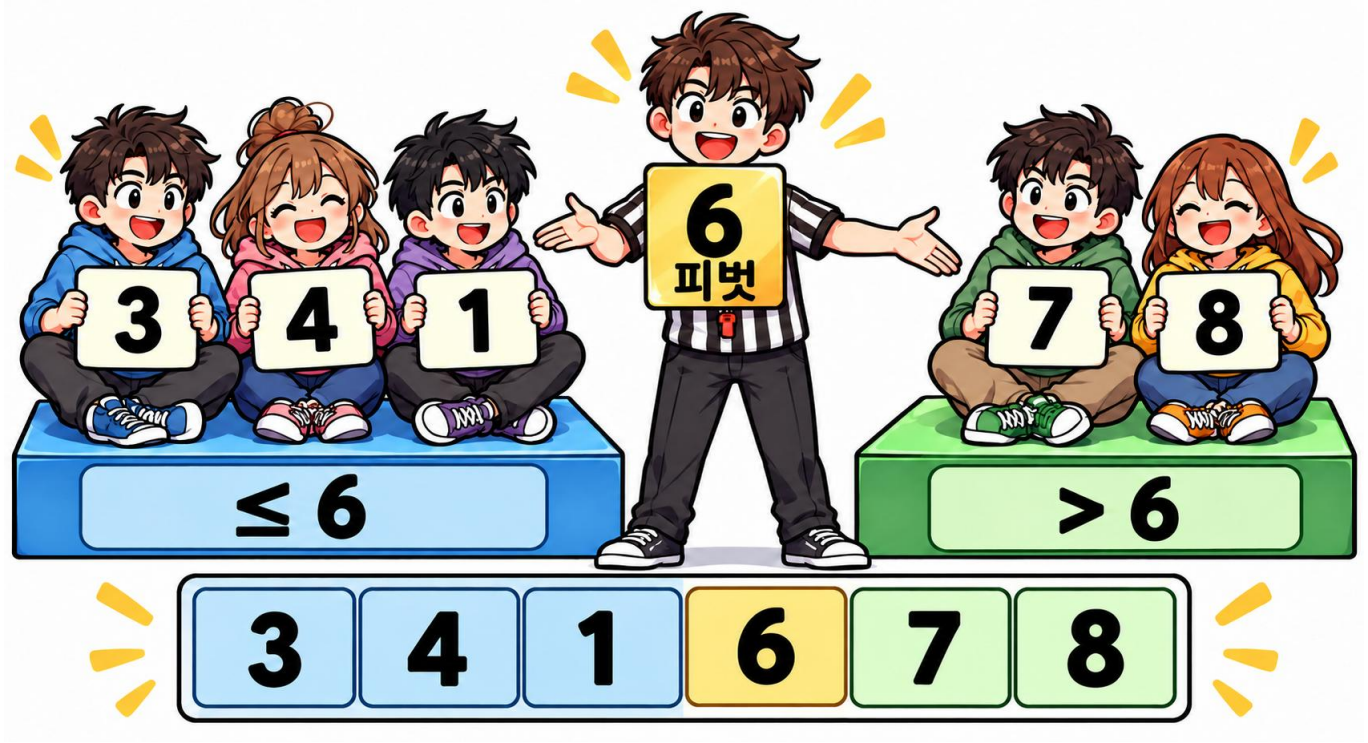
| 2 | 5 |
|---|---|

오른쪽

| 1 | 3 | 7 |
|---|---|---|

# 퀵 정렬: 피벗을 기준으로 구간 나누기

입력은 [8, 4, 6, 3, 7, 1]이고, 이번 비교 기준값인 피벗은 6입니다.
이 구현은 6 이하를 왼쪽에, 6보다 큰 값을 오른쪽에 모읍니다.

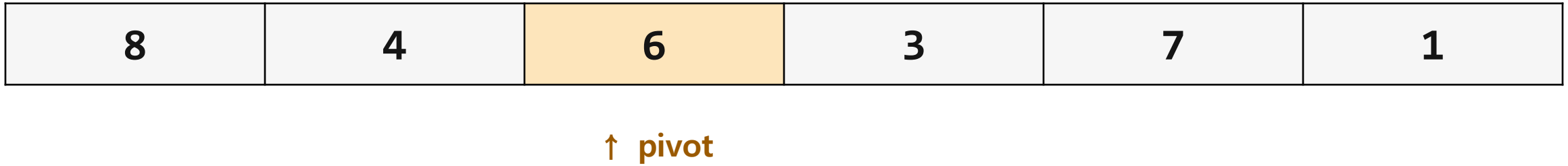



분할 직후에는 양쪽 구간이 아직 정렬되지 않을 수 있습니다. 각 구간을 다시 정렬합니다.

# 퀵 정렬 (Quick Sort)

피벗은 원소들을 나누는 비교 기준값입니다.
이번 구현은 피벗 이하를 왼쪽, 피벗 초과를 오른쪽에 모읍니다.

이번 입력




분할 뒤 피벗의 자리를 정하고, 피벗 양쪽 구간을 다시 정렬합니다.

# 피벗 선택: 가운데 칸의 6

현재 구간은 lo = 0부터 hi = 5까지입니다.
가운데 인덱스 (lo + hi) // 2 = 2에서 피벗 a[2] = 6을 고릅니다.



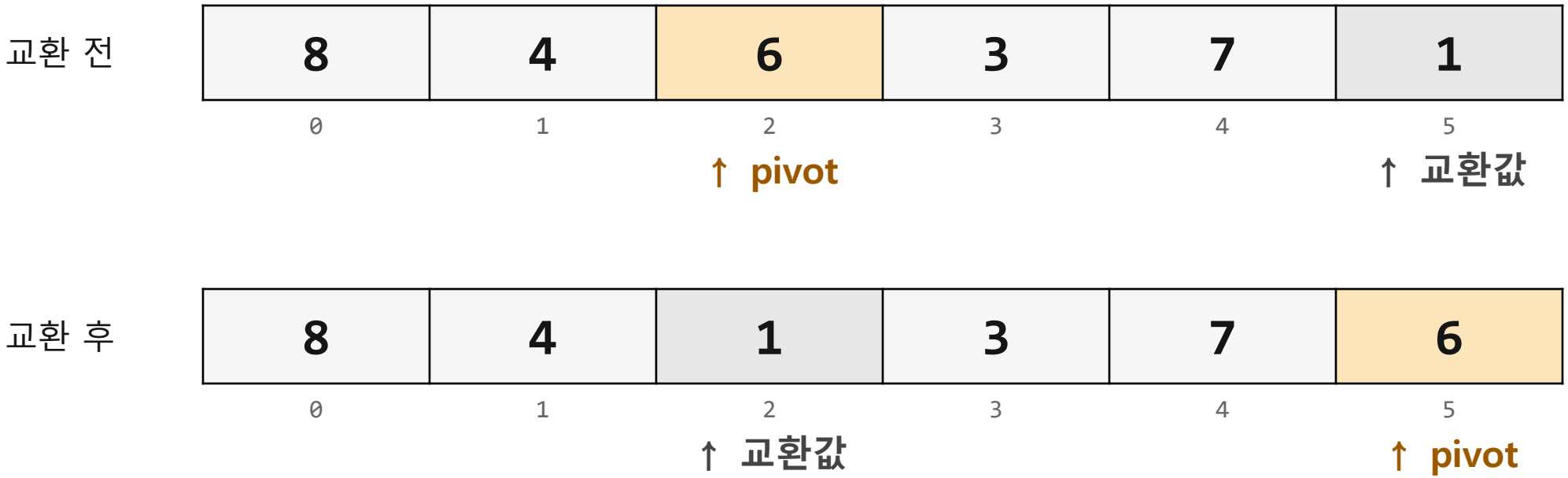

```
pivot_idx = (lo + hi) // 2
```

이번 코드가 사용하는 피벗 선택 규칙입니다.

# 피벗 6과 마지막 값 1의 교환

피벗 a[2] = 6과 마지막 값 a[5] = 1을 교환합니다.
6은 인덱스 5로, 1은 인덱스 2로 이동합니다.

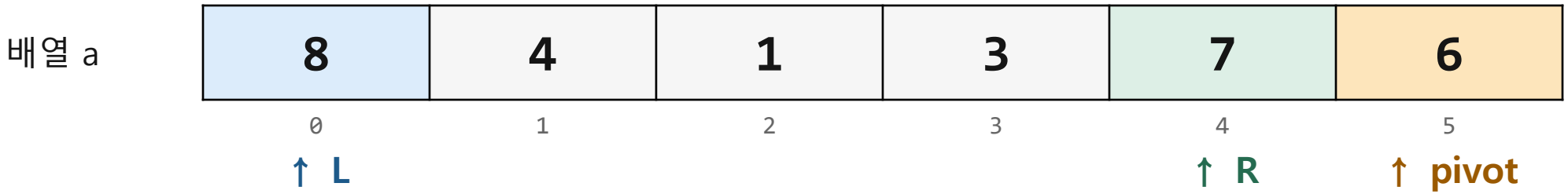
```
a[pivot_idx], a[hi] = a[hi], a[pivot_idx]
```




끝의 6은 검사 동안 임시로 두는 값입니다. 최종 위치는 분할이 끝난 뒤 정합니다.

# ① 탐색 시작: L = 0, R = 4

L = 0, R = 4에서 시작합니다.
마지막 칸의 피벗 6을 제외하고, 인덱스 0~4를 검사합니다.
첫 비교에서 8 > 6이므로 L은 인덱스 0에서 멈춥니다.

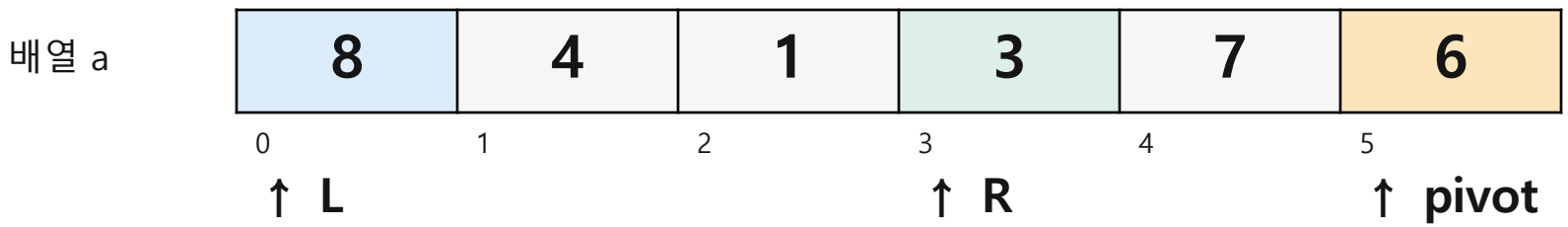



```
pivot = a[hi]
L, R = lo, hi - 1
```

다음: R이 7을 지나, 왼쪽으로 보낼 값 3을 찾습니다.

# ② R이 4에서 3으로 이동

a[4] = 7 > 6이므로 R을 왼쪽으로 한 칸 옮깁니다.
a[3] = 3 ≤ 6이므로 여기서 멈춥니다. L은 0에 그대로 있습니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# 두 포인터가 찾은 8과 3의 교환

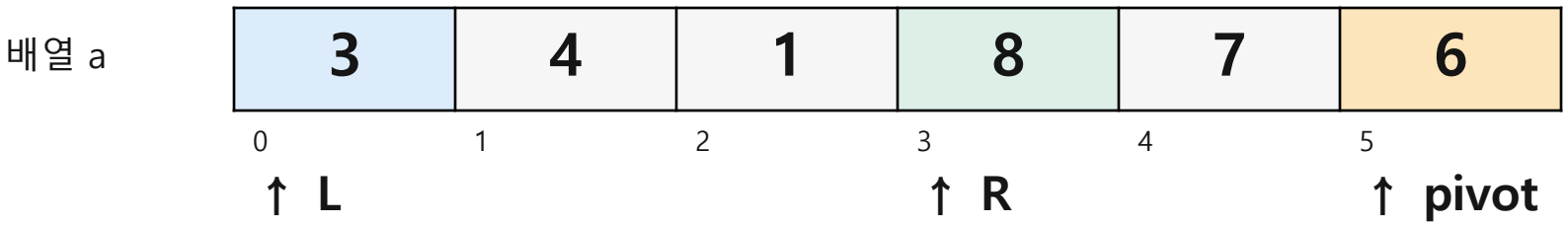
L = 0의 8은 오른쪽으로, R = 3의 3은 왼쪽으로 보내야 합니다. 피벗은 끝의 6입니다.




이번에는 두 값만 맞바꿉니다. L = 0, R = 3은 다음 탐색에서 이동합니다.

# ③ 두 포인터가 가리킨 8과 3 교환

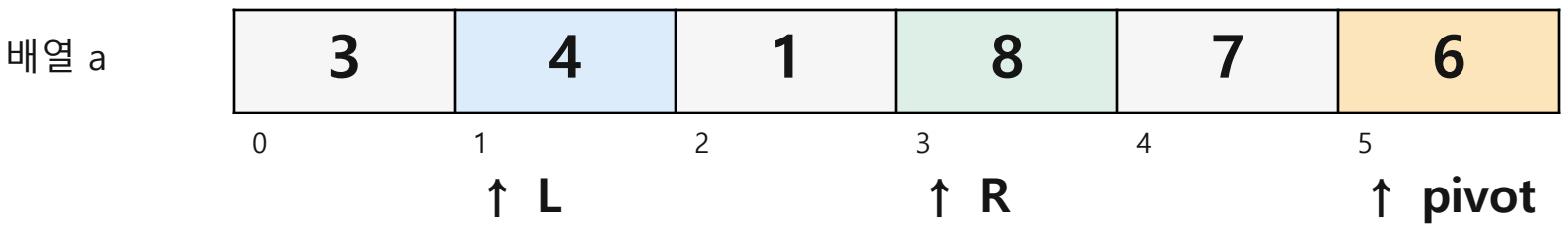
L = 0, R = 3이고 아직 L ≤ R입니다.
a[0]과 a[3]을 교환합니다. 포인터 위치는 그대로입니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# ④ L이 0에서 1로 이동

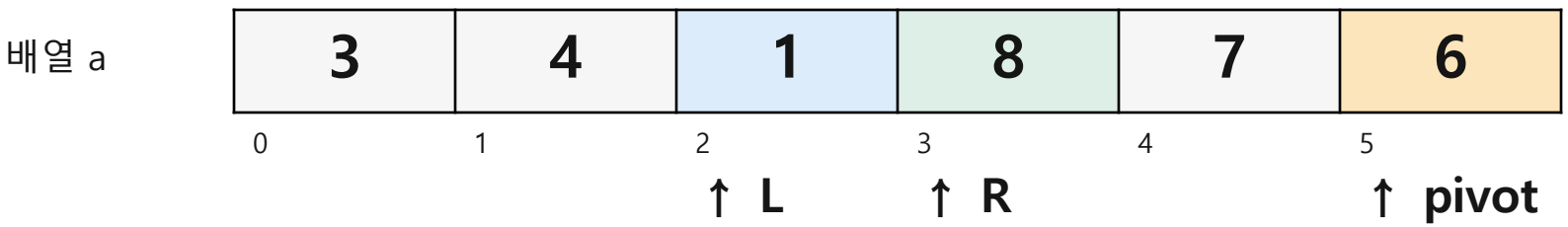
a[0] = 3 ≤ 6이므로 L을 오른쪽으로 한 칸 옮깁니다.
R은 인덱스 3에 그대로 있습니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# ⑤ L이 1에서 2로 이동

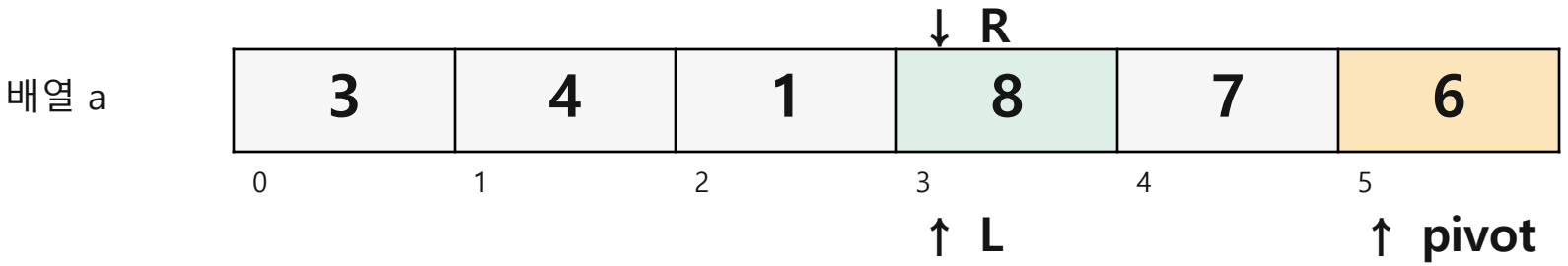
a[1] = 4 ≤ 6이므로 L을 다시 한 칸 옮깁니다.
지금 L이 가리키는 값은 a[2] = 1입니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# ⑥ L이 2에서 3으로 이동

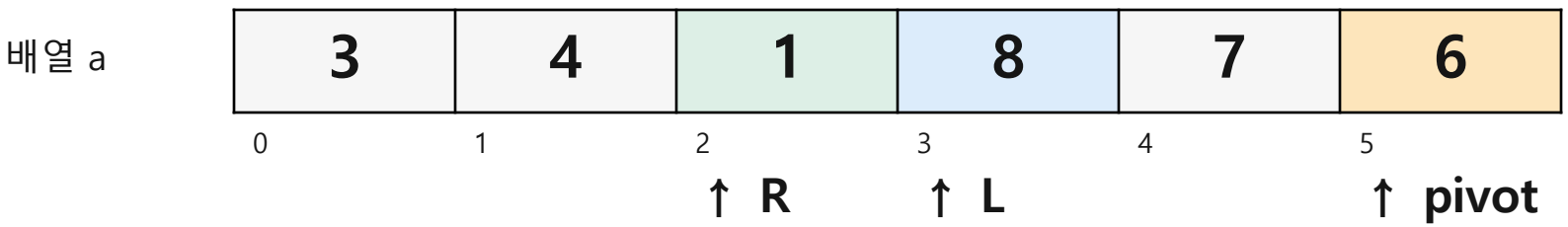
a[2] = 1 ≤ 6이므로 L을 한 칸 옮깁니다.
a[3] = 8 > 6이므로 L의 이동은 여기서 끝납니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# ⑦ R이 3에서 2로 이동 · 교차

a[3] = 8 > 6이므로 R을 왼쪽으로 한 칸 옮깁니다.
L = 3 > R = 2가 되었으므로 두 값의 교환을 끝냅니다.




한 페이지씩 넘기며 화살표 위치와 바뀐 값을 비교하세요.

# 마지막 교환: 피벗 6의 자리 확정

L = 3, R = 2로 교차했습니다. L이 가리키는 8과 맨 끝의 피벗 6을 바꿉니다.

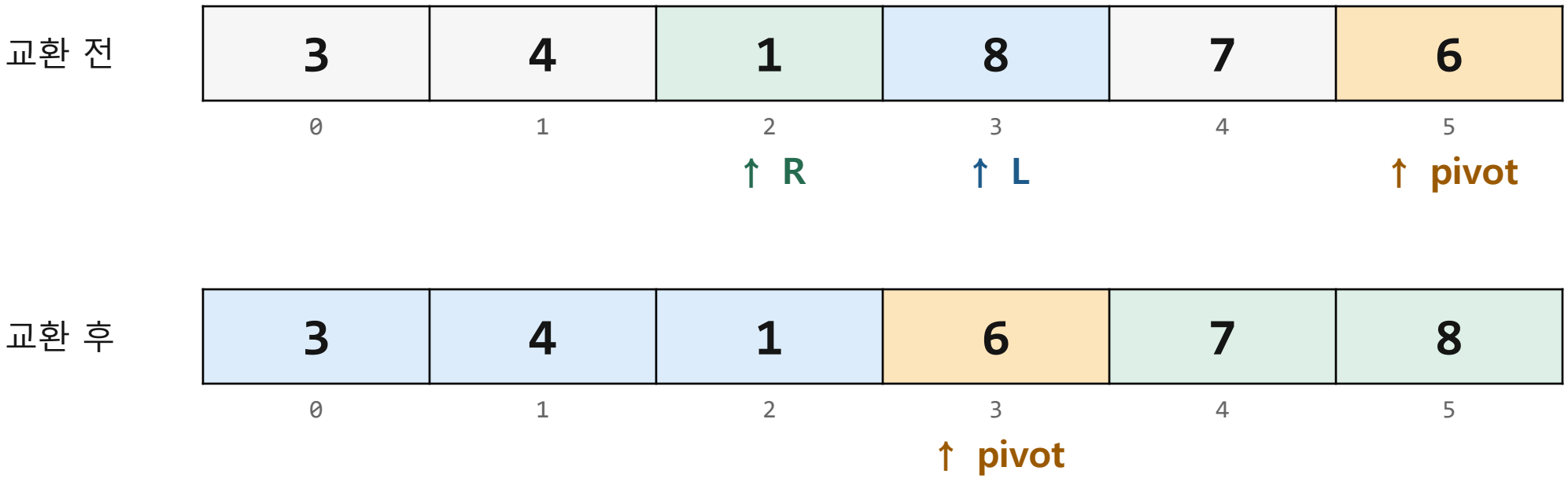



6의 자리는 확정되었습니다. 6을 제외한 왼쪽 세 칸과 오른쪽 두 칸을 다시 정렬합니다.

# ⑧ 피벗을 두 구간 사이에 놓는 교환

L = 3, R = 2로 교차했습니다. L은 큰 값 구간의 첫 칸입니다.
그 칸의 8과 맨 끝 피벗 6을 교환합니다. 피벗의 최종 위치는 3입니다.

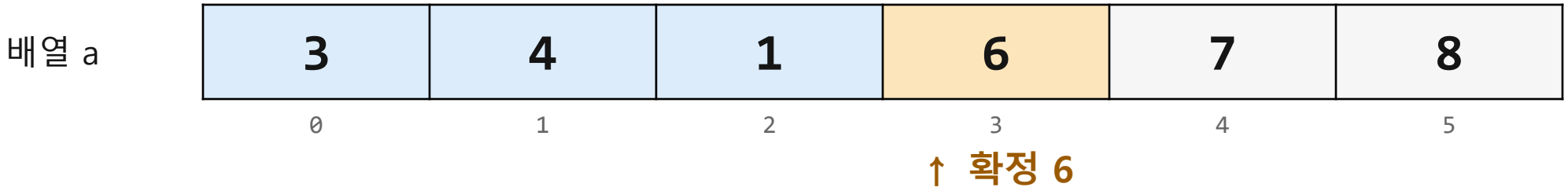
```
a[L], a[hi] = a[hi], a[L]
```




파랑: 6 이하인 왼쪽 구간 주황: 피벗 6 초록: 6보다 큰 오른쪽 구간

# 다음 호출: 왼쪽 구간 0~2부터

피벗 6의 위치 p = 3은 확정됐습니다. 다음 정렬에서 제외합니다.
왼쪽 [3,4,1]을 먼저 정렬하고, 그 호출이 끝나면 오른쪽 [7,8]을 정렬합니다.




```
quick_sort(a, 0, 2)
quick_sort(a, 4, 5)
```

먼저 왼쪽 호출을 끝냅니다. 오른쪽 호출은 그동안 기다립니다.

# 왼쪽 [3,4,1]: 피벗 4의 이동

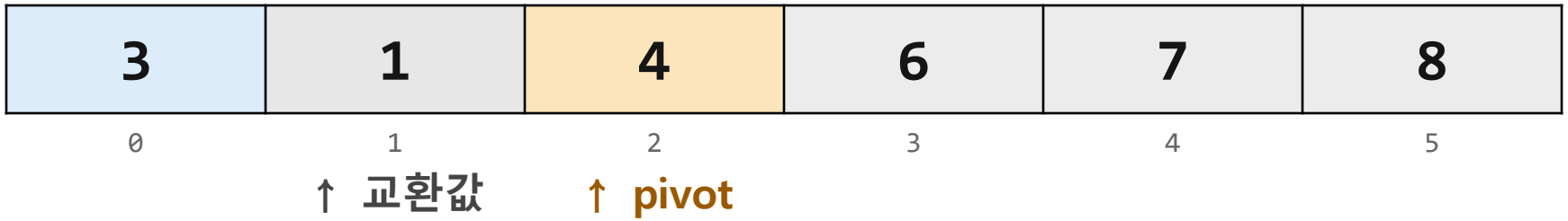
lo = 0, hi = 2이므로 가운데 인덱스는 1입니다. 피벗은 a[1] = 4입니다.
피벗 4와 이 구간의 마지막 값 a[2] = 1을 교환합니다.

```
a[1], a[2] = a[2], a[1]
```

교환 전




교환 후




현재 구간의 마지막 칸은 hi = 2입니다.

# 피벗 4: L = 0, R = 1에서 시작

검사할 값은 3과 1입니다. 피벗 4는 인덱스 2에 따로 둡니다.
L은 3을, R은 1을 가리킵니다. 먼저 L의 값부터 검사합니다.

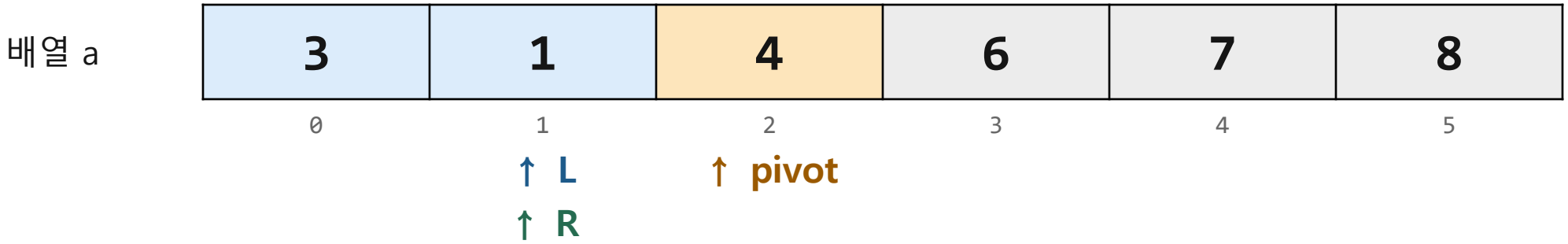



```
pivot = 4
L, R = 0, 1
```

3 ≤ 4이므로 다음에 L을 한 칸 늘립니다.

# 피벗 4: L이 0에서 1로 이동

a[0] = 3 ≤ 4이므로 L을 1로 늘립니다. 배열 값은 그대로입니다.
이제 L과 R은 모두 인덱스 1의 값 1을 가리킵니다.

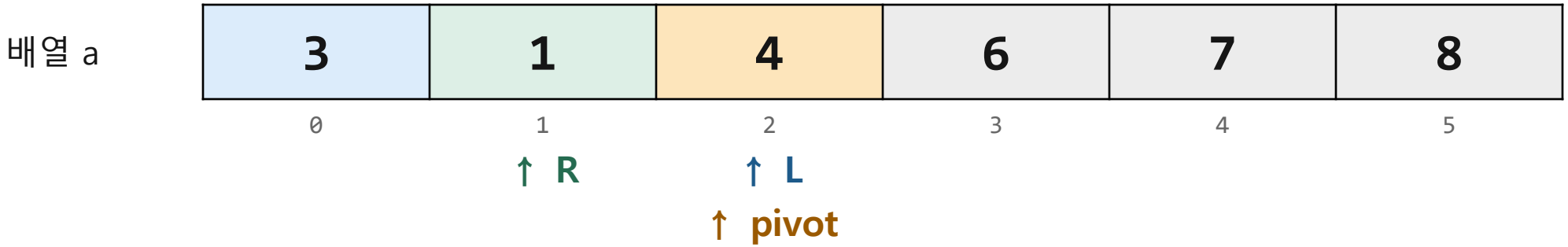



L = 1

L = R인 칸도 검사합니다. 1 ≤ 4이므로 L이 다시 이동합니다.

# 피벗 4: 교차 후 인덱스 2 확정

a[1] = 1 ≤ 4이므로 L = 2, R = 1로 교차합니다. 탐색을 끝냅니다.
L = hi = 2이므로 마지막 교환은 4와 자기 자신입니다. 배열은 그대로입니다.



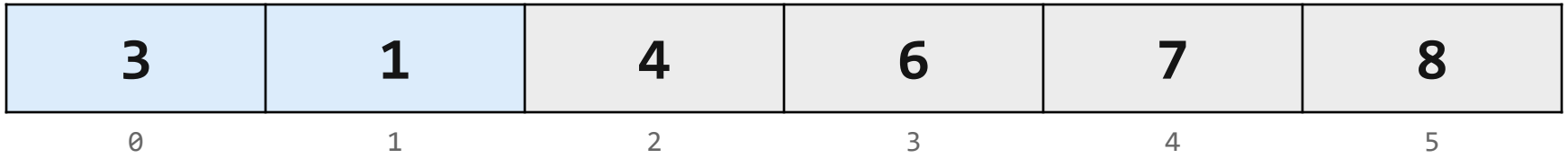

```
a[2], a[2] = a[2], a[2]
```

partition의 반환값 p = 2입니다. 앞의 [3,1]은 아직 정렬해야 합니다.

# 다음 왼쪽 호출: 구간 0~1

피벗 4의 위치 p = 2를 제외하고, 왼쪽 [3,1]을 다시 정렬합니다.
현재 lo = 0, hi = 1입니다. 4와 6의 칸은 이번 호출에서 움직이지 않습니다.

배열 a




```
quick_sort(a, 0, 1)
```

원소가 두 개이므로 한 번 더 분할합니다.

# 왼쪽 [3,1]: 피벗 3의 이동

가운데 인덱스 (0 + 1) // 2 = 0이므로 피벗은 a[0] = 3입니다.
피벗 3과 현재 구간의 마지막 값 a[1] = 1을 교환합니다.

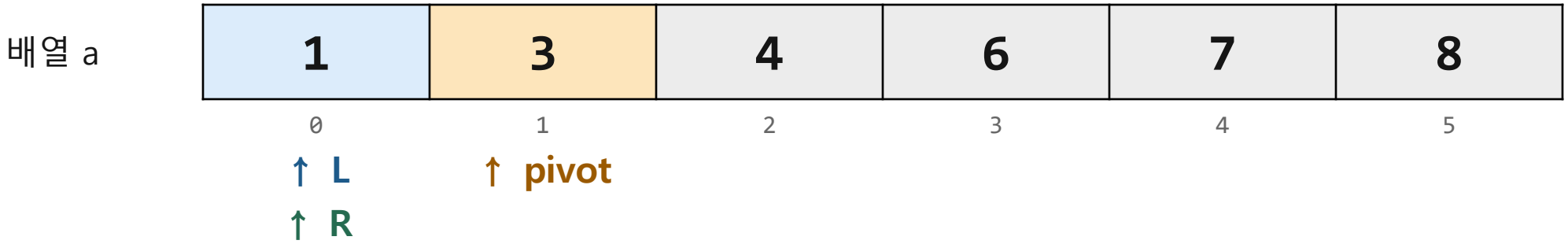
```
a[0], a[1] = a[1], a[0]
```




이번 구간의 hi는 1입니다. 그 뒤의 칸은 바꾸지 않습니다.

# 피벗 3: L = R = 0에서 시작

피벗 3은 인덱스 1에 두고, 앞의 값 1만 검사합니다.
L과 R은 모두 인덱스 0을 가리킵니다. 1 ≤ 3이므로 L을 늘립니다.

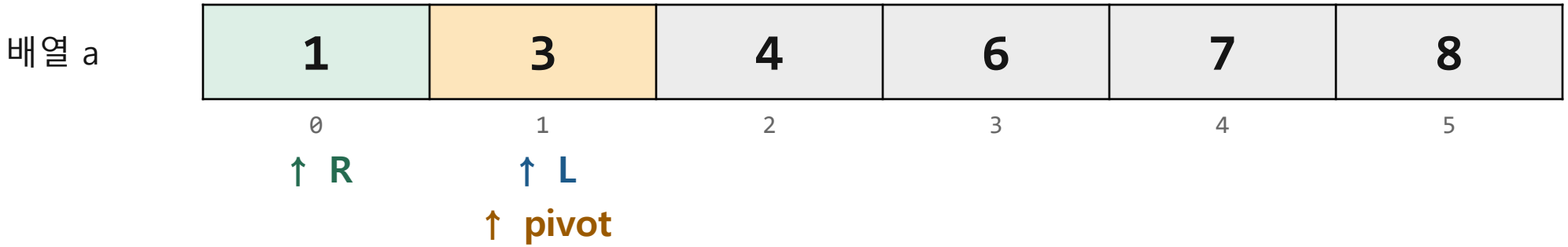



```
pivot = 3
L, R = 0, 0
```

L = R이어도 현재 칸을 한 번 검사합니다.

# 피벗 3: 교차 후 인덱스 1 확정

1 ≤ 3이므로 L = 1, R = 0이 되어 탐색을 끝냅니다.
L = hi = 1입니다. 마지막 교환은 3과 자기 자신이므로 배열은 그대로입니다.

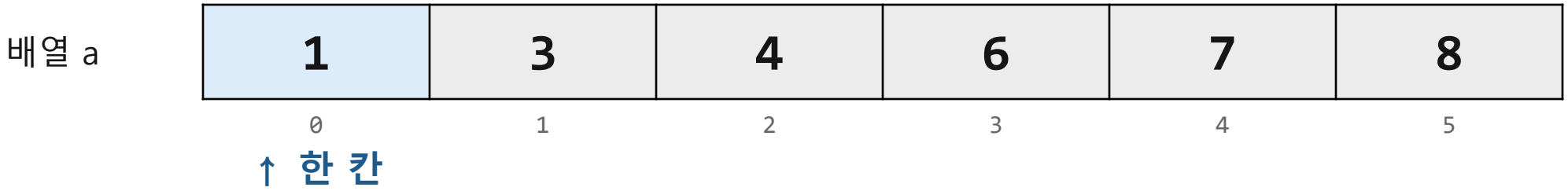



```
a[1], a[1] = a[1], a[1]
```

partition의 반환값 p = 1입니다. 다음 구간은 [0,0]과 [2,1]입니다.

# 종료 조건: 한 칸과 빈 구간

quick_sort(a, 0, 0)은 원소 하나입니다. quick_sort(a, 2, 1)은 빈 구간입니다.
두 호출 모두 lo ≥ hi이므로 partition을 호출하지 않고 바로 돌아옵니다.

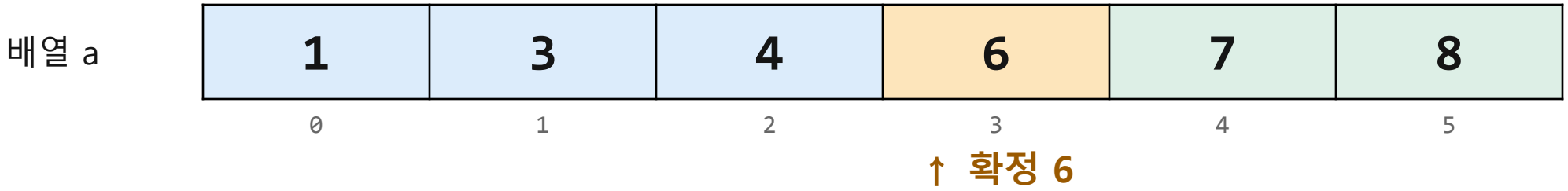



```
quick_sort(a, 0, 0)
quick_sort(a, 2, 1)
```

종료 조건 lo ≥ hi는 원소 하나인 경우와 빈 구간을 함께 처리합니다.

# 왼쪽 정렬 종료와 원래 호출로 복귀

구간 0~1을 끝내고 0~2 호출로 돌아옵니다. 오른쪽 [3,2]도 빈 구간입니다.
왼쪽 [1,3,4]를 모두 끝냈으므로, 원래 0~5 호출에서 오른쪽 4~5를 시작합니다.



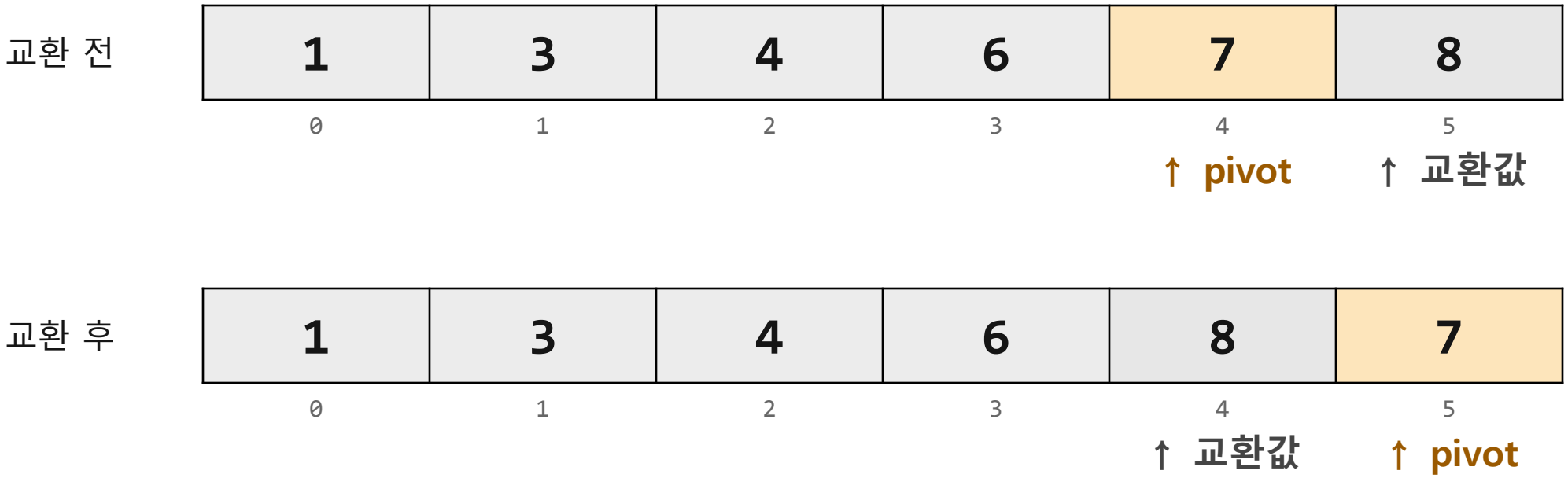

```
quick_sort(a, 3, 2)
quick_sort(a, 4, 5)
```

왼쪽 [1,3,4]는 완료했습니다. 다음 검사 범위는 인덱스 4~5입니다.

# 오른쪽 [7,8]: 피벗 7의 이동

lo = 4, hi = 5입니다. 가운데 인덱스는 4이고 피벗은 a[4] = 7입니다.
피벗 7과 현재 구간의 마지막 값 a[5] = 8을 교환합니다.

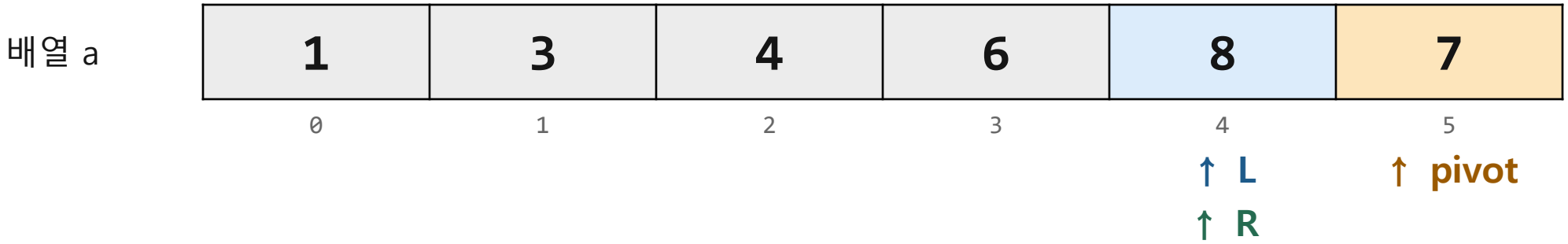
```
a[4], a[5] = a[5], a[4]
```




원래 [7,8]이어도 이 코드의 준비 교환을 그대로 실행합니다.

# 피벗 7: L = R = 4에서 시작

검사할 값은 a[4] = 8 하나입니다. 피벗 7은 인덱스 5에 둡니다.
8 > 7이므로 L은 멈춥니다. 이어서 R의 조건을 검사합니다.

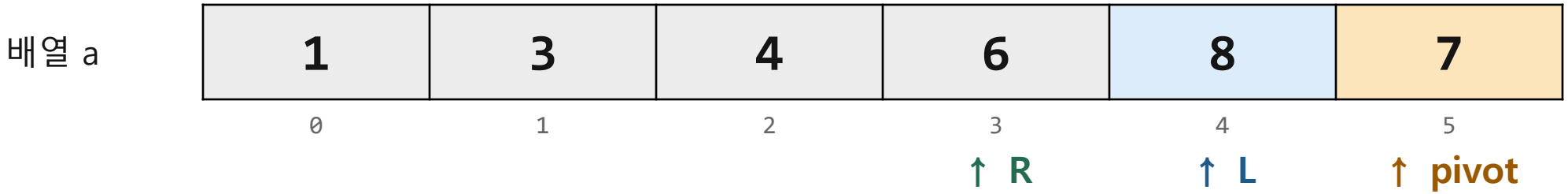



```
pivot = 7
L, R = 4, 4
```

L은 8 > 7에서 멈춥니다. R은 같은 8을 지나 왼쪽으로 이동합니다.

# 피벗 7: R이 4에서 3으로 이동

a[4] = 8 > 7이므로 R을 3으로 줄입니다. L = 4, R = 3으로 교차합니다.
L ≤ R 조건이 거짓이므로 a[3] = 6은 비교하지 않습니다. 이번 탐색은 끝입니다.




```
L, R = 4, 3
```

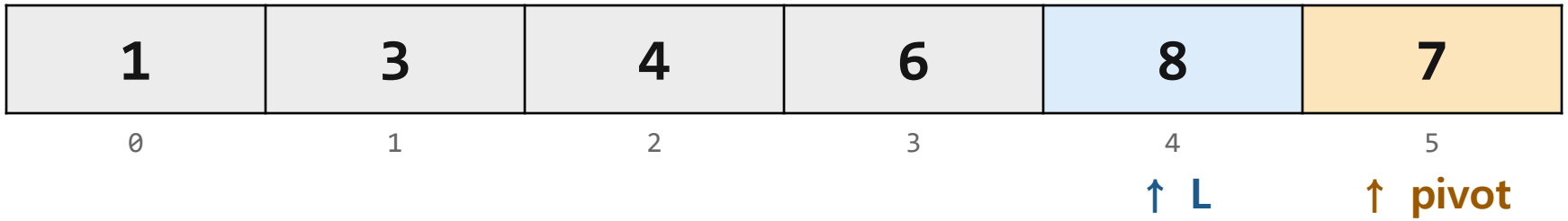
R은 범위 밖으로 이동했지만, 범위 밖의 값 6은 검사하거나 바꾸지 않습니다.

# 피벗 7과 8 교환: 인덱스 4 확정

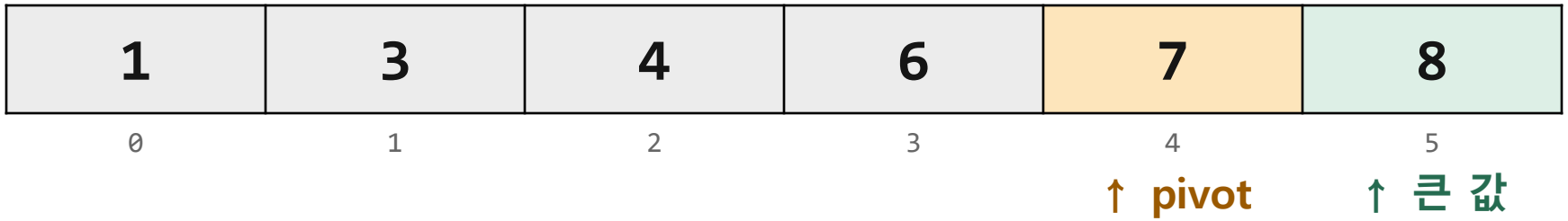
L = 4의 값 8과 hi = 5의 피벗 7을 교환합니다.
피벗 7은 인덱스 4, 큰 값 8은 인덱스 5에 놓입니다.

```
a[4], a[5] = a[5], a[4]
```

교환 전



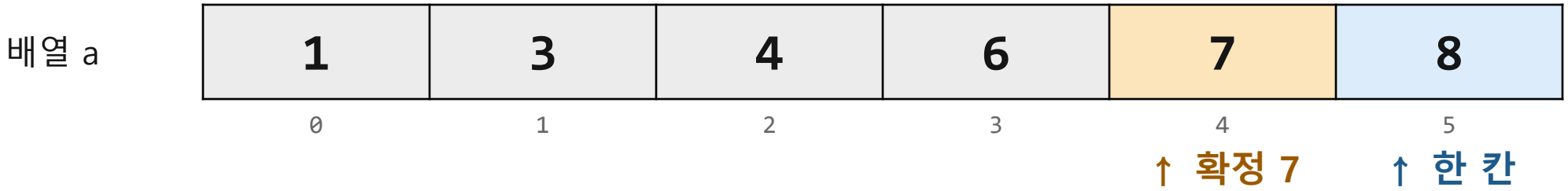

교환 후




partition의 반환값 p = 4입니다. 다음 구간은 [4,3]과 [5,5]입니다.

# 오른쪽의 남은 호출도 종료

왼쪽 [4,3]은 빈 구간이고, 오른쪽 [5,5]는 값 8 하나입니다.
두 호출 모두 lo ≥ hi에서 끝납니다. 오른쪽 4~5의 정렬도 완료했습니다.

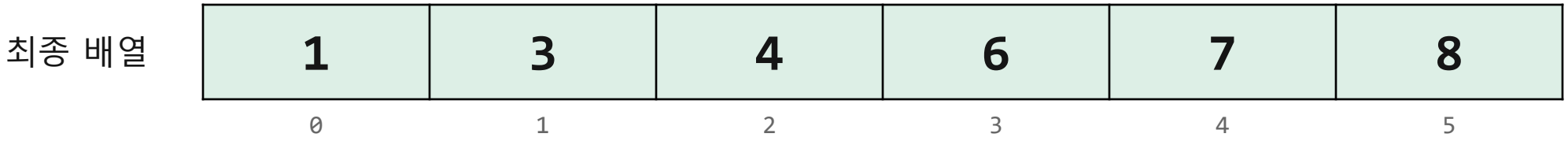



```
quick_sort(a, 4, 3)
quick_sort(a, 5, 5)
```

이 두 호출은 partition 없이 끝납니다. 원래 0~5 호출로 돌아갑니다.

# 퀵 정렬 완료: 모든 호출 종료

최종 배열은 [1,3,4,6,7,8]입니다. 원래 배열 안에서 정렬했습니다.
왼쪽 호출과 오른쪽 호출이 모두 끝나, 처음 quick_sort(a, 0, 5)도 종료합니다.




```
quick_sort(a, 0, 5)
```

피벗 확정 순서: 6(인덱스 3), 4(인덱스 2), 3(인덱스 1), 7(인덱스 4)

# Python: 그림과 같은 partition

```
def partition(a, lo, hi):
    pivot_idx = (lo + hi) // 2
    a[pivot_idx], a[hi] = a[hi], a[pivot_idx]
    pivot = a[hi]
    L, R = lo, hi - 1
    while True:
        while L <= R and a[L] <= pivot:
            L += 1
        while L <= R and a[R] > pivot:
            R -= 1
        if L > R:
            break
        a[L], a[R] = a[R], a[L]
    a[L], a[hi] = a[hi], a[L]
    return L
```

# Python: 피벗을 제외한 재귀 호출

```
def quick_sort(a, lo, hi):
    if lo >= hi:
        return
    p = partition(a, lo, hi)
    quick_sort(a, lo, p - 1)
    quick_sort(a, p + 1, hi)

a = [8, 4, 6, 3, 7, 1]
quick_sort(a, 0, len(a) - 1)
print(a)  # [1, 3, 4, 6, 7, 8]
```

# C: 양쪽 포인터 partition

```
void swap(int *x, int *y) {
    int t = *x; *x = *y; *y = t;
}
int partition(int a[], int lo, int hi) {
    int m = lo + (hi - lo) / 2;
    swap(&a[m], &a[hi]);
    int pivot = a[hi], L = lo, R = hi - 1;
    while (1) {
        while (L <= R && a[L] <= pivot) L++;
        while (L <= R && a[R] > pivot) R--;
        if (L > R) break;
        swap(&a[L], &a[R]);
    }
    swap(&a[L], &a[hi]);
    return L;
}
```

# C: 재귀 호출과 실행

```
/* swap과 partition을 이 함수 앞에 함께 넣는다. */
void quick_sort(int a[], int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quick_sort(a, lo, p - 1);
    quick_sort(a, p + 1, hi);
}
/* 실행 예: main 안에서 */
int a[] = {8, 4, 6, 3, 7, 1};
quick_sort(a, 0, 5);
/* a는 {1, 3, 4, 6, 7, 8}이 된다. */
```

# 같은 값과 빈 구간도 처리한다

이 구현은 피벗 이하를 왼쪽, 초과를 오른쪽에 둔다.
같은 값이 있어도 L이 계속 이동하여 무한 반복을 피한다.
모두 같은 값에서는 한쪽으로 치우쳐 최악 $\Theta(n^2)$가 된다.
빈 배열의 첫 호출은 quick_sort(a, 0, −1), 즉시 종료

모두 같은 값

| 2 | 2 | 2 |
|---|---|---|

분할 크기

| 왼쪽 2개 | 피벗 1개 | 오른쪽 0개 |
|---|---|---|

파랑 L: 왼쪽 포인터 초록 R: 오른쪽 포인터 주황: 피벗

# 피벗에 따라 달라지는 시간

분할 한 번은 Θ(n)
균형 분할은 Θ(log n) 단계, 전체 Θ(n log n)
한쪽에 n − 1개씩 남으면 최악 Θ($n^2$)
서로 다른 값의 무작위 순열에서 평균 Θ(n log n)
가운데 인덱스의 값이 중앙값이라는 보장은 없다.

균형 분할

| 절반 | 피벗 | 절반 |
|---|---|---|

치우친 분할

| n − 1개 | 피벗 | 0개 |
|---|---|---|

# 분할 한 번: 포인터는 뒤로 돌아가지 않습니다

| 포인터 | 진행 방향 | 이 예제의 이동 |
|---|---|---|
| L | 왼쪽 → 오른쪽 | 0 → 1 → 2 → 3 |
| R | 오른쪽 → 왼쪽 | 4 → 3 → 2 |

**교환 후에도 처음부터 다시 훑지 않습니다.**

## 길이 k인 구간의 partition: O(k)

중첩된 while문의 모양보다, 포인터의 총 이동 횟수를 봅니다.

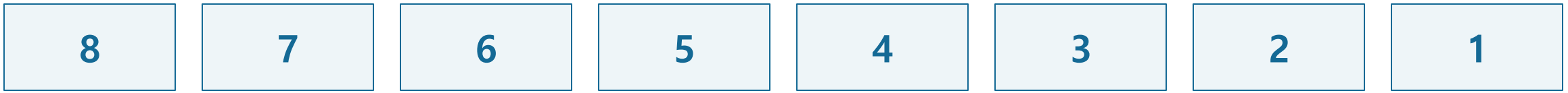
# 비슷하게 나뉘면: 층 수가 짧아집니다




## 한 층의 합계 O(n) × 약 O(log n)층

비슷한 크기로 나뉠 때 O(n log n). 가운데 칸의 값이 중앙값이라는 보장은 없습니다.

# 같은 값이 계속 나오면: 긴 재귀 사슬




입력: `[5, 5, 5, 5, 5, 5, 5, 5]`

**피벗 밖의 검사: 7 + 6 + … + 1 = 28**

**한쪽에 n−1개가 남는 경우: $O(n^2)$**

이 코드의 "피벗 이하를 왼쪽" 규칙에서는 같은 값만 있어도 최악의 분할이 가능합니다.

# 두 정렬의 차이 확인

병합 정렬: 위치로 반씩 나누고, 정렬된 결과를 병합
퀵 정렬: 피벗 값으로 분할하고, 양쪽을 재귀 정렬
병합은 최악에도 $\Theta(n \log n)$, 퀵은 최악 $\Theta(n^2)$
현재 병합 코드는 안정적, 퀵 코드는 안정성 보장 없음
확인: 퀵의 첫 분할만 끝나면 전체가 정렬되어 있을까?

4.4

# 최대 구간 합

Maximum Subarray Sum
주가 등락폭 데이터에서 가장 큰 이익을 내는 연속 구간을 찾아라

# 문제: 가장 큰 이익을 내는 연속 구간

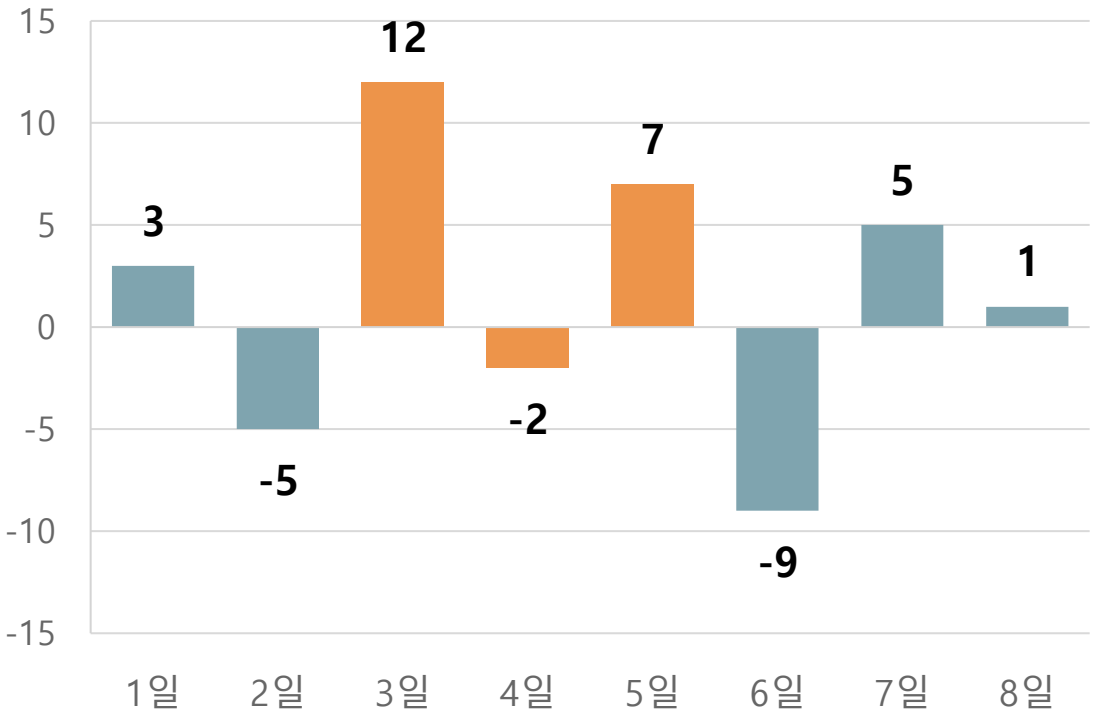
일별 손익 변동이 배열로 주어집니다.
연속한 날짜의 합을 최대로 만드세요.
하루 이상 고르며,
중간 날짜는 건너뛸 수 없습니다.

| 3 | -5 | 12 | -2 | 7 | -9 | 5 | 1 |
|---|---|---|---|---|---|---|---|
| 1일 | 2일 | 3일 | 4일 | 5일 | 6일 | 7일 | 8일 |



전체 8일 합: 12

**3~5일 합: 17**



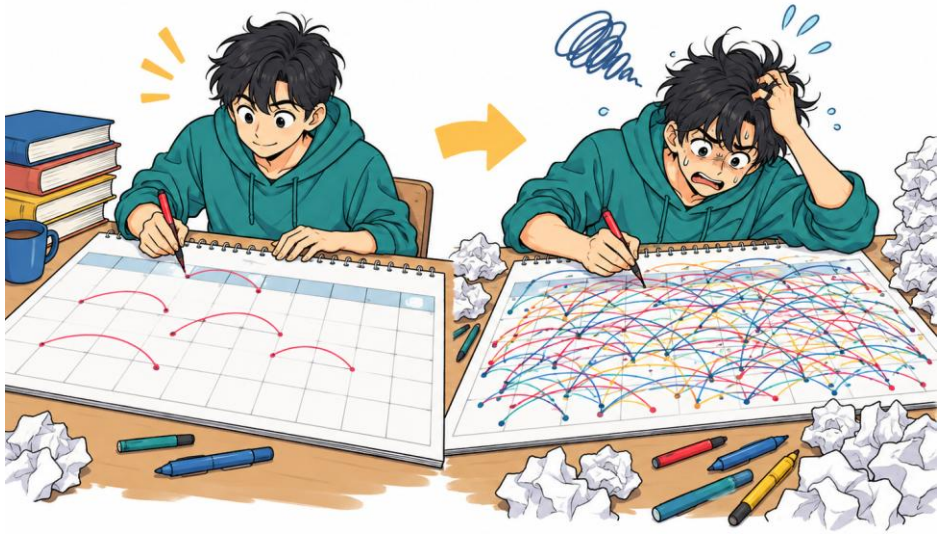

**3~5일: 12 − 2 + 7 = 17**

# 모든 구간을 하나씩 확인하는 방법

시작 날짜를 하나 고정하고,
끝 날짜를 한 칸씩 늘립니다.

첫날부터 시작한 구간

```
[3]                 →   3
[3, -5]             →  -2
[3, -5, 12]         →  10
…
```



8일의 후보: 8 + 7 + … + 1 = 36개

입력이 커지면 후보 수가 $O(n^2)$로 늘어납니다.

끝점을 한 칸 늘릴 때 누적합도 갱신합니다. 더 빠른 방법으로 분할 정복을 사용해 봅시다.

# 왼쪽·오른쪽·중앙을 걸친 구간

## 왼쪽 절반 안

그 절반에서 찾은 최대 합: left_max

## 오른쪽 절반 안

그 절반에서 찾은 최대 합: right_max

## 중앙 경계를 걸친 구간

경계 양쪽을 이어 붙인 최대 합: cross_max

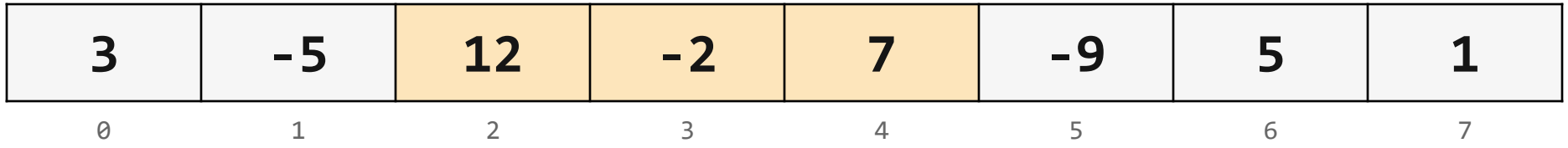


**정답 = max(left_max, right_max, cross_max)**

중앙을 걸친 구간은 mid와 mid+1을 모두 포함해야 합니다. 중간 칸을 뺄 수 없습니다.

# 최대 구간 합: 연속한 칸을 고릅니다

a = [3, −5, 12, −2, 7, −9, 5, 1]에서 빈 구간은 고르지 않습니다.
가장 큰 합은 인덱스 2~4의 12 − 2 + 7 = 17입니다.



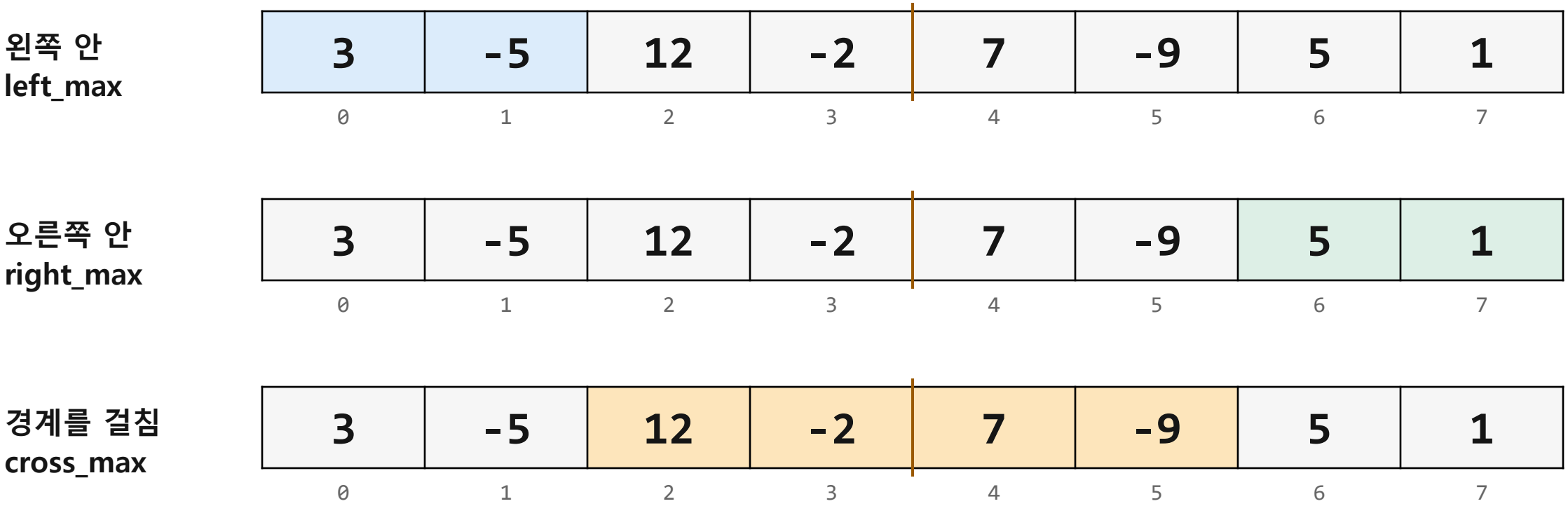

**12 − 2 + 7 = 17**

배열 전체 합 = 12

중간 칸을 건너뛸 수 없습니다. 음수 한 칸도 답이 될 수 있습니다.

# 가운데에서 나누면 후보는 세 종류

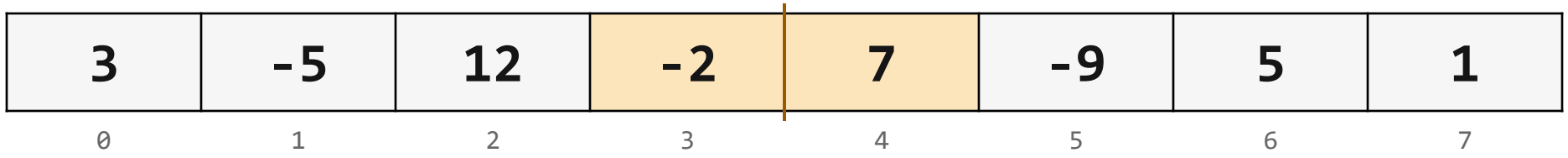
low = 0, high = 7, mid = 3입니다. 경계는 인덱스 3과 4 사이입니다.
왼쪽 안, 오른쪽 안, 경계를 걸치는 구간 중 가장 큰 합을 고릅니다.




세 줄은 후보의 종류를 보여 주는 예입니다. 실제 최댓값은 다음 과정에서 구합니다.

# 걸친 구간은 경계의 두 칸을 포함

왼쪽 마지막 칸 a[mid]와 오른쪽 첫 칸 a[mid + 1]은 빠질 수 없습니다.
왼쪽은 mid에서 왼쪽으로, 오른쪽은 mid + 1에서 오른쪽으로 늘립니다.




← 왼쪽 시작을 늘림

오른쪽 끝을 늘림 →

왼쪽 끝 = `mid`

오른쪽 시작 = `mid + 1`

left_sum과 right_sum은 경계에 붙은 구간의 합입니다.

# left_max + right_max는 답이 아닙니다

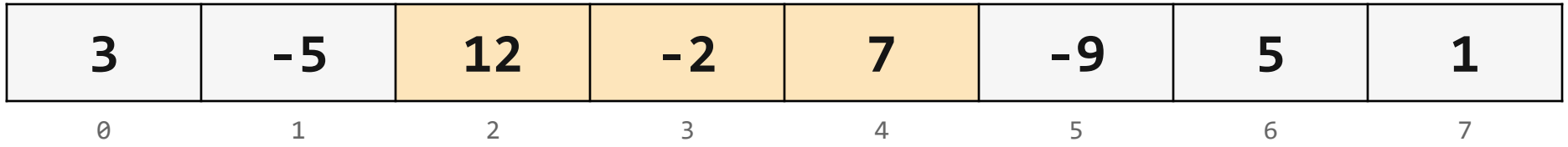
왼쪽에서 12, 오른쪽에서 7을 따로 찾더라도 12 + 7 = 19는 안 됩니다.
경계를 걸치려면 두 값 사이의 −2를 함께 더해야 합니다.

**잘못 고른 칸**




**12 + 7 = 19 → 중간 −2를 빠뜨림**

**연속한 구간**




가능한 합: 12 − 2 + 7 = 17. 아래에서는 경계 칸을 빼놓지 않았습니다.

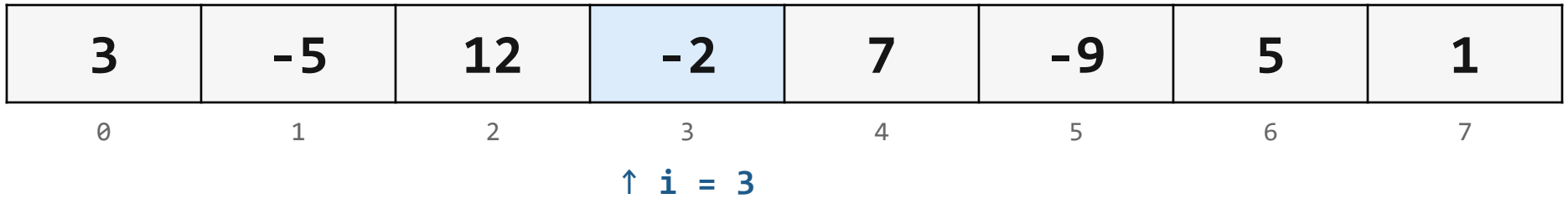
# 현재 합과 저장한 최댓값을 구별

current_sum은 지금 늘리는 구간의 합입니다. 다음 원소를 여기에 더합니다.
left_sum 또는 right_sum은 지금까지 본 후보 중 가장 큰 합을 저장합니다.

**current_sum = 0**

**left_sum = −∞**

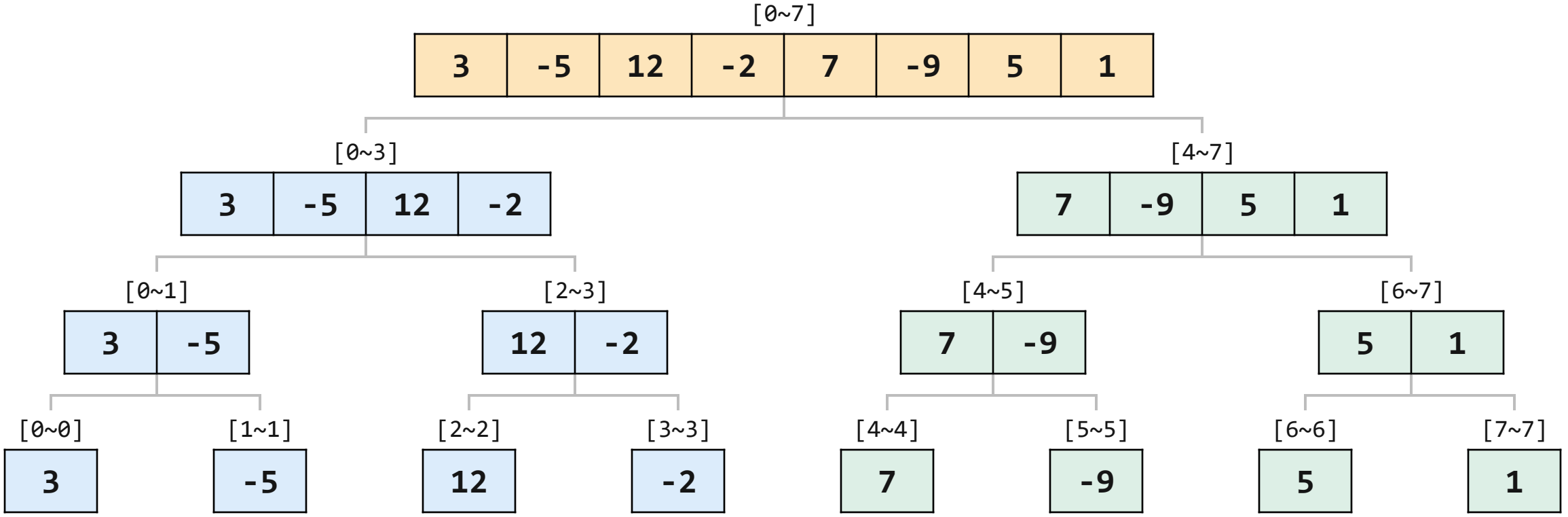
첫 칸을 읽음




첫 검사 뒤: current_sum = −2, left_sum = −2. 초기 0은 후보가 아닙니다.

# 실행 순서: 왼쪽을 끝내고 오른쪽으로

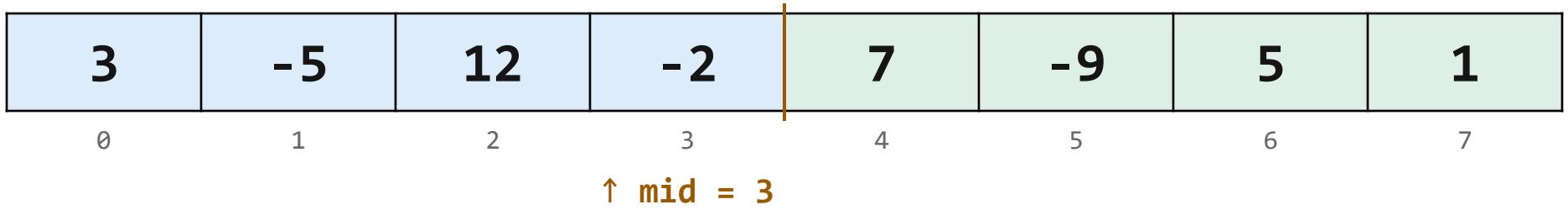
각 호출은 자신의 구간을 다시 반으로 나눕니다. 원소 하나면 그 값을 반환합니다.
왼쪽 호출이 끝나기 전에는 오른쪽 호출을 시작하지 않습니다.




이제 맨 위 호출을 시작해서 실제 코드 순서대로 끝까지 따라갑니다.

# [0~7] 호출: mid = 3에서 나눔

현재 범위 low = 0, high = 7, mid = 3입니다.
먼저 왼쪽 [0~3]를 호출합니다. 오른쪽 [4~7]는 기다립니다.



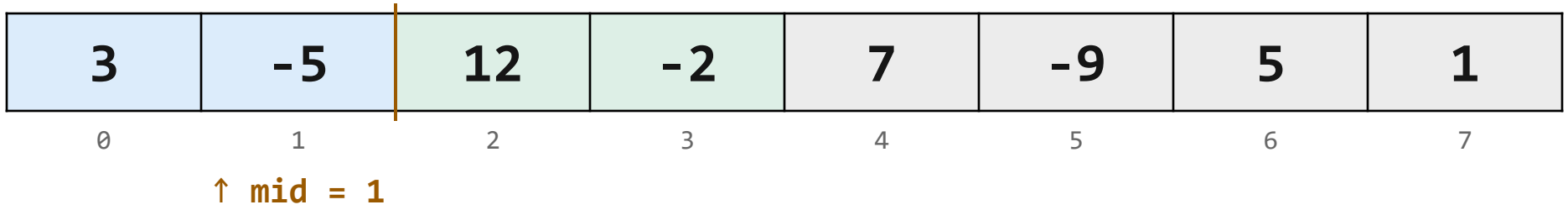

```
left_max  = max_subarray_sum(a, 0, 3)
right_max = max_subarray_sum(a, 4, 7)
```

지금은 전체 여덟 칸의 답을 구하는 호출입니다.

# [0~3] 호출: mid = 1에서 나눔

현재 범위 low = 0, high = 3, mid = 1입니다.
먼저 왼쪽 [0~1]를 호출합니다. 오른쪽 [2~3]는 기다립니다.

**현재 호출**



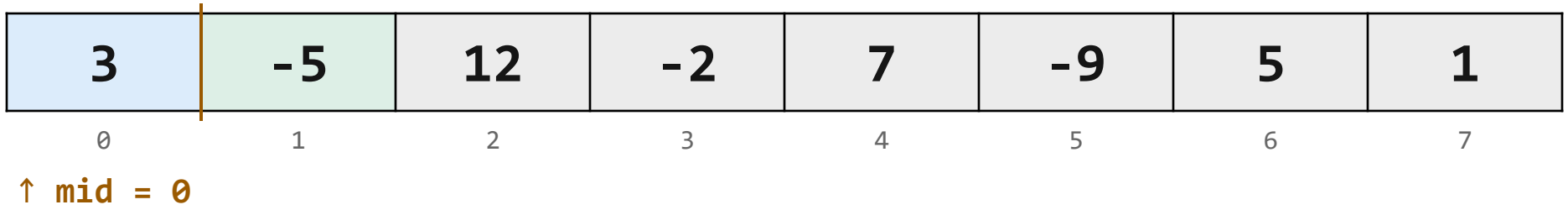

```
left_max  = max_subarray_sum(a, 0, 1)
right_max = max_subarray_sum(a, 2, 3)
```

[0~7] 부모는 left_max의 반환을 기다립니다.

# [0~1] 호출: mid = 0에서 나눔

현재 범위 low = 0, high = 1, mid = 0입니다.
먼저 왼쪽 [0~0]를 호출합니다. 오른쪽 [1~1]는 기다립니다.

**현재 호출**



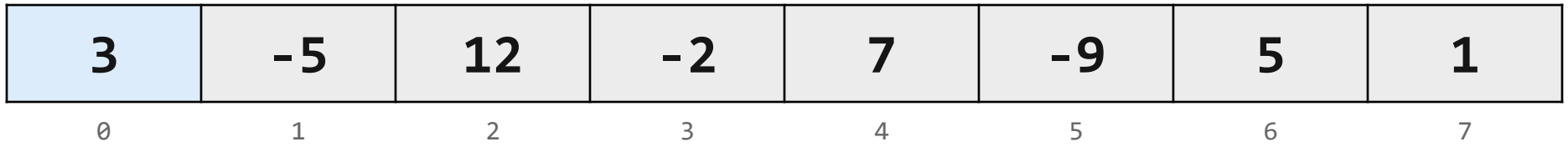

```
left_max  = max_subarray_sum(a, 0, 0)
right_max = max_subarray_sum(a, 1, 1)
```

[0~3] 부모는 left_max의 반환을 기다립니다.

# [0~0] 원소 하나: 3 반환

low = high = 0입니다. 남은 원소는 a[0] = 3 하나입니다.
이 값 3을 [0~1] 호출의 left_max에 돌려줍니다.

**현재 호출**



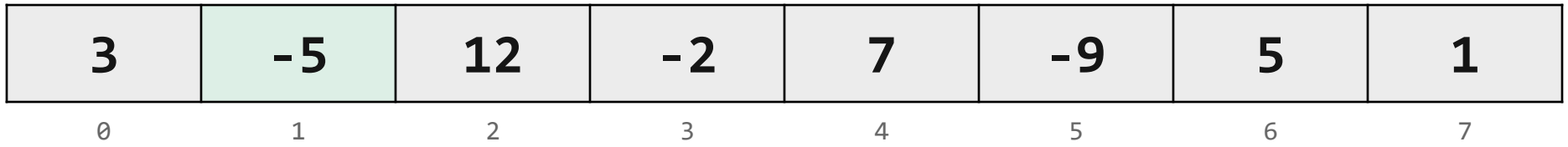

```
if low == high:
    return a[low]     # 3
```

[0~1] 호출의 left_max = 3. 원소 한 개의 호출이 끝났습니다.

# [1~1] 원소 하나: −5 반환

low = high = 1입니다. 남은 원소는 a[1] = −5 하나입니다.
이 값 −5을 [0~1] 호출의 right_max에 돌려줍니다.

**현재 호출**

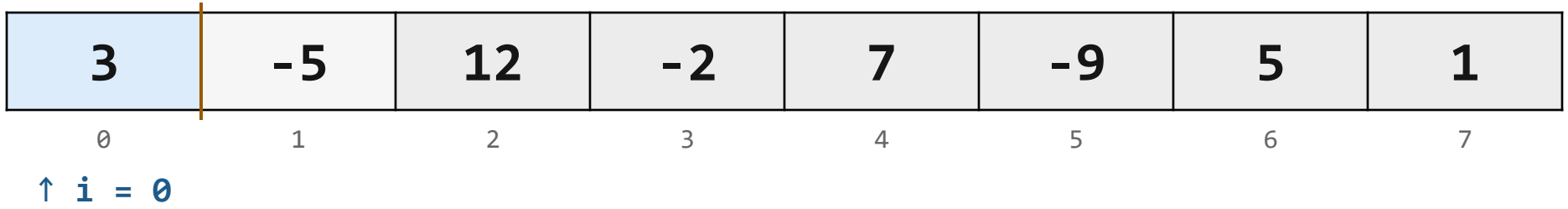



```
if low == high:
    return a[low]     # -5
```

[0~1] 호출의 right_max = −5. 원소 한 개의 호출이 끝났습니다.

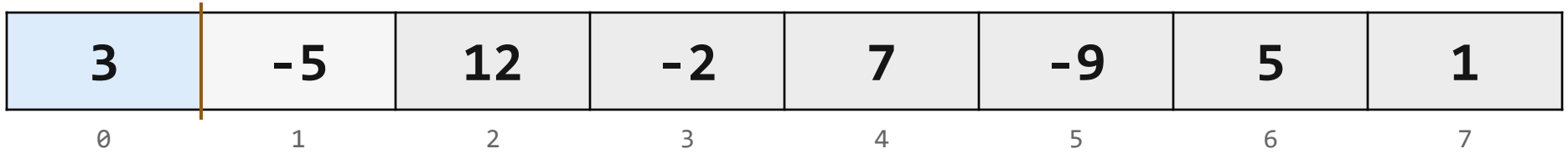
# [0~1] 왼쪽 스캔 1/1

i = 0, a[0] = 3을 더합니다. 현재 구간은 [0~0]입니다.
현재 합 3 > 이전 기록 −∞ → left_sum 갱신.




`current_sum = 0 + (3) = 3`

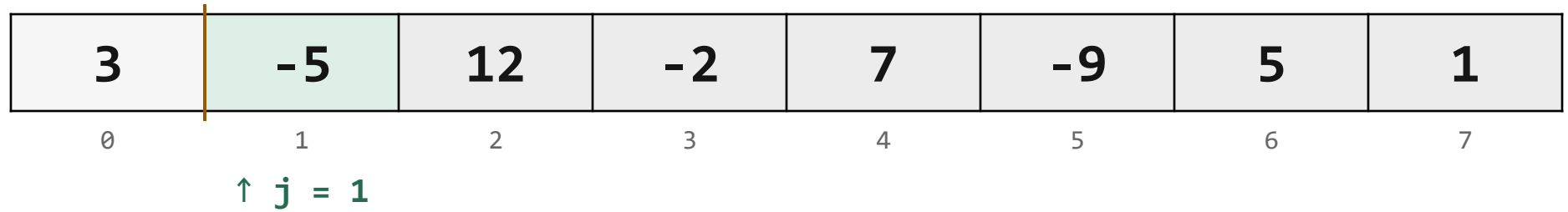
`left_sum: −∞ → 3`




왼쪽 스캔 끝. 이 기록을 보관하고 오른쪽 현재 합을 0으로 다시 시작합니다.

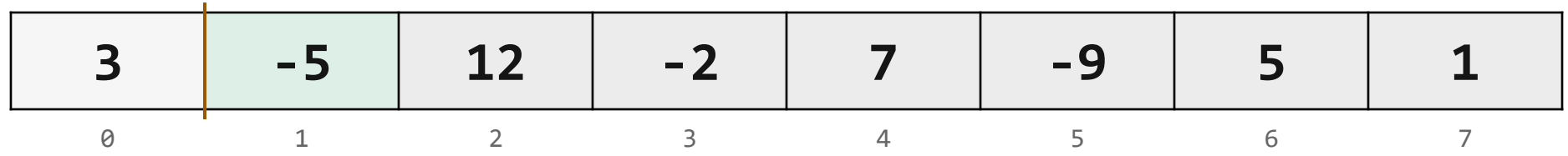
# [0~1] 오른쪽 스캔 1/1

j = 1, a[1] = −5을 더합니다. 현재 구간은 [1~1]입니다.
현재 합 −5 > 이전 기록 −∞ → right_sum 갱신.




```
current_sum = 0 + (-5) = -5
```

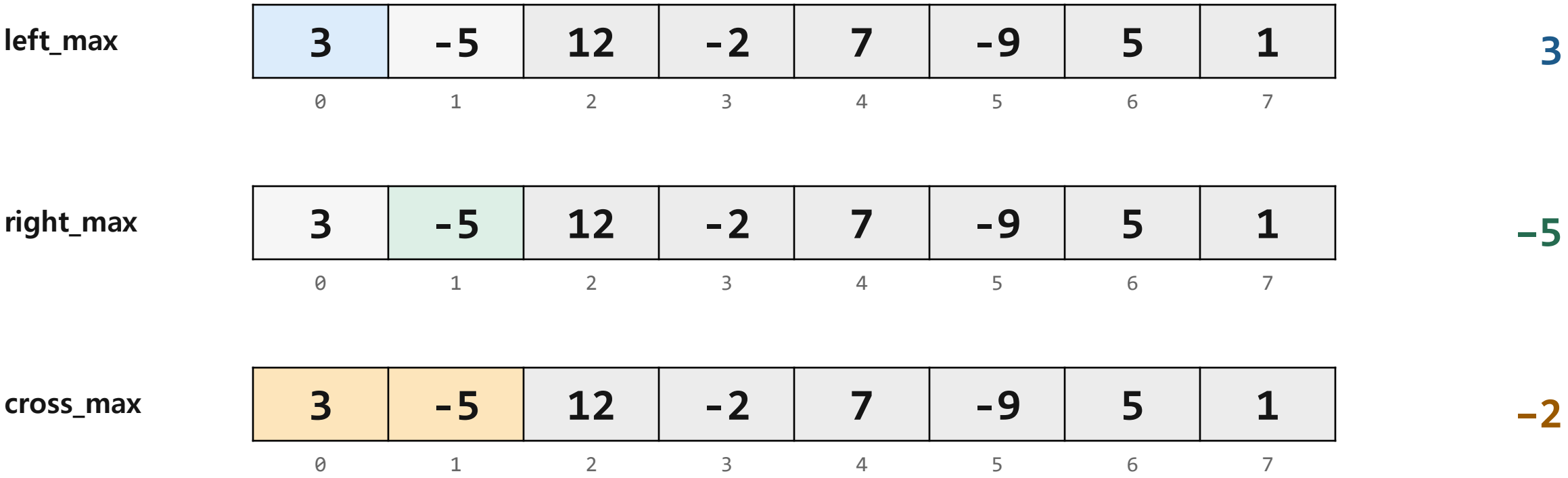
```
right_sum: -∞ → -5
```




오른쪽 스캔 끝. 왼쪽 기록과 오른쪽 기록을 더해 cross_max를 구합니다.

# [0~1] 세 후보 비교 → 3 반환

왼쪽 3, 오른쪽 −5, 경계를 걸친 합 −2을 비교합니다.
max(3, −5, −2) = 3입니다.

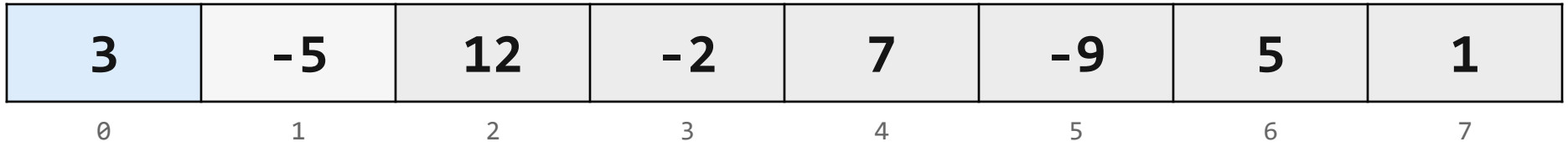



[0~3] 부모의 left_max에 3을 반환합니다.

# [0~3] 왼쪽 답 3을 받음

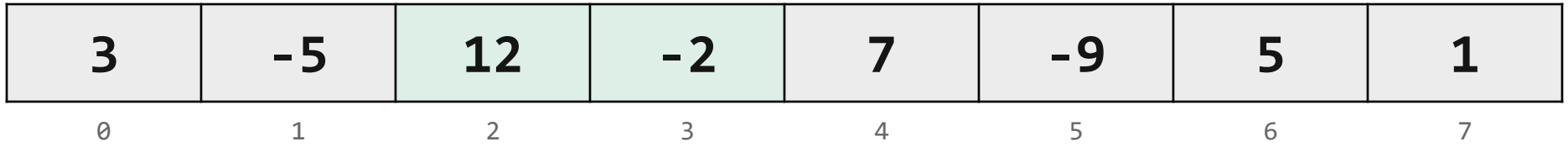
왼쪽 [0~1] 호출이 3을 반환했습니다. left_max = 3입니다.
이제 오른쪽 [2~3] 호출을 시작합니다.

**left_max = 3**




다음 호출: `max_subarray_sum(a, 2, 3)`

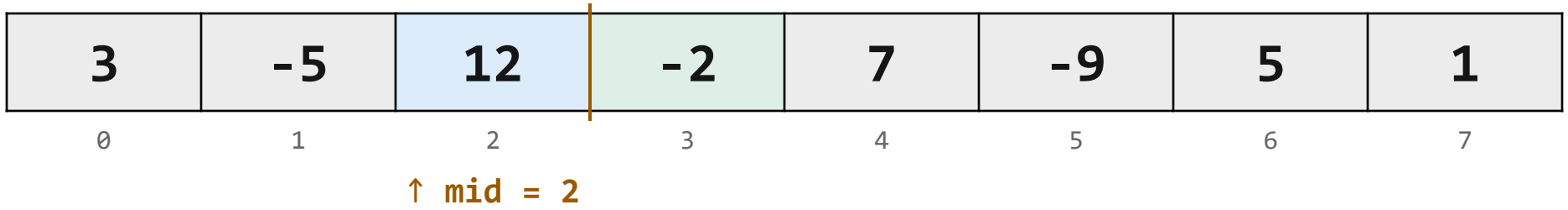
**다음 오른쪽**




left_max를 저장해 두고 오른쪽 호출이 끝날 때까지 기다립니다.

# [2~3] 호출: mid = 2에서 나눔

현재 범위 low = 2, high = 3, mid = 2입니다.
먼저 왼쪽 [2~2]를 호출합니다. 오른쪽 [3~3]는 기다립니다.



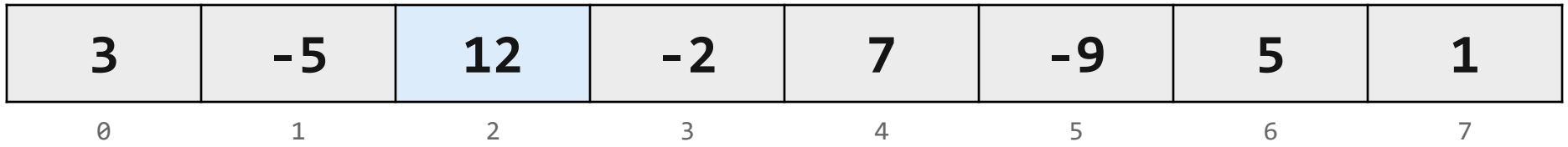

```
left_max  = max_subarray_sum(a, 2, 2)
right_max = max_subarray_sum(a, 3, 3)
```

[0~3] 부모는 right_max의 반환을 기다립니다.

# [2~2] 원소 하나: 12 반환

low = high = 2입니다. 남은 원소는 a[2] = 12 하나입니다.
이 값 12을 [2~3] 호출의 left_max에 돌려줍니다.

**현재 호출**



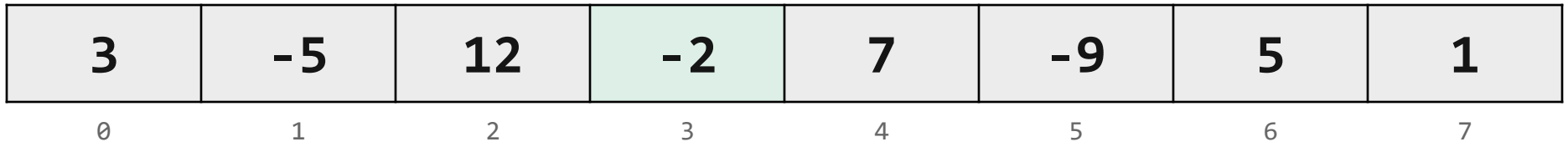

```
if low == high:
    return a[low]     # 12
```

[2~3] 호출의 left_max = 12. 원소 한 개의 호출이 끝났습니다.

# [3~3] 원소 하나: −2 반환

low = high = 3입니다. 남은 원소는 a[3] = −2 하나입니다.
이 값 −2을 [2~3] 호출의 right_max에 돌려줍니다.

**현재 호출**

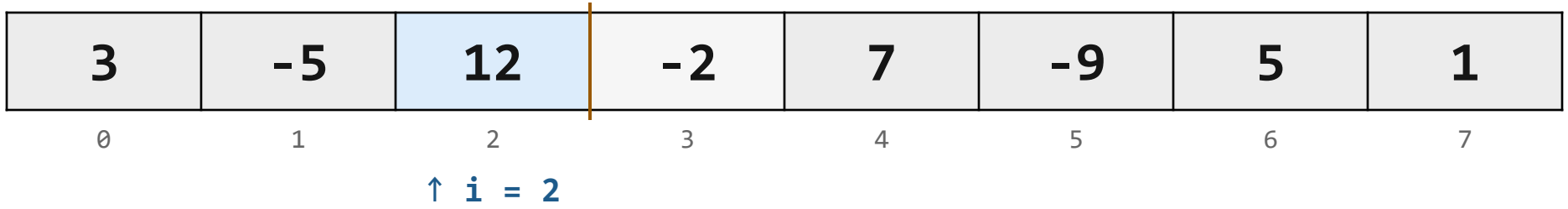



```
if low == high:
    return a[low]     # -2
```

[2~3] 호출의 right_max = −2. 원소 한 개의 호출이 끝났습니다.

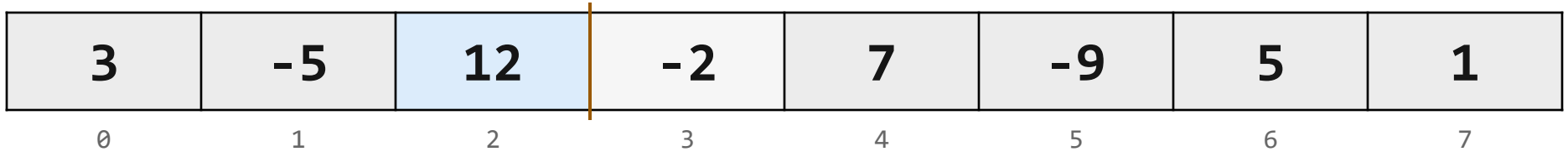
# [2~3] 왼쪽 스캔 1/1

i = 2, a[2] = 12을 더합니다. 현재 구간은 [2~2]입니다.
현재 합 12 > 이전 기록 −∞ → left_sum 갱신.




```
current_sum = 0 + (12) = 12
```

```
left_sum: -∞ → 12
```

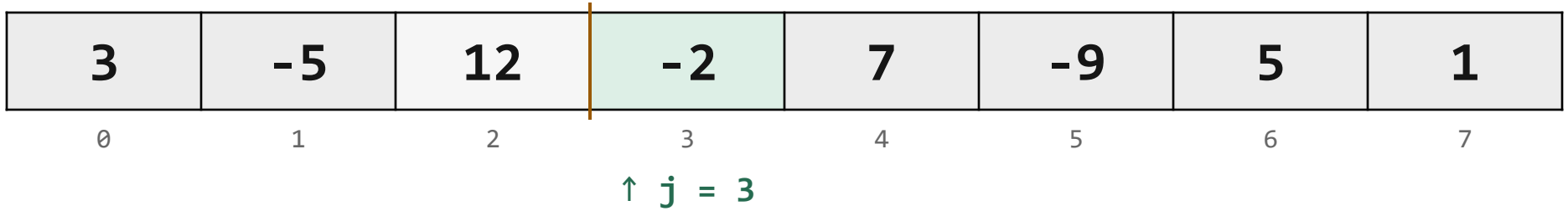



왼쪽 스캔 끝. 이 기록을 보관하고 오른쪽 현재 합을 0으로 다시 시작합니다.

# [2~3] 오른쪽 스캔 1/1

j = 3, a[3] = −2을 더합니다. 현재 구간은 [3~3]입니다.
현재 합 −2 > 이전 기록 −∞ → right_sum 갱신.

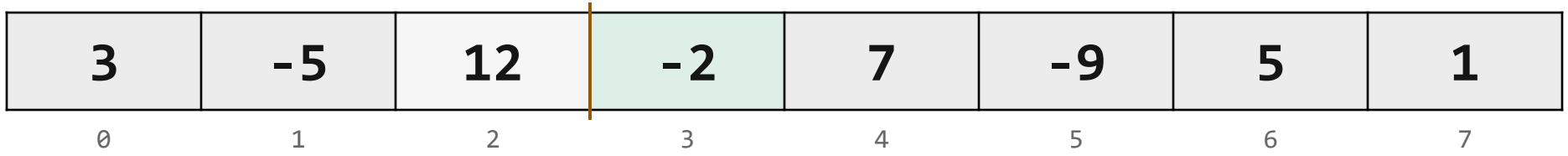
**현재 구간**




```
current_sum = 0 + (-2) = -2
```

```
right_sum: -∞ → -2
```

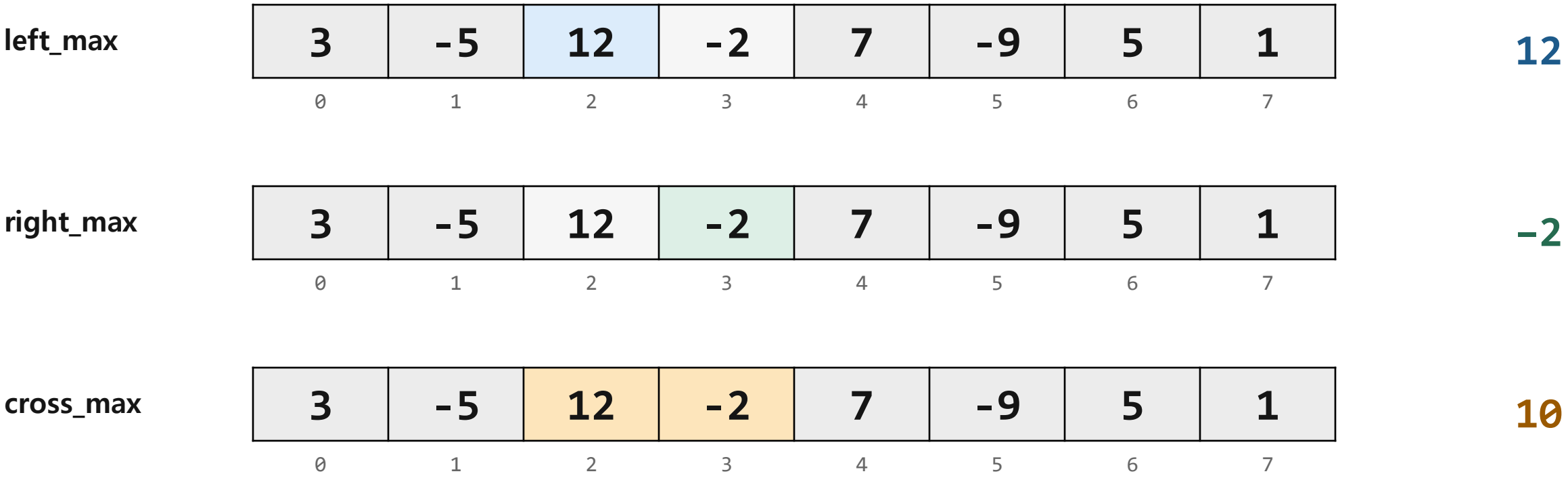
**저장한 최선**




오른쪽 스캔 끝. 왼쪽 기록과 오른쪽 기록을 더해 cross_max를 구합니다.

# [2~3] 세 후보 비교 → 12 반환

왼쪽 12, 오른쪽 −2, 경계를 걸친 합 10을 비교합니다.
max(12, −2, 10) = 12입니다.

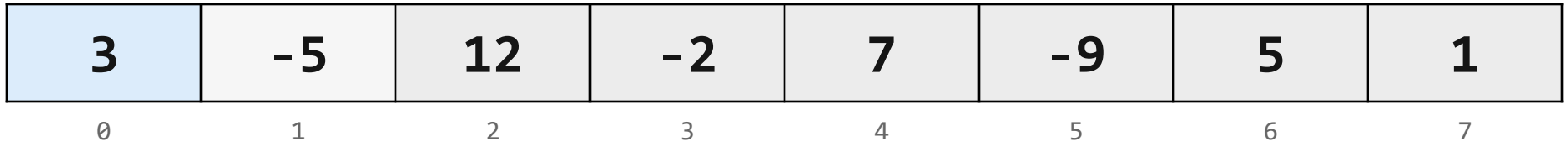



[0~3] 부모의 right_max에 12을 반환합니다.

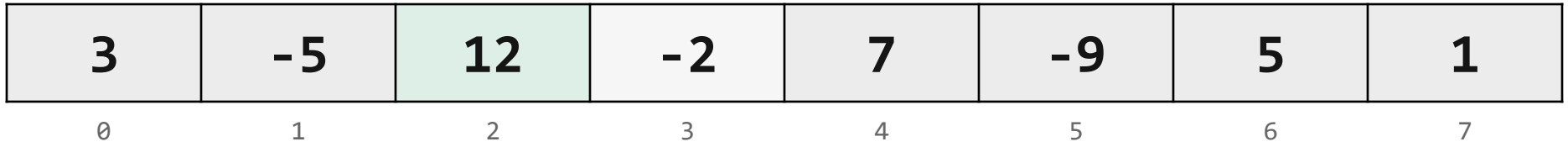
# [0~3] 오른쪽 답 12도 받음

left_max = 3, right_max = 12입니다. 두 재귀 호출을 마쳤습니다.
아직 경계를 걸치는 cross_max를 계산하고 비교해야 합니다.

**left_max = 3**




**right_max = 12**

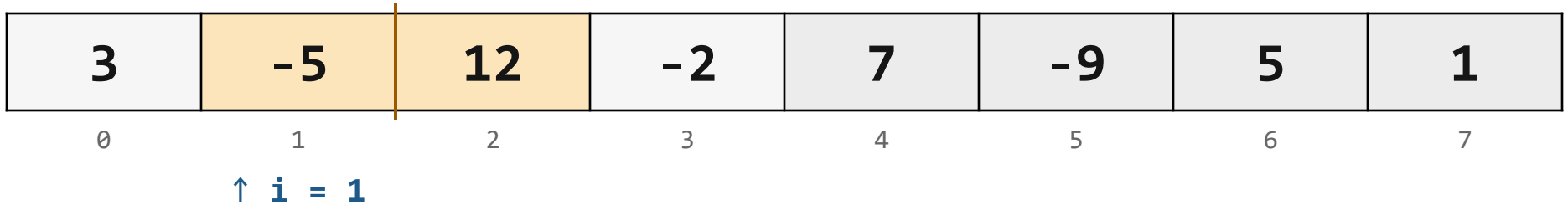



다음은 경계를 걸치는 후보입니다. left_max + right_max를 계산하지 않습니다.

# [0~3] 이제 경계를 걸치는 합 계산

mid = 1입니다. 왼쪽 스캔은 a[1] = −5부터 시작합니다.
current_sum = 0, left_sum = −∞로 두고 i를 왼쪽으로 움직입니다.

현재 경계



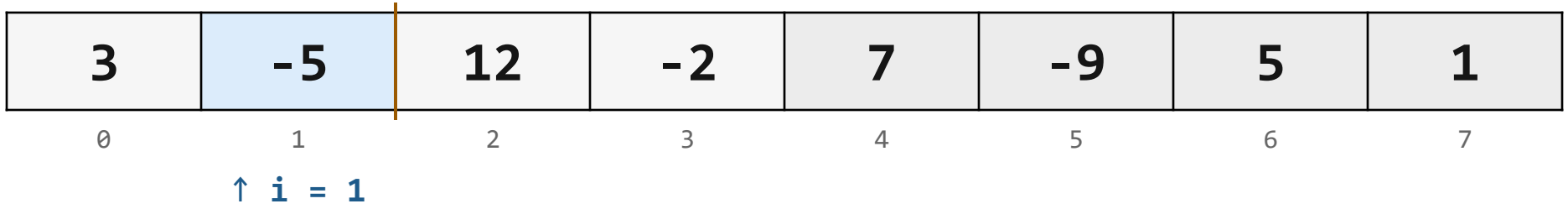

```
current_sum = 0        left_sum = -∞
```

재귀 답은 유지: left_max = 3, right_max = 12.

# [0~3] 왼쪽 스캔 1/2

i = 1, a[1] = −5을 더합니다. 현재 구간은 [1~1]입니다.
현재 합 −5 > 이전 기록 −∞ → left_sum 갱신.

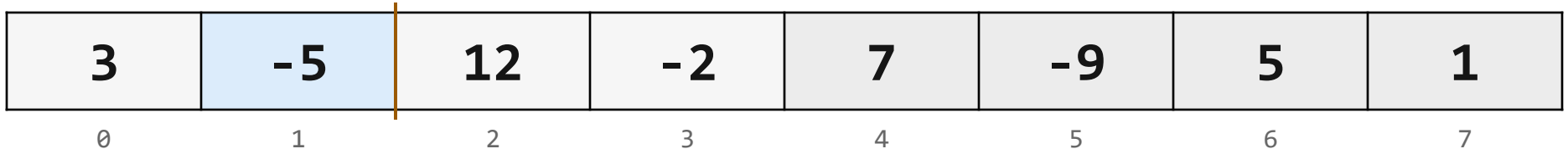
**현재 구간**




```
current_sum = 0 + (-5) = -5
```

```
left_sum: -∞ → -5
```

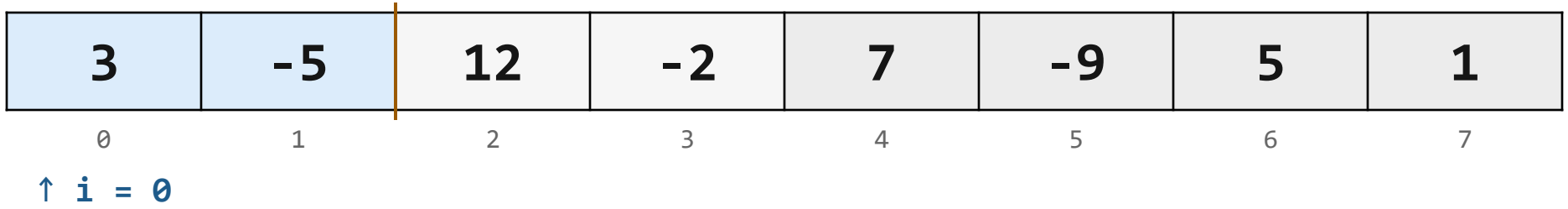
**저장한 최선**




다음 i = 0: a[0] = 3을 현재 합 −5에 더합니다.

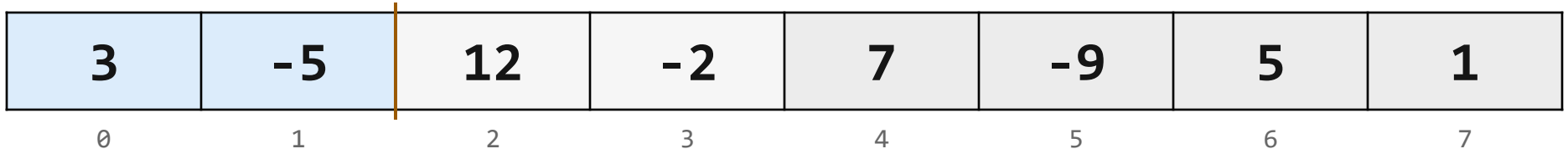
# [0~3] 왼쪽 스캔 2/2

i = 0, a[0] = 3을 더합니다. 현재 구간은 [0~1]입니다.
현재 합 −2 > 이전 기록 −5 → left_sum 갱신.




```
current_sum = -5 + (3) = -2
```

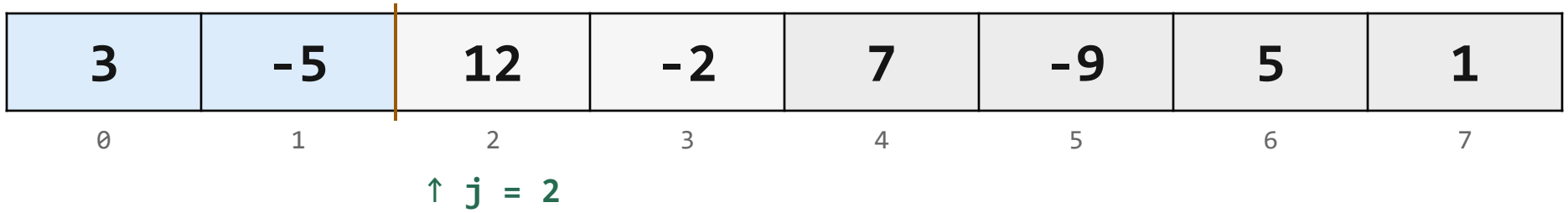
```
left_sum: -5 → -2
```




왼쪽 스캔 끝. 이 기록을 보관하고 오른쪽 현재 합을 0으로 다시 시작합니다.

# [0~3] 오른쪽은 합을 0으로 다시 시작

왼쪽 기록 left_sum = −2은 보관합니다. current_sum만 0으로 바꿉니다.
right_sum = −∞로 두고 j = mid + 1 = 2부터 오른쪽을 훑습니다.




```
current_sum = 0

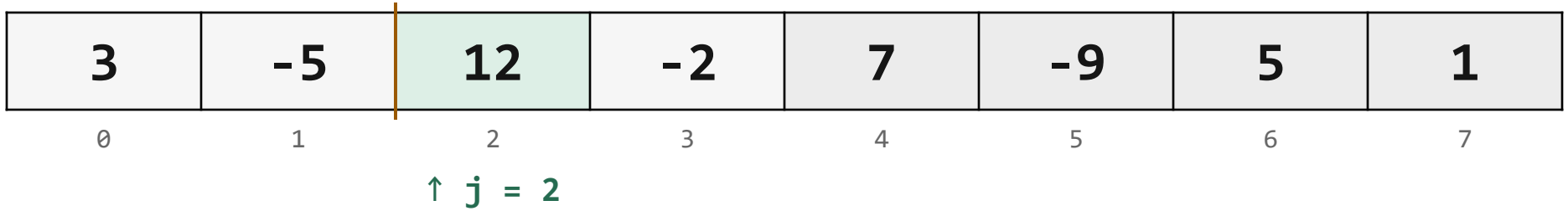
left_sum = -2 (보관)                    right_sum = -∞
```

오른쪽 첫 칸은 a[2] = 12입니다. mid를 다시 읽지 않습니다.

# [0~3] 오른쪽 스캔 1/2

j = 2, a[2] = 12을 더합니다. 현재 구간은 [2~2]입니다.
현재 합 12 > 이전 기록 −∞ → right_sum 갱신.

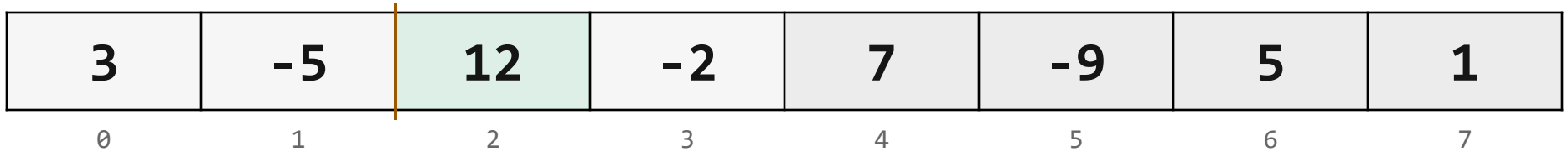
현재 구간




```
current_sum = 0 + (12) = 12
```

```
right_sum: -∞ → 12
```

저장한 최선

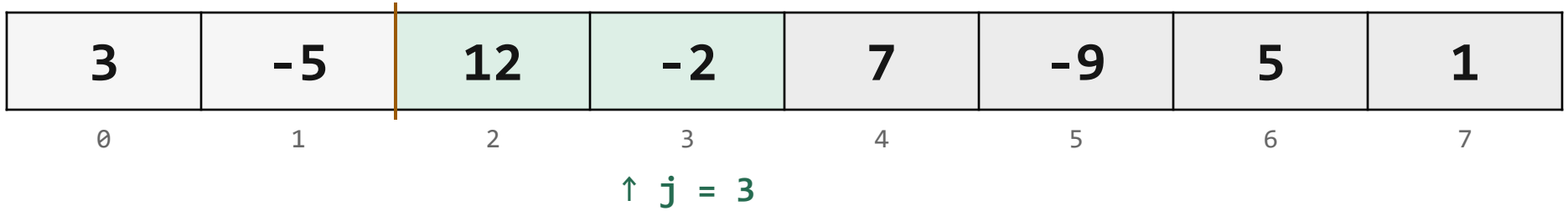



다음 j = 3: a[3] = −2을 현재 합 12에 더합니다.

# [0~3] 오른쪽 스캔 2/2

j = 3, a[3] = −2을 더합니다. 현재 구간은 [2~3]입니다.
현재 합 10 ≤ 이전 기록 12 → right_sum 유지.

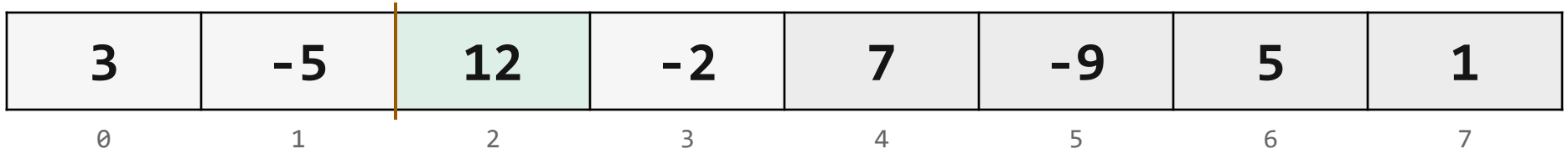
**현재 구간**




**current_sum = 12 + (-2) = 10**

**right_sum = 12 유지**

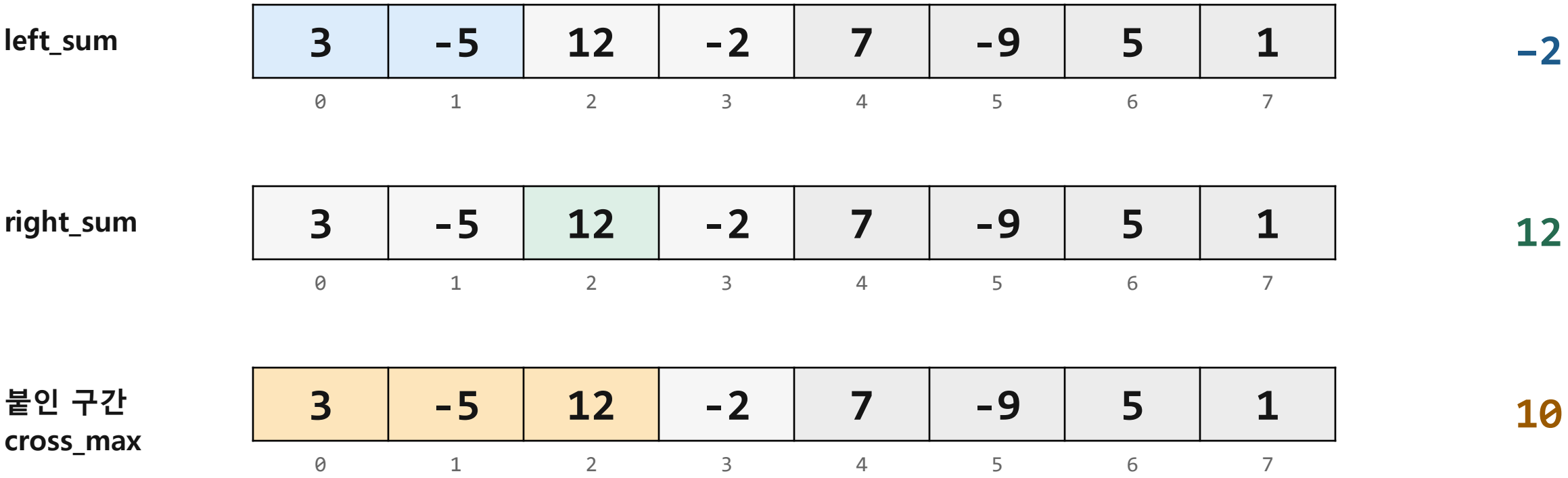
**저장한 최선**




오른쪽 스캔 끝. 왼쪽 기록과 오른쪽 기록을 더해 cross_max를 구합니다.

# [0~3] 경계에서 두 기록을 붙입니다

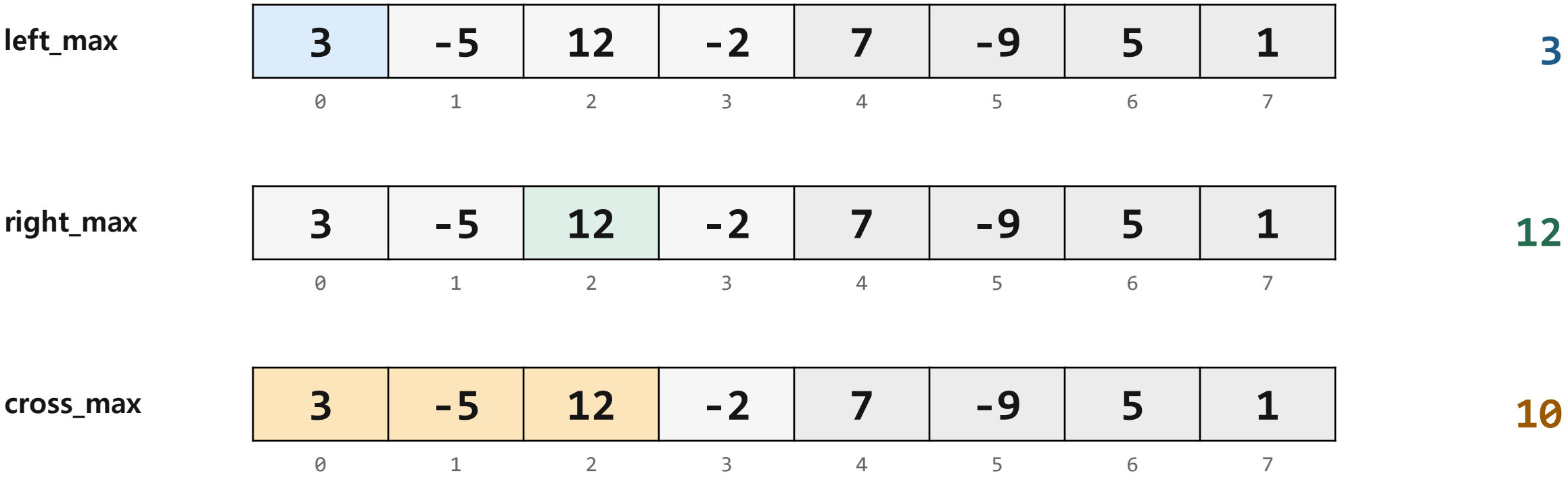
left_sum = −2, right_sum = 12이므로 cross_max = 10입니다.
두 구간은 인덱스 1와 2에서 맞닿아 빈틈 없이 이어집니다.




−2 + (12) = 10. 마지막에 읽은 합이 아니라 최선의 기록을 더합니다.

# [0~3] 세 후보 비교 → 12 반환

왼쪽 3, 오른쪽 12, 경계를 걸친 합 10을 비교합니다.
max(3, 12, 10) = 12입니다.

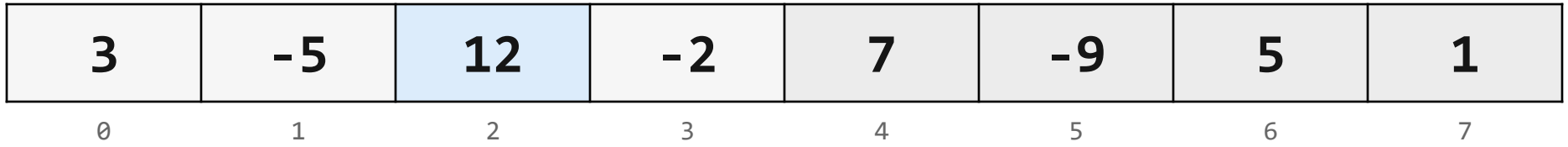



[0~7] 부모의 left_max에 12을 반환합니다.

# [0~7] 왼쪽 답 12을 받음

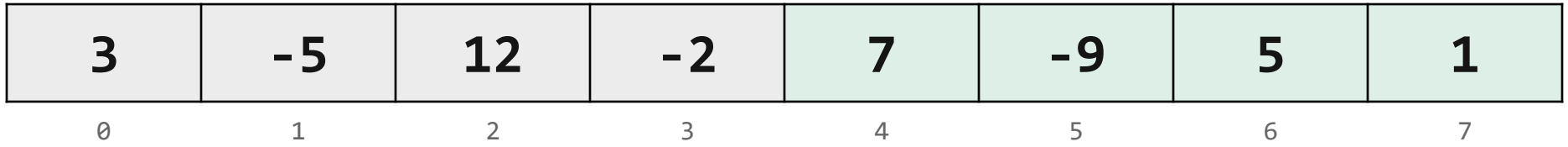
왼쪽 [0~3] 호출이 12을 반환했습니다. left_max = 12입니다.
이제 오른쪽 [4~7] 호출을 시작합니다.

**left_max = 12**




다음 호출: `max_subarray_sum(a, 4, 7)`

**다음 오른쪽**

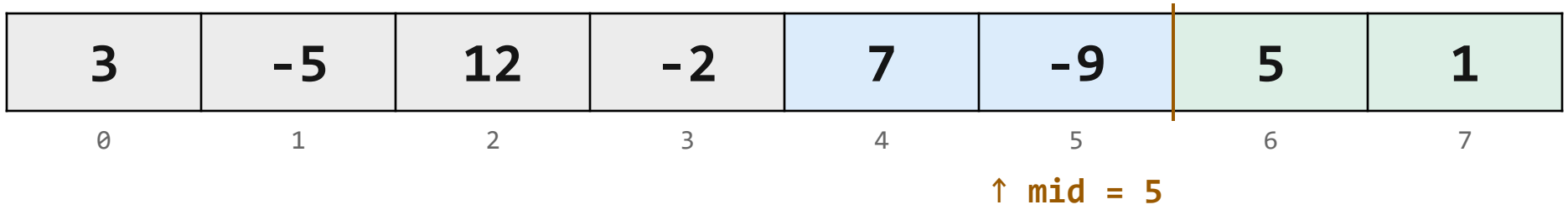



left_max를 저장해 두고 오른쪽 호출이 끝날 때까지 기다립니다.

# [4~7] 호출: mid = 5에서 나눔

현재 범위 low = 4, high = 7, mid = 5입니다.
먼저 왼쪽 [4~5]를 호출합니다. 오른쪽 [6~7]는 기다립니다.

**현재 호출**

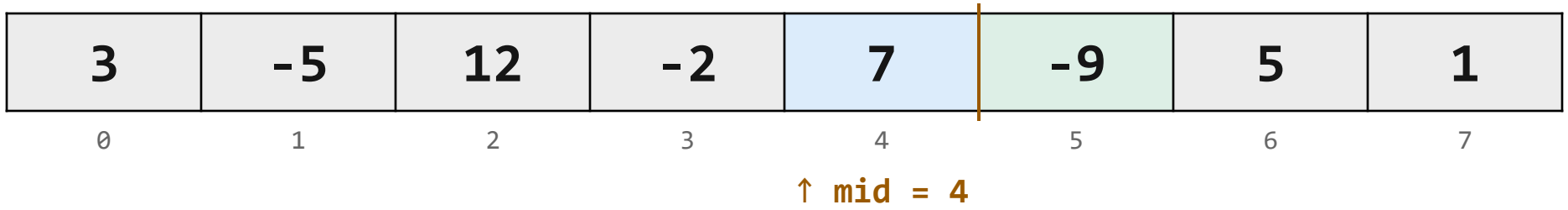



```
left_max  = max_subarray_sum(a, 4, 5)
right_max = max_subarray_sum(a, 6, 7)
```

[0~7] 부모는 right_max의 반환을 기다립니다.

# [4~5] 호출: mid = 4에서 나눔

현재 범위 low = 4, high = 5, mid = 4입니다.
먼저 왼쪽 [4~4]를 호출합니다. 오른쪽 [5~5]는 기다립니다.



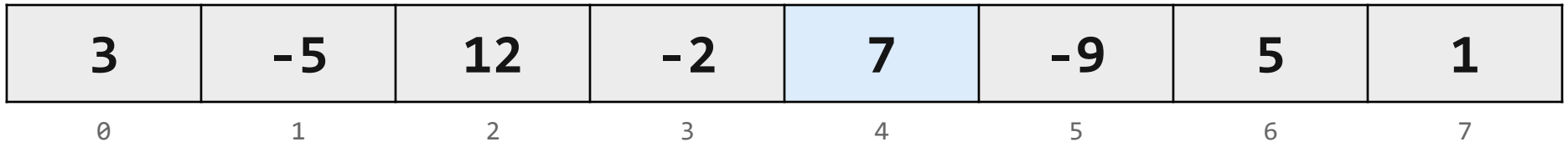

```
left_max  = max_subarray_sum(a, 4, 4)
right_max = max_subarray_sum(a, 5, 5)
```

[4~7] 부모는 left_max의 반환을 기다립니다.

# [4~4] 원소 하나: 7 반환

low = high = 4입니다. 남은 원소는 a[4] = 7 하나입니다.
이 값 7을 [4~5] 호출의 left_max에 돌려줍니다.

**현재 호출**



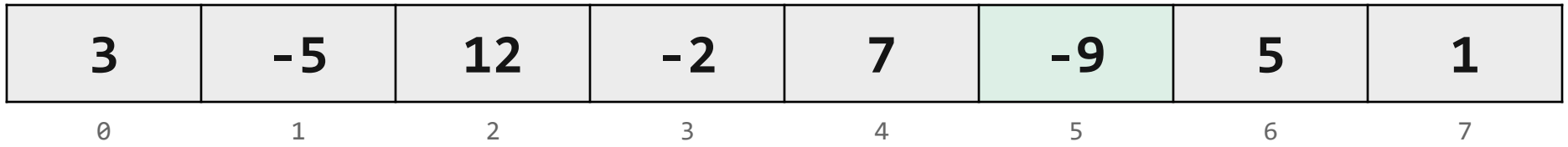

```
if low == high:
    return a[low]     # 7
```

[4~5] 호출의 left_max = 7. 원소 한 개의 호출이 끝났습니다.

# [5~5] 원소 하나: −9 반환

low = high = 5입니다. 남은 원소는 a[5] = −9 하나입니다.
이 값 −9을 [4~5] 호출의 right_max에 돌려줍니다.

**현재 호출**



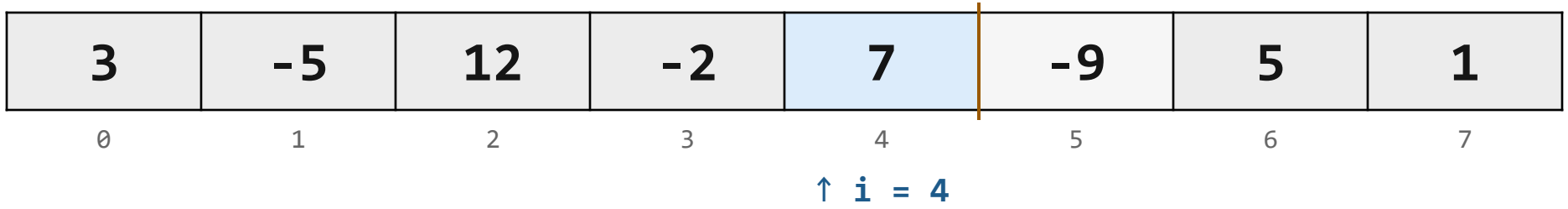

```
if low == high:
    return a[low]     # -9
```

[4~5] 호출의 right_max = −9. 원소 한 개의 호출이 끝났습니다.

# [4~5] 왼쪽 스캔 1/1

i = 4, a[4] = 7을 더합니다. 현재 구간은 [4~4]입니다.
현재 합 7 > 이전 기록 −∞ → left_sum 갱신.

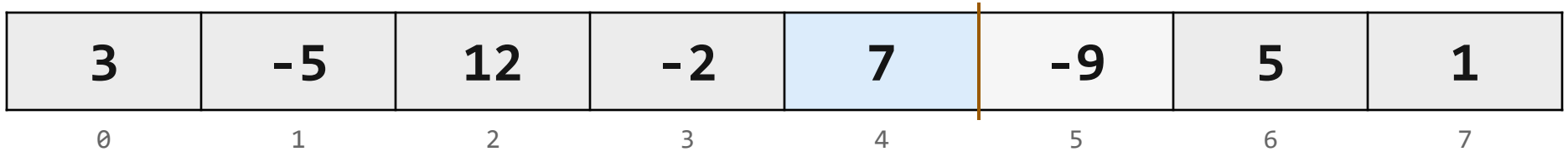
**현재 구간**




```
current_sum = 0 + (7) = 7
```

```
left_sum: -∞ → 7
```

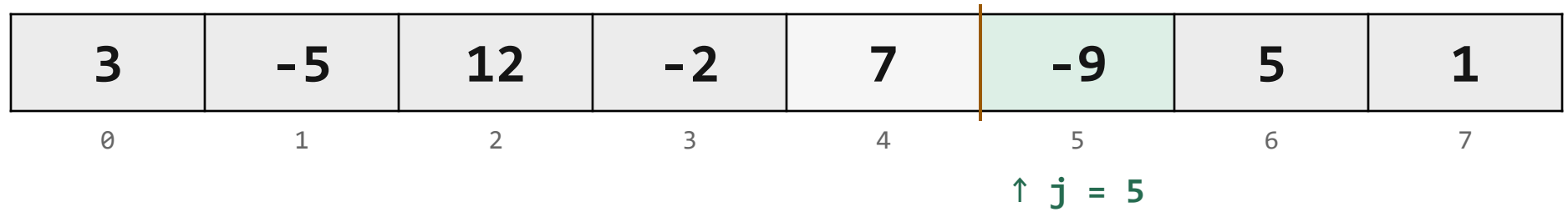
**저장한 최선**




왼쪽 스캔 끝. 이 기록을 보관하고 오른쪽 현재 합을 0으로 다시 시작합니다.

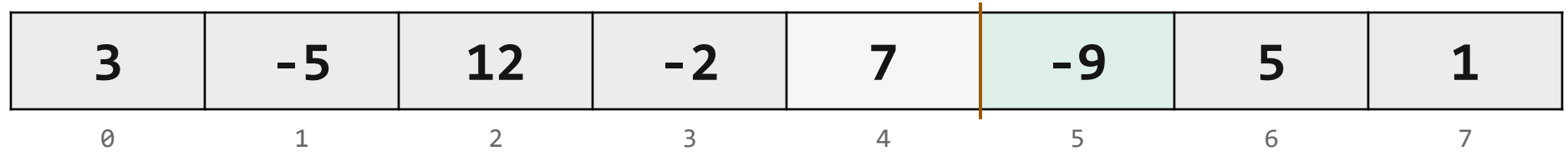
# [4~5] 오른쪽 스캔 1/1

j = 5, a[5] = −9을 더합니다. 현재 구간은 [5~5]입니다.
현재 합 −9 > 이전 기록 −∞ → right_sum 갱신.




```
current_sum = 0 + (-9) = -9
```

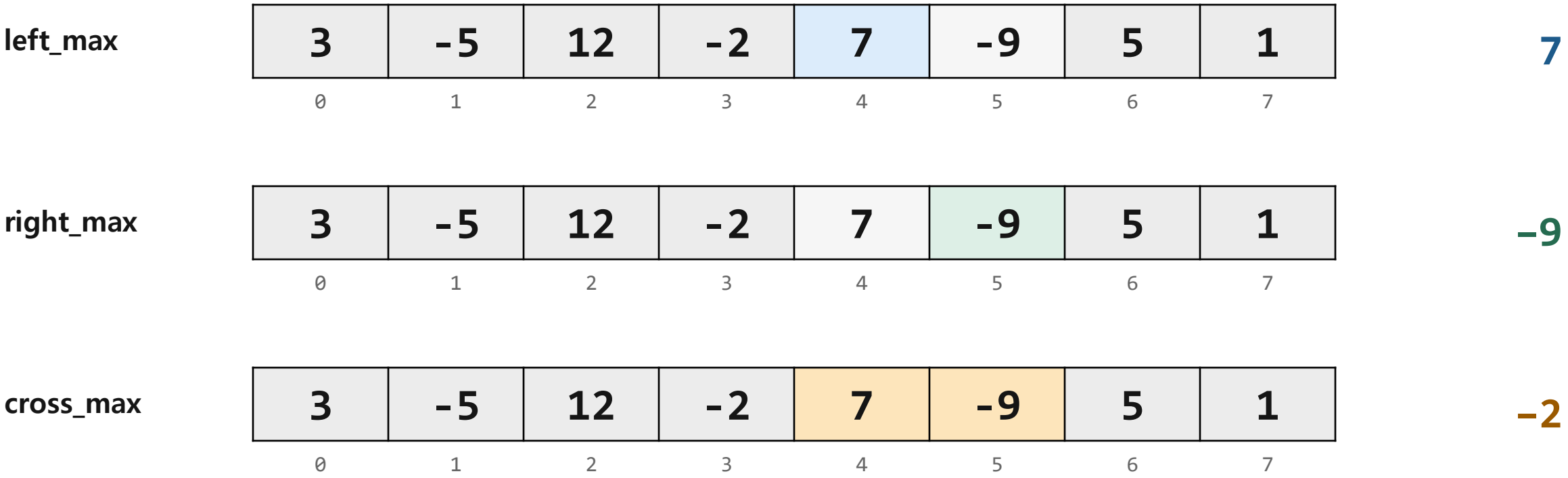
```
right_sum: -∞ → -9
```




오른쪽 스캔 끝. 왼쪽 기록과 오른쪽 기록을 더해 cross_max를 구합니다.

# [4~5] 세 후보 비교 → 7 반환

왼쪽 7, 오른쪽 −9, 경계를 걸친 합 −2을 비교합니다.
max(7, −9, −2) = 7입니다.

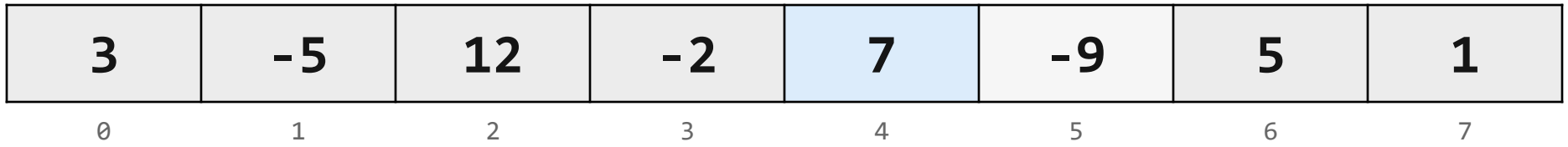



[4~7] 부모의 left_max에 7을 반환합니다.

# [4~7] 왼쪽 답 7을 받음

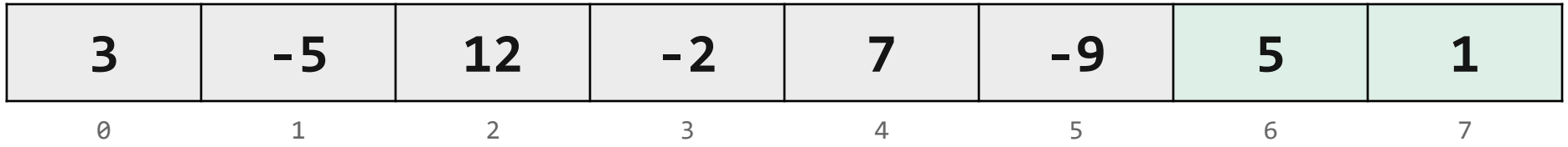
왼쪽 [4~5] 호출이 7을 반환했습니다. left_max = 7입니다.
이제 오른쪽 [6~7] 호출을 시작합니다.

**left_max = 7**




다음 호출: `max_subarray_sum(a, 6, 7)`

**다음 오른쪽**

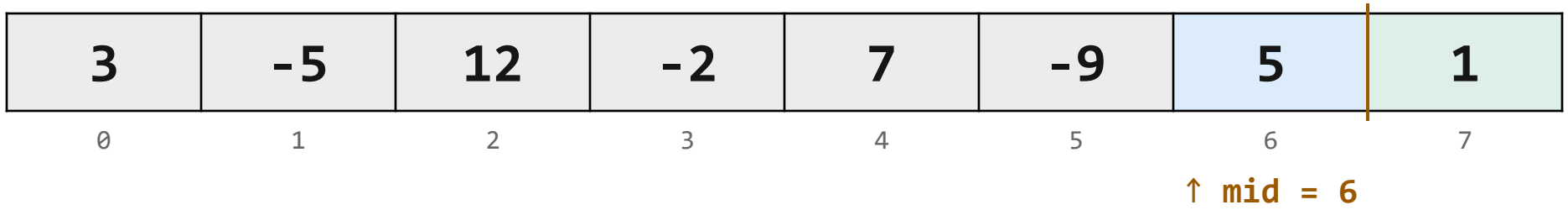



left_max를 저장해 두고 오른쪽 호출이 끝날 때까지 기다립니다.

# [6~7] 호출: mid = 6에서 나눔

현재 범위 low = 6, high = 7, mid = 6입니다.
먼저 왼쪽 [6~6]를 호출합니다. 오른쪽 [7~7]는 기다립니다.

**현재 호출**



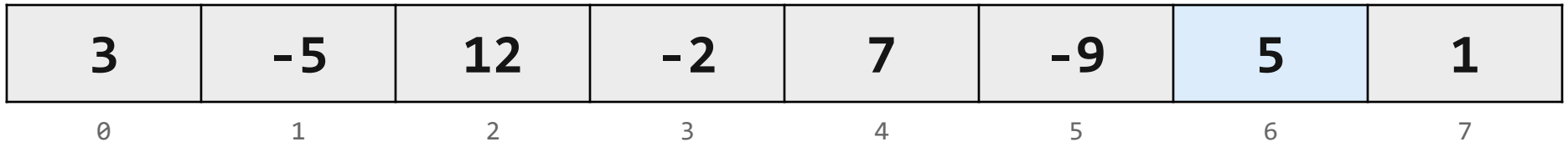

```
left_max  = max_subarray_sum(a, 6, 6)
right_max = max_subarray_sum(a, 7, 7)
```

[4~7] 부모는 right_max의 반환을 기다립니다.

# [6~6] 원소 하나: 5 반환

low = high = 6입니다. 남은 원소는 a[6] = 5 하나입니다.
이 값 5을 [6~7] 호출의 left_max에 돌려줍니다.

**현재 호출**



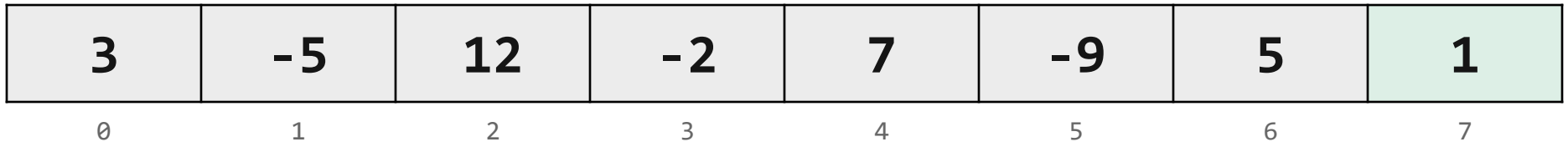

```
if low == high:
    return a[low]     # 5
```

[6~7] 호출의 left_max = 5. 원소 한 개의 호출이 끝났습니다.

# [7~7] 원소 하나: 1 반환

low = high = 7입니다. 남은 원소는 a[7] = 1 하나입니다.
이 값 1을 [6~7] 호출의 right_max에 돌려줍니다.

**현재 호출**



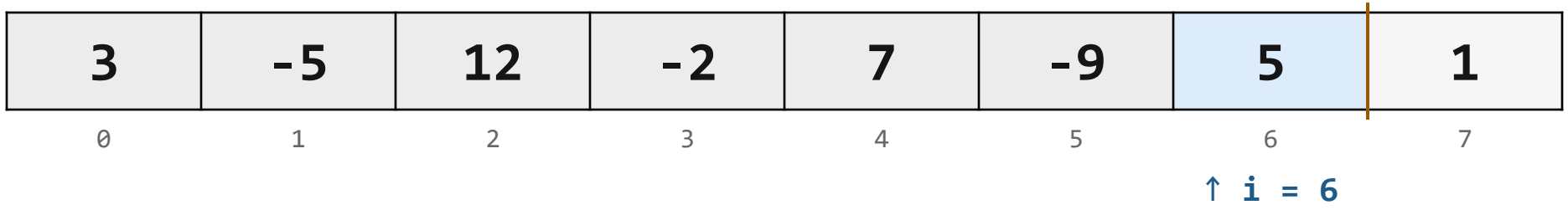

```
if low == high:
    return a[low]     # 1
```

[6~7] 호출의 right_max = 1. 원소 한 개의 호출이 끝났습니다.

# [6~7] 왼쪽 스캔 1/1

i = 6, a[6] = 5을 더합니다. 현재 구간은 [6~6]입니다.
현재 합 5 > 이전 기록 −∞ → left_sum 갱신.

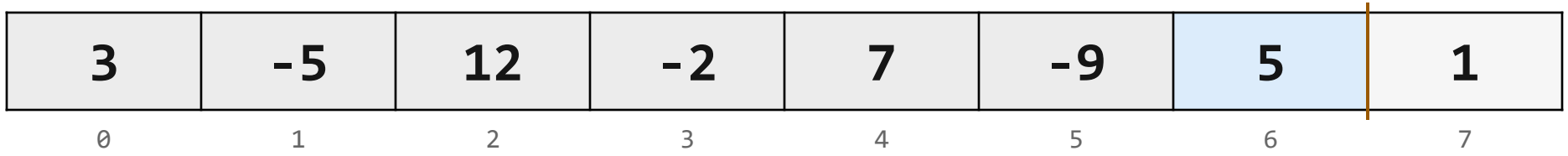
**현재 구간**




**current_sum = 0 + (5) = 5**

**left_sum: −∞ → 5**

**저장한 최선**

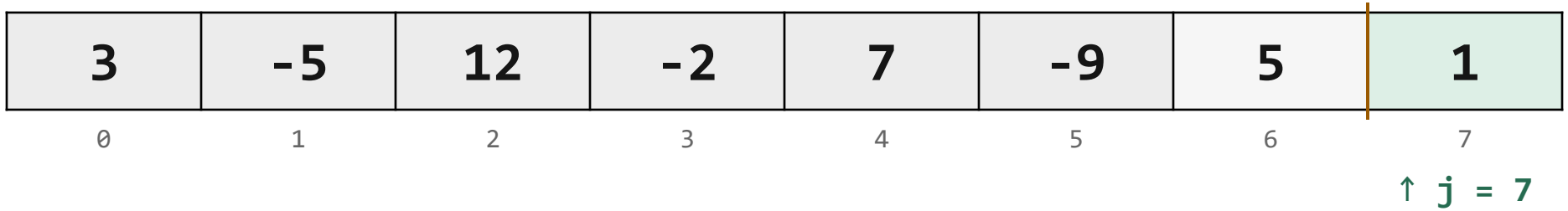



왼쪽 스캔 끝. 이 기록을 보관하고 오른쪽 현재 합을 0으로 다시 시작합니다.

# [6~7] 오른쪽 스캔 1/1

j = 7, a[7] = 1을 더합니다. 현재 구간은 [7~7]입니다.
현재 합 1 > 이전 기록 −∞ → right_sum 갱신.

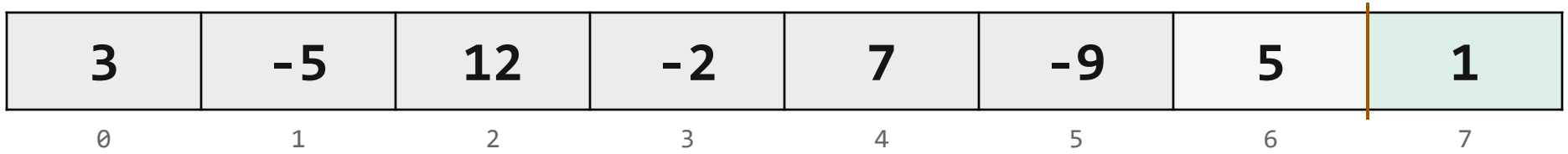
**현재 구간**




**current_sum = 0 + (1) = 1**

**right_sum: −∞ → 1**

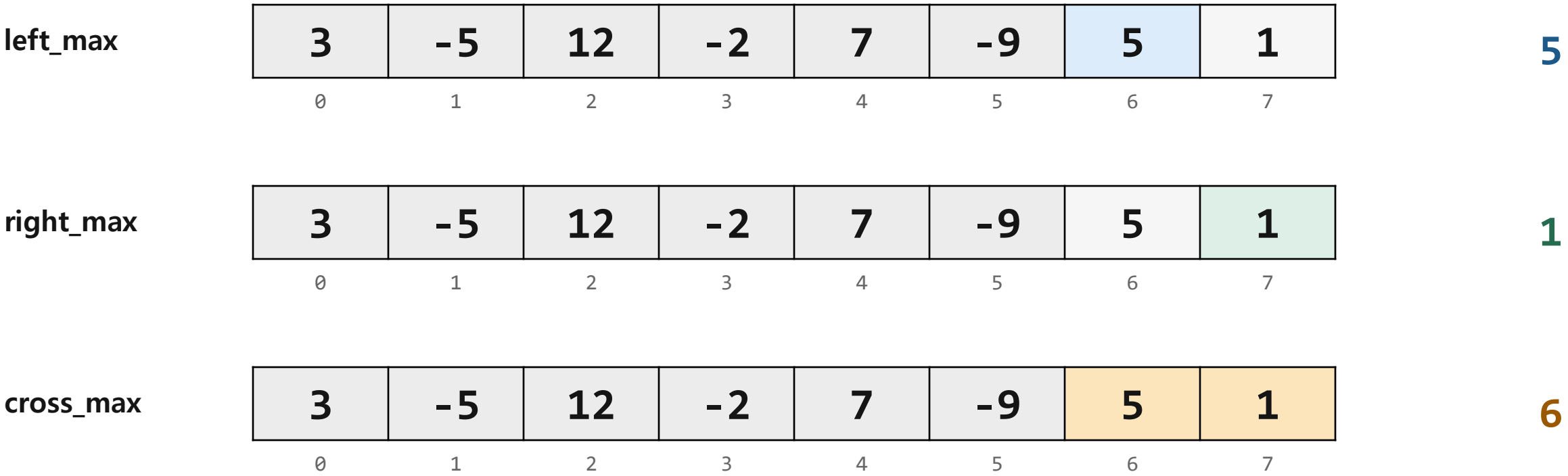
**저장한 최선**




오른쪽 스캔 끝. 왼쪽 기록과 오른쪽 기록을 더해 cross_max를 구합니다.

# [6~7] 세 후보 비교 → 6 반환

왼쪽 5, 오른쪽 1, 경계를 걸친 합 6을 비교합니다.
max(5, 1, 6) = 6입니다.

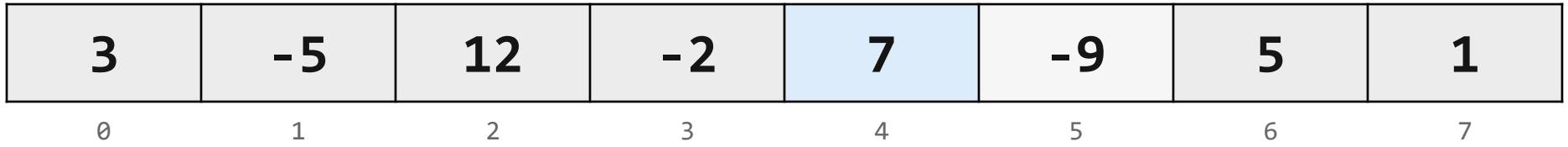



[4~7] 부모의 right_max에 6을 반환합니다.

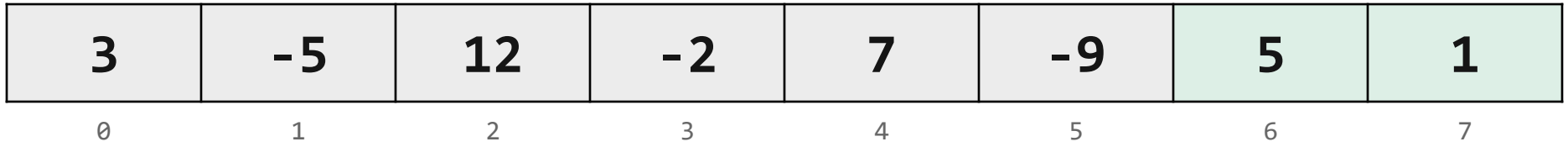
# [4~7] 오른쪽 답 6도 받음

left_max = 7, right_max = 6입니다. 두 재귀 호출을 마쳤습니다.
아직 경계를 걸치는 cross_max를 계산하고 비교해야 합니다.

**left_max = 7**




**right_max = 6**

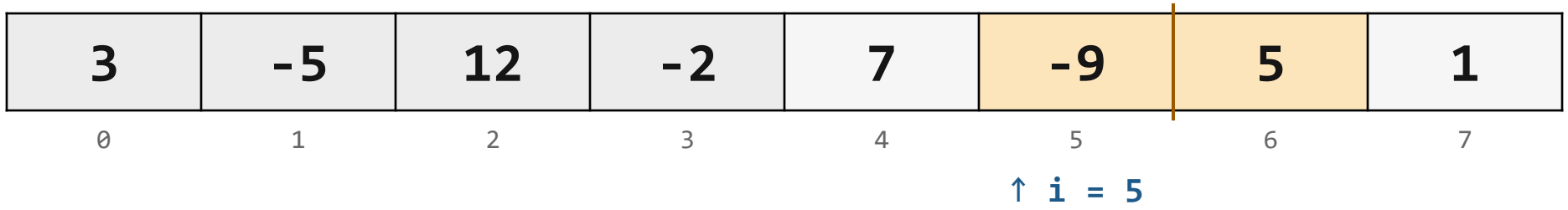



다음은 경계를 걸치는 후보입니다. left_max + right_max를 계산하지 않습니다.

# [4~7] 이제 경계를 걸치는 합 계산

mid = 5입니다. 왼쪽 스캔은 a[5] = −9부터 시작합니다.
current_sum = 0, left_sum = −∞로 두고 i를 왼쪽으로 움직입니다.

**현재 경계**



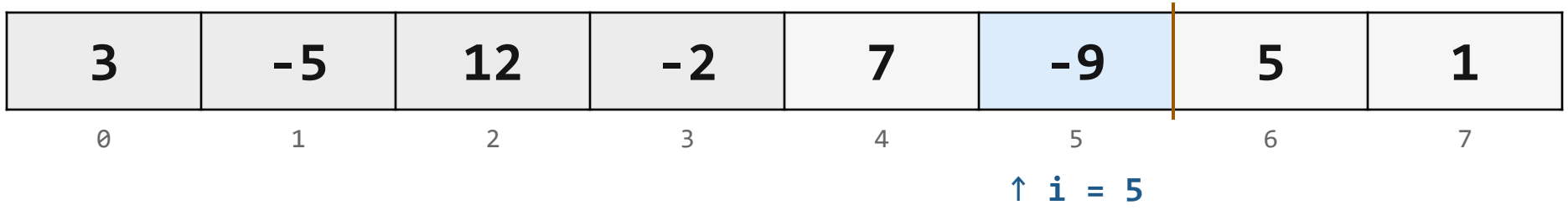

```
current_sum = 0        left_sum = -∞
```

재귀 답은 유지: left_max = 7, right_max = 6.

# [4~7] 왼쪽 스캔 1/2

i = 5, a[5] = −9을 더합니다. 현재 구간은 [5~5]입니다.
현재 합 −9 > 이전 기록 −∞ → left_sum 갱신.

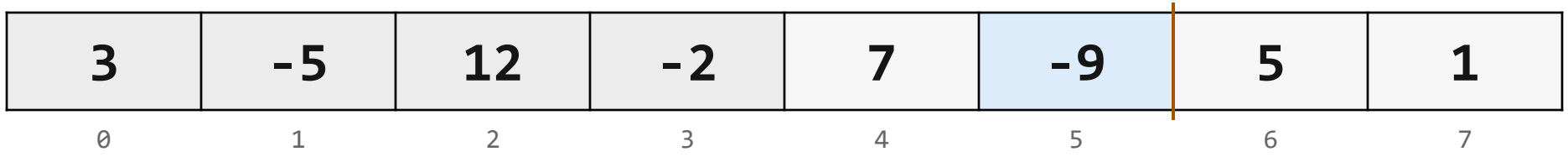
**현재 구간**




```
current_sum = 0 + (-9) = -9
```

```
left_sum: -∞ → -9
```

**저장한 최선**

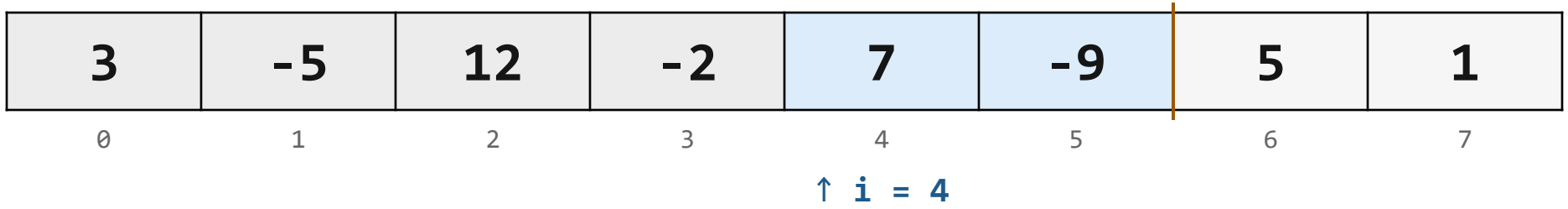



다음 i = 4: a[4] = 7을 현재 합 −9에 더합니다.

# [4~7] 왼쪽 스캔 2/2

i = 4, a[4] = 7을 더합니다. 현재 구간은 [4~5]입니다.
현재 합 −2 > 이전 기록 −9 → left_sum 갱신.

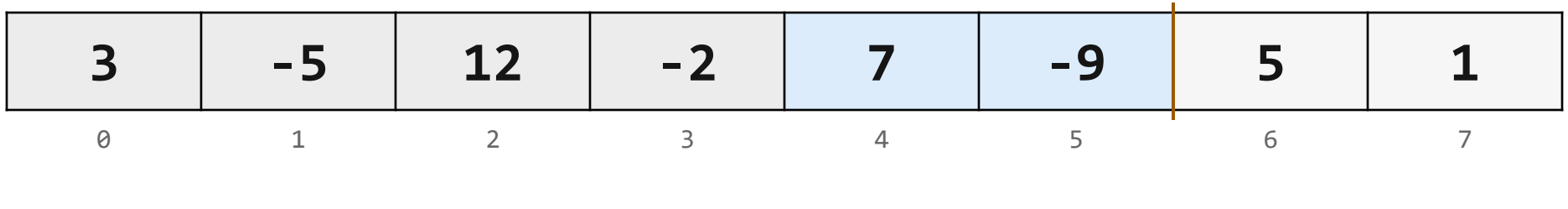
**현재 구간**




**current_sum = -9 + (7) = -2**

**left_sum: -9 → -2**

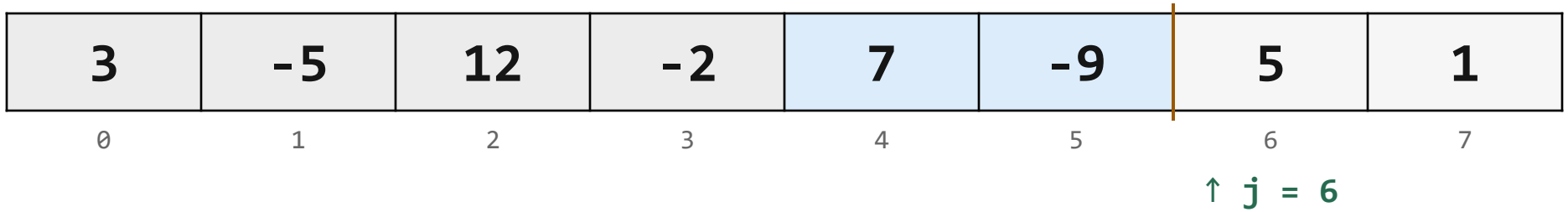
**저장한 최선**




왼쪽 스캔 끝. 이 기록을 보관하고 오른쪽 현재 합을 0으로 다시 시작합니다.

# [4~7] 오른쪽은 합을 0으로 다시 시작

왼쪽 기록 left_sum = −2은 보관합니다. current_sum만 0으로 바꿉니다.
right_sum = −∞로 두고 j = mid + 1 = 6부터 오른쪽을 훑습니다.




```
current_sum = 0

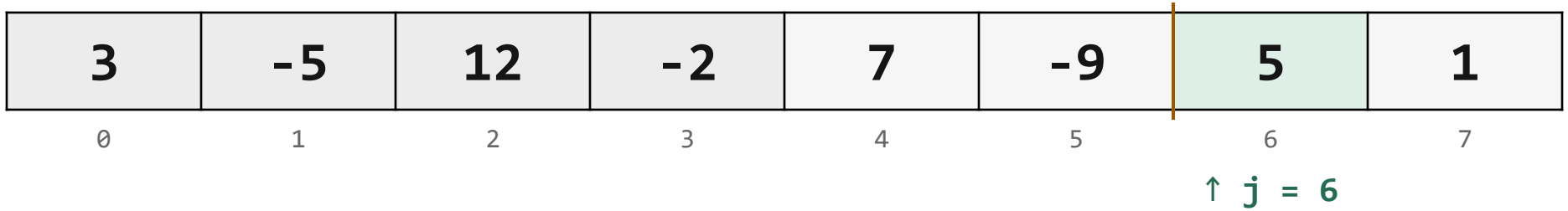
left_sum = -2 (보관)                    right_sum = -∞
```

오른쪽 첫 칸은 a[6] = 5입니다. mid를 다시 읽지 않습니다.

# [4~7] 오른쪽 스캔 1/2

j = 6, a[6] = 5을 더합니다. 현재 구간은 [6~6]입니다.
현재 합 5 > 이전 기록 −∞ → right_sum 갱신.

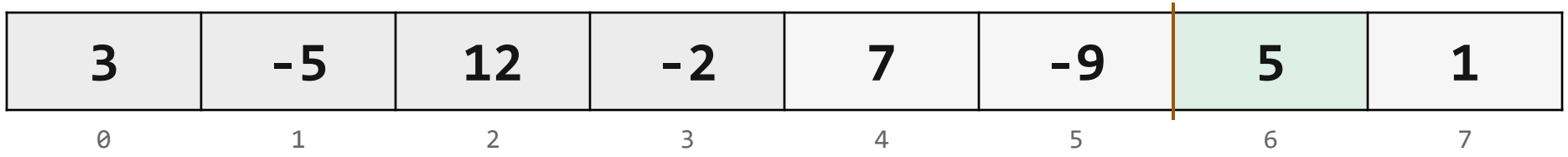
**현재 구간**




```
current_sum = 0 + (5) = 5
```

```
right_sum: -∞ → 5
```

**저장한 최선**

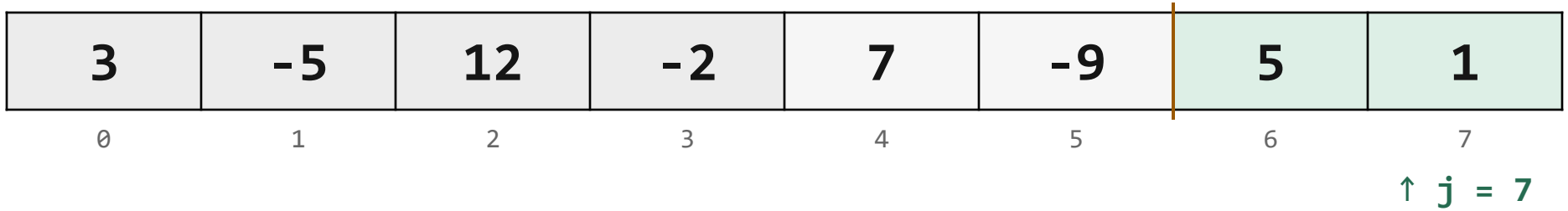



다음 j = 7: a[7] = 1을 현재 합 5에 더합니다.

# [4~7] 오른쪽 스캔 2/2

j = 7, a[7] = 1을 더합니다. 현재 구간은 [6~7]입니다.
현재 합 6 > 이전 기록 5 → right_sum 갱신.

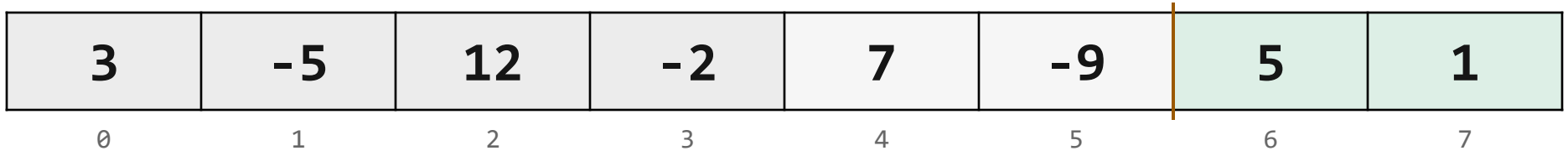
**현재 구간**




**current_sum = 5 + (1) = 6**

**right_sum: 5 → 6**

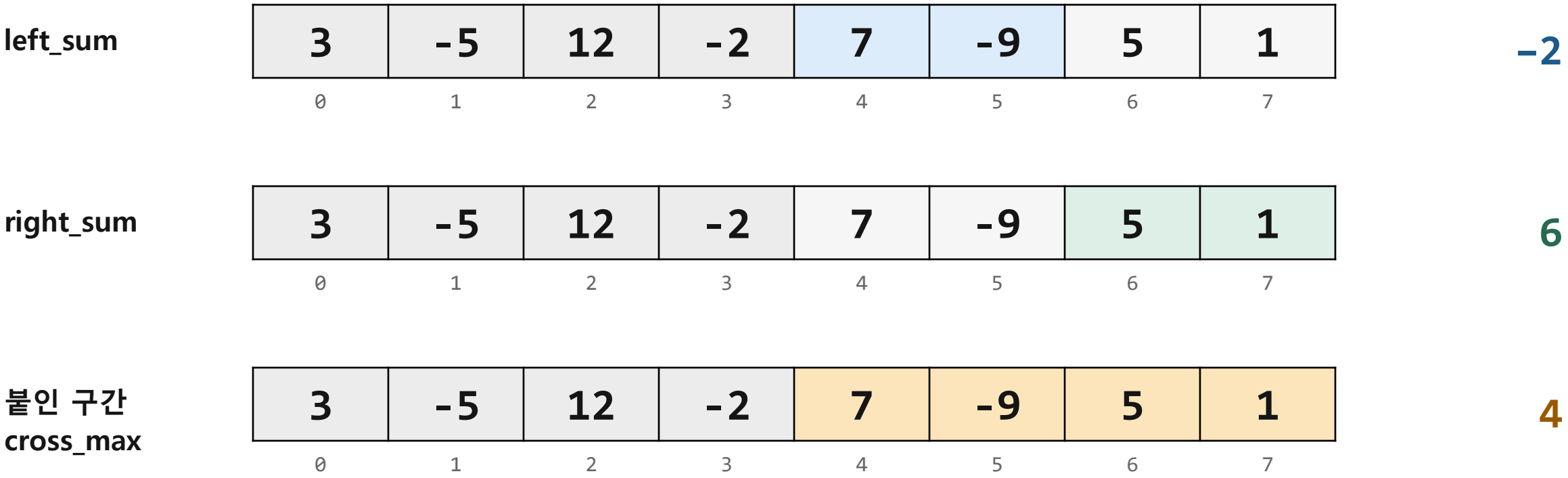
**저장한 최선**




오른쪽 스캔 끝. 왼쪽 기록과 오른쪽 기록을 더해 cross_max를 구합니다.

# [4~7] 경계에서 두 기록을 붙입니다

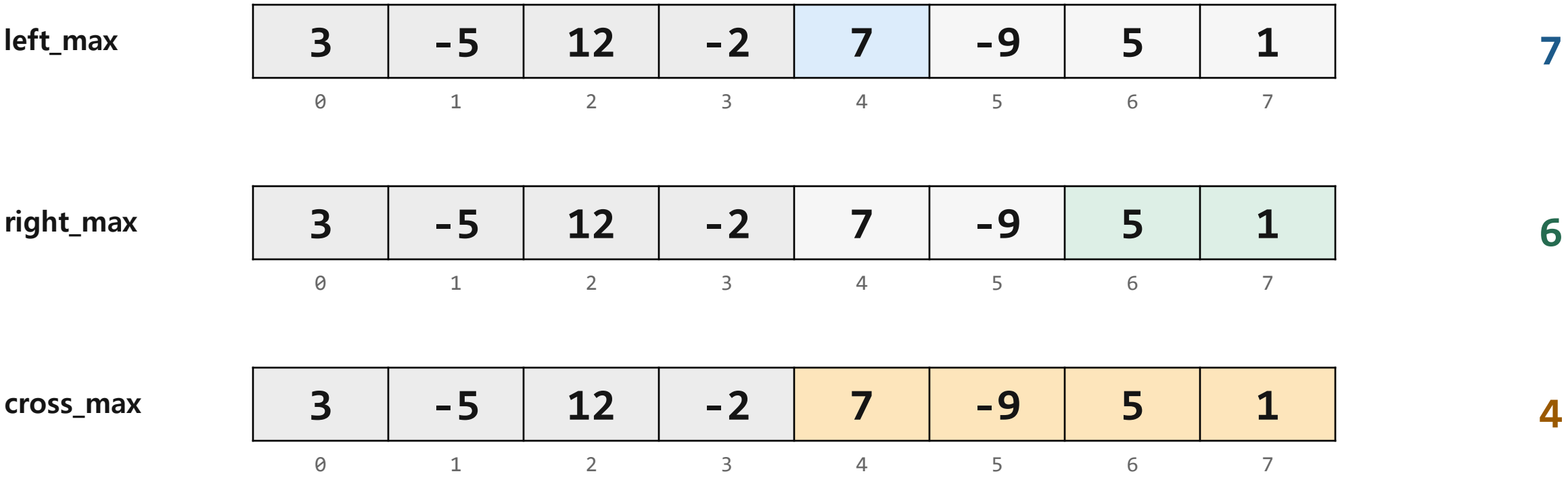
left_sum = −2, right_sum = 6이므로 cross_max = 4입니다.
두 구간은 인덱스 5와 6에서 맞닿아 빈틈 없이 이어집니다.




−2 + (6) = 4. 마지막에 읽은 합이 아니라 최선의 기록을 더합니다.

# [4~7] 세 후보 비교 → 7 반환

왼쪽 7, 오른쪽 6, 경계를 걸친 합 4을 비교합니다.
max(7, 6, 4) = 7입니다.

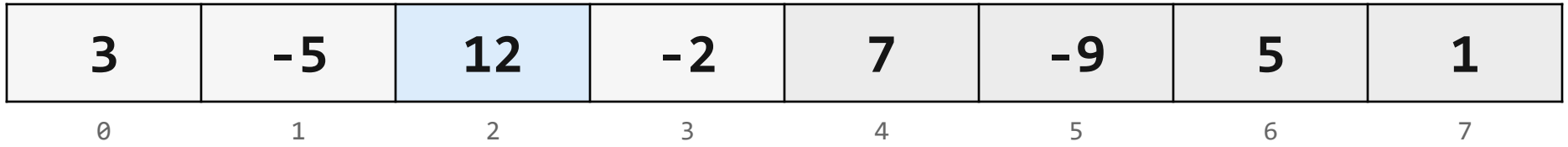



[0~7] 부모의 right_max에 7을 반환합니다.

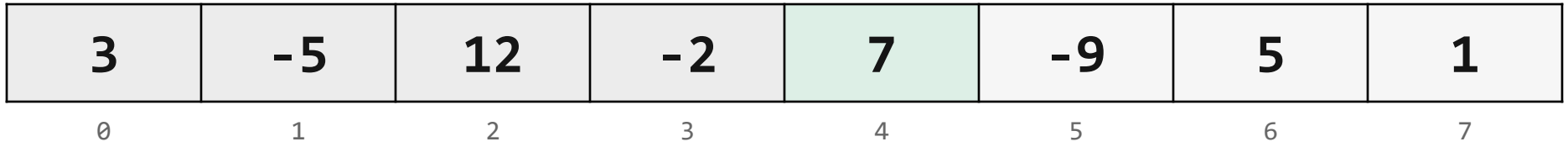
# [0~7] 오른쪽 답 7도 받음

left_max = 12, right_max = 7입니다. 두 재귀 호출을 마쳤습니다.
아직 경계를 걸치는 cross_max를 계산하고 비교해야 합니다.

**left_max = 12**




**right_max = 7**




다음은 경계를 걸치는 후보입니다. left_max + right_max를 계산하지 않습니다.

# 중앙을 걸친 구간: 10 + 7 = 17

mid = 3의 −2에서 왼쪽으로, mid+1 = 4의 7에서 오른쪽으로 훑습니다.

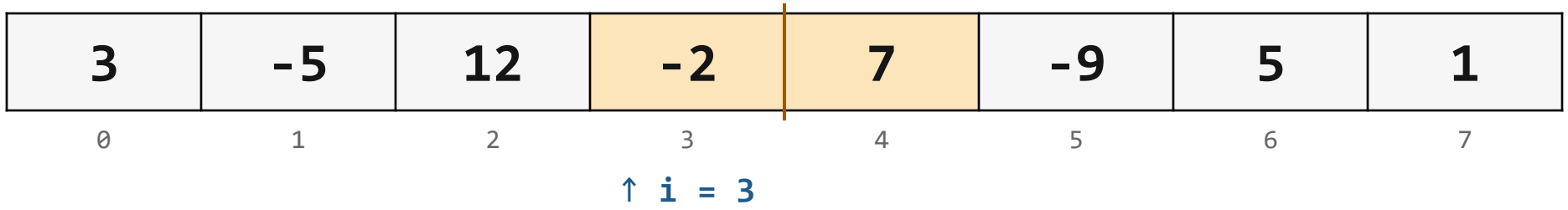



그림의 최대 합은 중앙에서 훑으며 저장한 합입니다. 절반 전체의 left_max·right_max와 구분하세요.

# [0~7] 이제 경계를 걸치는 합 계산

mid = 3입니다. 왼쪽 스캔은 a[3] = −2부터 시작합니다.
current_sum = 0, left_sum = −∞로 두고 i를 왼쪽으로 움직입니다.

**현재 경계**



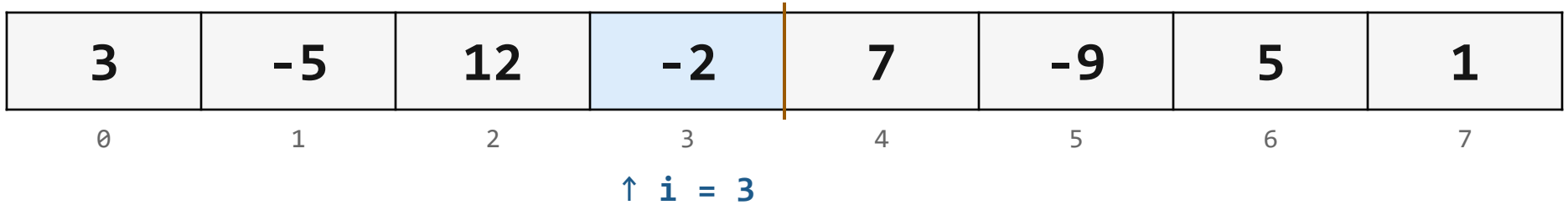

```
current_sum = 0        left_sum = -∞
```

재귀 답은 유지: left_max = 12, right_max = 7.

# [0~7] 왼쪽 스캔 1/4

i = 3, a[3] = −2을 더합니다. 현재 구간은 [3~3]입니다.
현재 합 −2 > 이전 기록 −∞ → left_sum 갱신.

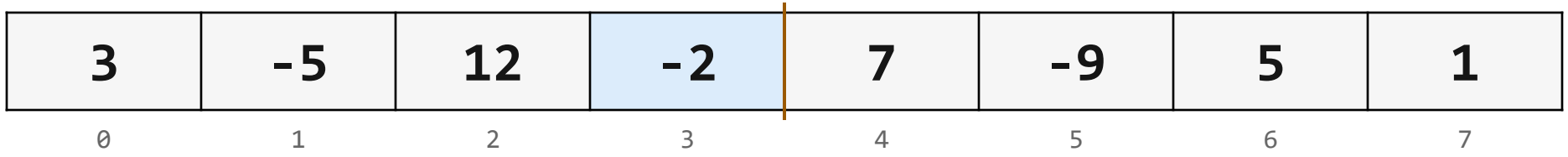
**현재 구간**




```
current_sum = 0 + (-2) = -2
```

```
left_sum: -∞ → -2
```

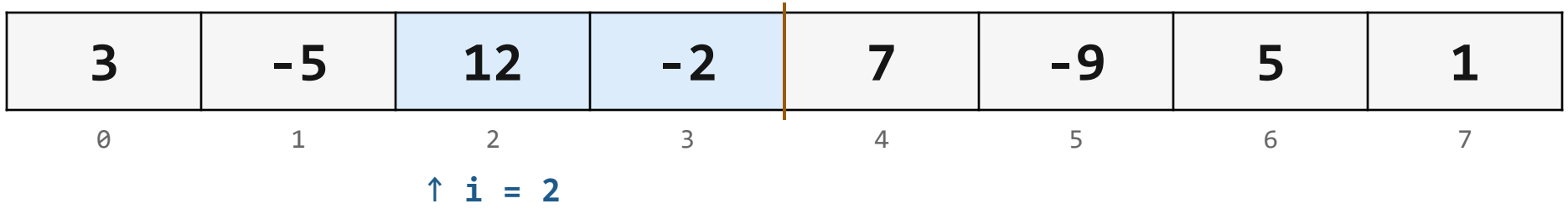
**저장한 최선**




다음 i = 2: a[2] = 12을 현재 합 −2에 더합니다.

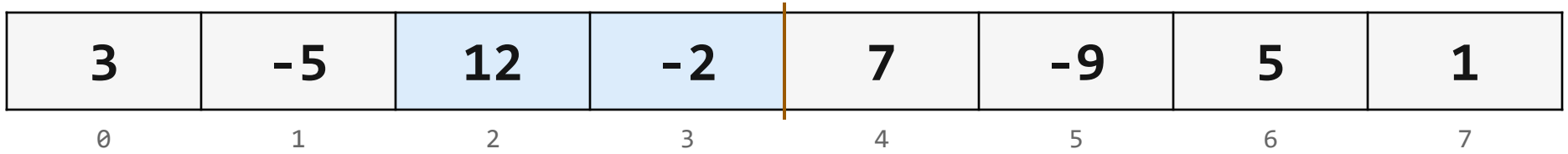
# [0~7] 왼쪽 스캔 2/4

i = 2, a[2] = 12을 더합니다. 현재 구간은 [2~3]입니다.
현재 합 10 > 이전 기록 −2 → left_sum 갱신.




**current_sum = -2 + (12) = 10**

**left_sum: -2 → 10**

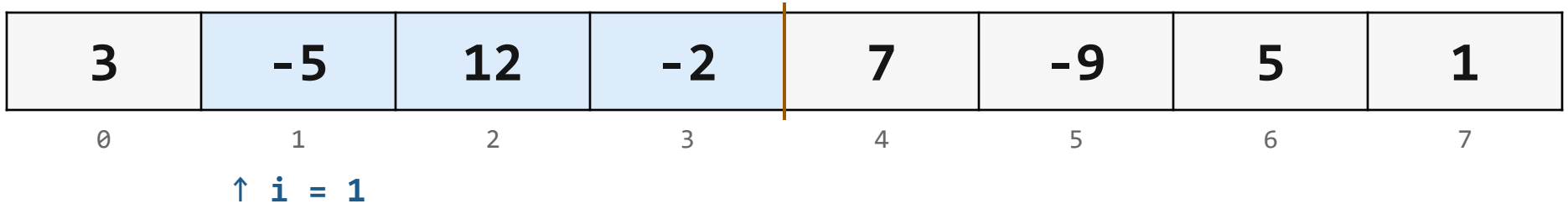



다음 i = 1: a[1] = −5을 현재 합 10에 더합니다.

# [0~7] 왼쪽 스캔 3/4

i = 1, a[1] = −5을 더합니다. 현재 구간은 [1~3]입니다.
현재 합 5 ≤ 이전 기록 10 → left_sum 유지.

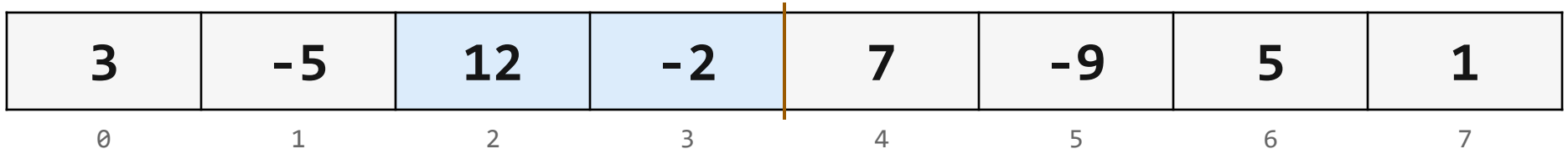
**현재 구간**




**current_sum = 10 + (-5) = 5**

**left_sum = 10 유지**

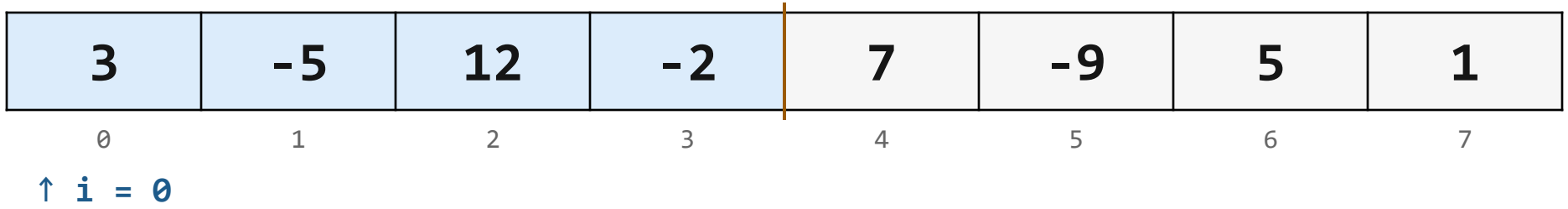
**저장한 최선**




다음 i = 0: a[0] = 3을 현재 합 5에 더합니다.

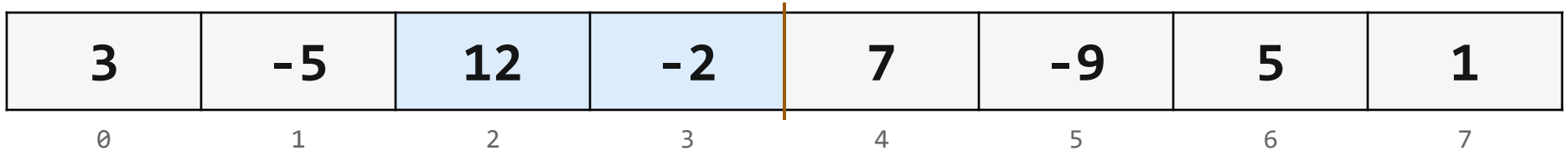
# [0~7] 왼쪽 스캔 4/4

i = 0, a[0] = 3을 더합니다. 현재 구간은 [0~3]입니다.
현재 합 8 ≤ 이전 기록 10 → left_sum 유지.




`current_sum = 5 + (3) = 8`

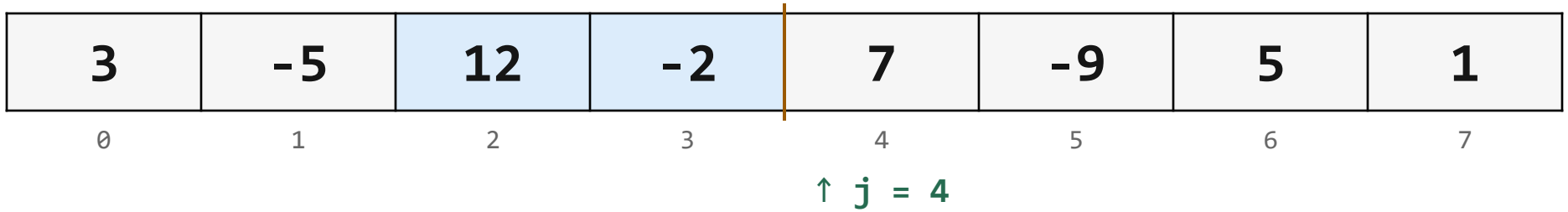
`left_sum = 10 유지`




왼쪽 스캔 끝. 이 기록을 보관하고 오른쪽 현재 합을 0으로 다시 시작합니다.

# [0~7] 오른쪽은 합을 0으로 다시 시작

왼쪽 기록 left_sum = 10은 보관합니다. current_sum만 0으로 바꿉니다.
right_sum = −∞로 두고 j = mid + 1 = 4부터 오른쪽을 훑습니다.




```
current_sum = 0

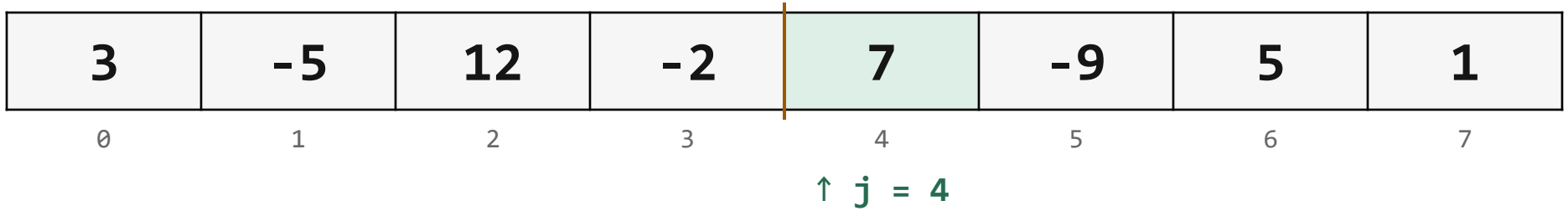
left_sum = 10 (보관)                    right_sum = −∞
```

오른쪽 첫 칸은 a[4] = 7입니다. mid를 다시 읽지 않습니다.

# [0~7] 오른쪽 스캔 1/4

j = 4, a[4] = 7을 더합니다. 현재 구간은 [4~4]입니다.
현재 합 7 > 이전 기록 −∞ → right_sum 갱신.

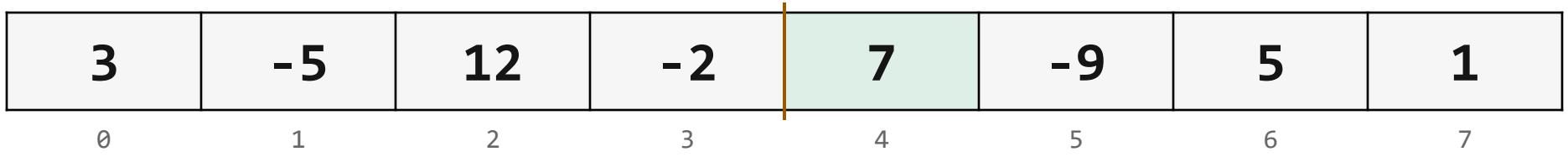
**현재 구간**




```
current_sum = 0 + (7) = 7
```

```
right_sum: -∞ → 7
```

**저장한 최선**

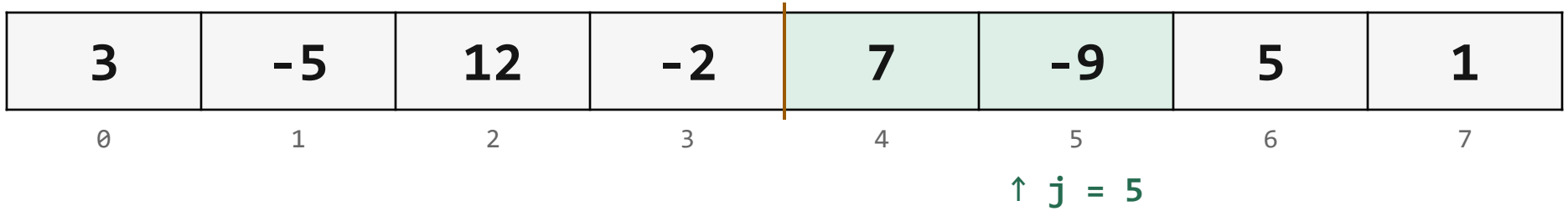



다음 j = 5: a[5] = −9을 현재 합 7에 더합니다.

# [0~7] 오른쪽 스캔 2/4

j = 5, a[5] = −9을 더합니다. 현재 구간은 [4~5]입니다.
현재 합 −2 ≤ 이전 기록 7 → right_sum 유지.

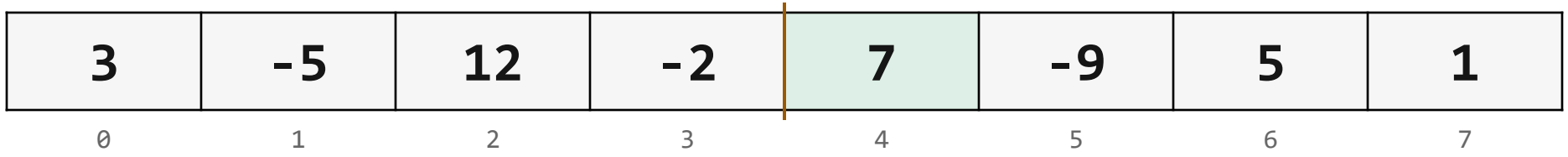
**현재 구간**




**current_sum = 7 + (-9) = -2**

**right_sum = 7 유지**

**저장한 최선**

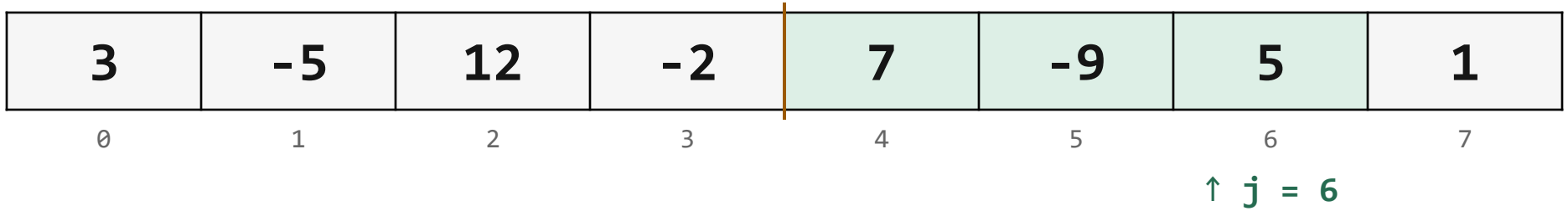



다음 j = 6: a[6] = 5을 현재 합 −2에 더합니다.

# [0~7] 오른쪽 스캔 3/4

j = 6, a[6] = 5을 더합니다. 현재 구간은 [4~6]입니다.
현재 합 3 ≤ 이전 기록 7 → right_sum 유지.

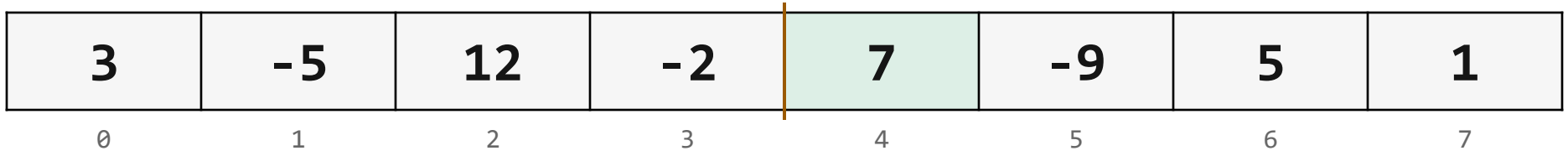
**현재 구간**




**current_sum = -2 + (5) = 3**

**right_sum = 7 유지**

**저장한 최선**

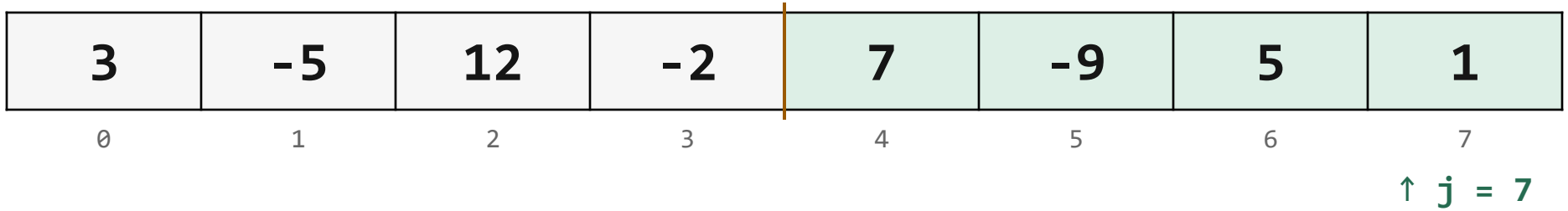



다음 j = 7: a[7] = 1을 현재 합 3에 더합니다.

# [0~7] 오른쪽 스캔 4/4

j = 7, a[7] = 1을 더합니다. 현재 구간은 [4~7]입니다.
현재 합 4 ≤ 이전 기록 7 → right_sum 유지.

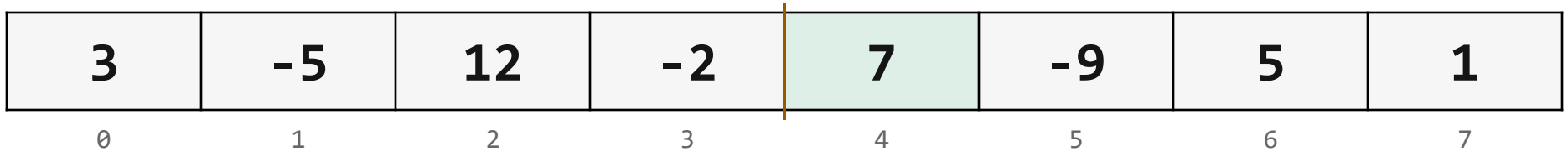
**현재 구간**




**current_sum = 3 + (1) = 4**

**right_sum = 7 유지**

**저장한 최선**

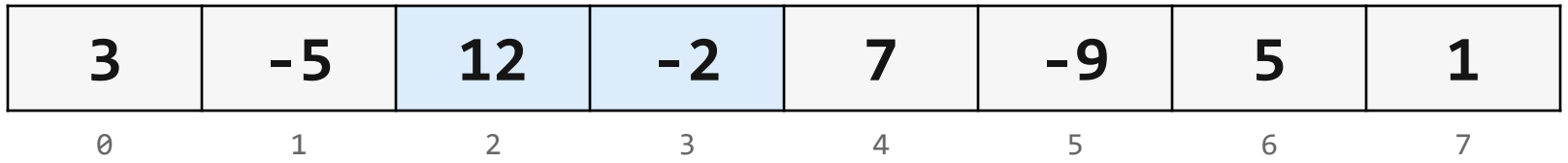



오른쪽 스캔 끝. 왼쪽 기록과 오른쪽 기록을 더해 cross_max를 구합니다.

# [0~7] 경계에서 두 기록을 붙입니다

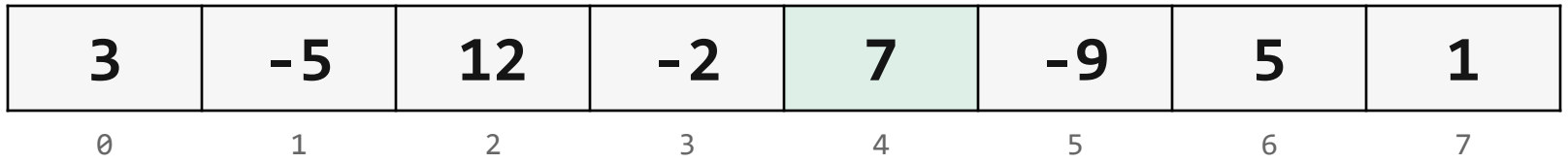
left_sum = 10, right_sum = 7이므로 cross_max = 17입니다.
두 구간은 인덱스 3와 4에서 맞닿아 빈틈 없이 이어집니다.

**left_sum**



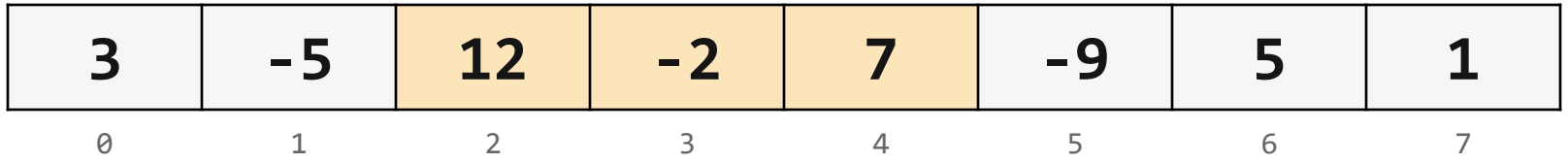

10

**right_sum**




7

**붙인 구간**
**cross_max**

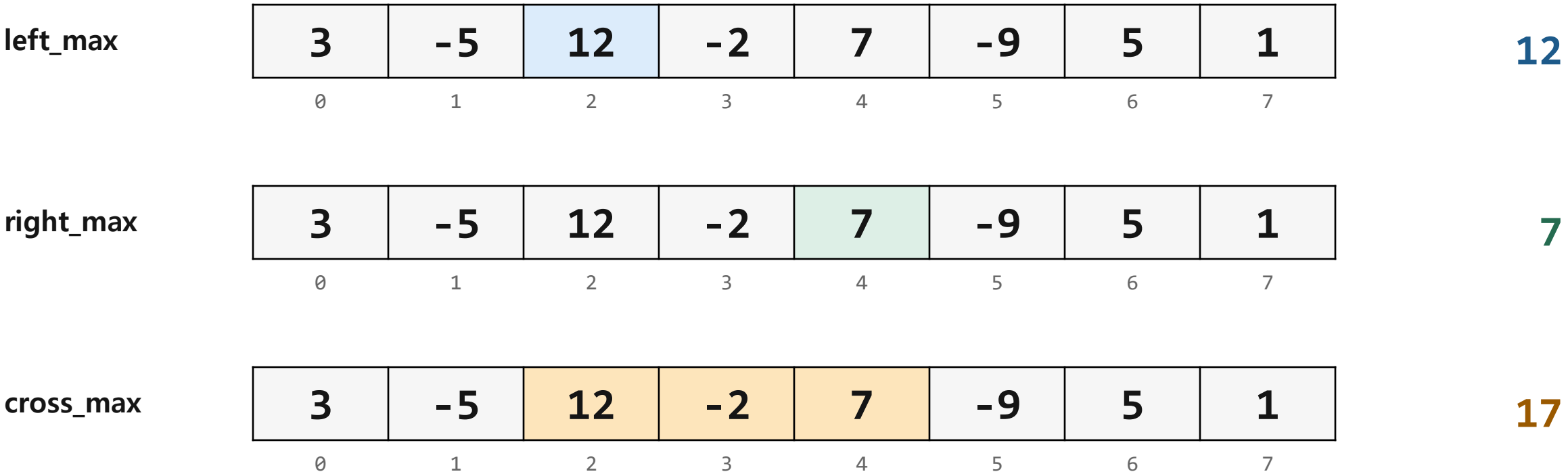



17

10 + (7) = 17. 마지막에 읽은 합이 아니라 최선의 기록을 더합니다.

# [0~7] 세 후보 비교 → 17 반환

왼쪽 12, 오른쪽 7, 경계를 걸친 합 17을 비교합니다.
max(12, 7, 17) = 17입니다.




전체 호출의 최종 답 17을 반환합니다.

# 전체 호출의 코드와 방금 본 값

각 호출은 왼쪽 답, 오른쪽 답, 걸친 답을 차례로 구하고 비교합니다.
원소 하나일 때만 재귀 호출 없이 그 값을 반환합니다.

```
def max_subarray_sum(a, low, high):
    if low == high:
        return a[low]
    mid = (low + high) // 2
    left_max = max_subarray_sum(a, low, mid)
    right_max = max_subarray_sum(a, mid + 1, high)
    cross_max = max_crossing_sum(a, low, mid, high)
    return max(left_max, right_max, cross_max)
```

전체 호출: max_subarray_sum(a, 0, 7) → max(12, 7, 17) → 17

# 경계 계산 코드 ① 왼쪽 스캔

left_sum은 mid에서 끝나는 구간 중 최댓값입니다.
range(mid, low − 1, −1)은 mid부터 low까지 한 칸씩 왼쪽으로 이동합니다.

```
def max_crossing_sum(a, low, mid, high):
    left_sum = float('-inf')
    current_sum = 0
    for i in range(mid, low - 1, -1):
        current_sum += a[i]
        if current_sum > left_sum:
            left_sum = current_sum
```

**i: 3 → 2 → 1 → 0　　기록: -2 → 10 → 10 → 10**

새 원소는 current_sum에 더합니다. left_sum은 최댓값을 보관하는 기록입니다.

# 경계 계산 코드 ② 오른쪽과 반환

current_sum만 0으로 다시 설정합니다. left_sum은 그대로 보관합니다.
오른쪽은 mid + 1부터 high까지 읽고, 두 최댓값 기록을 더해 반환합니다.

```
right_sum = float('-inf')
current_sum = 0
for j in range(mid + 1, high + 1):
    current_sum += a[j]
    if current_sum > right_sum:
        right_sum = current_sum
return left_sum + right_sum
```

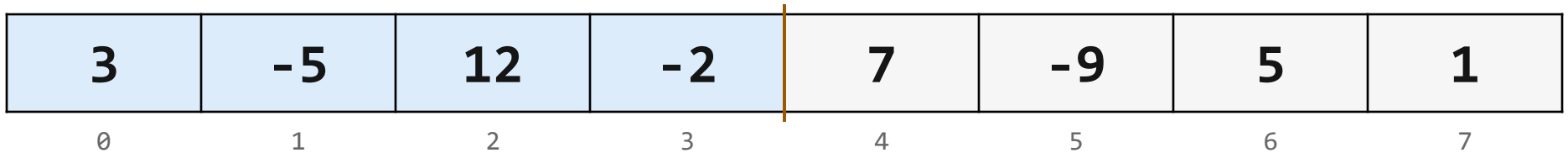
**j: 4 → 5 → 6 → 7** **반환: 10 + 7 = 17**

두 반복문은 다른 칸을 읽습니다. 경계 원소를 중복해서 더하지 않습니다.

# range의 끝값은 읽지 않습니다

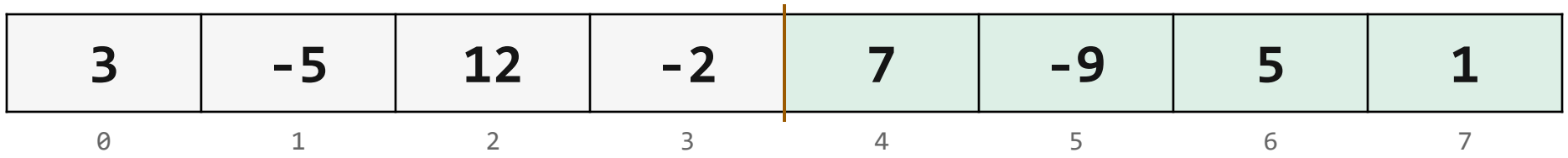
왼쪽은 3, 2, 1, 0을 읽습니다. 오른쪽은 4, 5, 6, 7을 읽습니다.
−1과 8은 range를 멈추기 위한 값이며, 배열에서 접근할 인덱스가 아닙니다.

왼쪽 인덱스




```
range(3, -1, -1)  →  3, 2, 1, 0
```

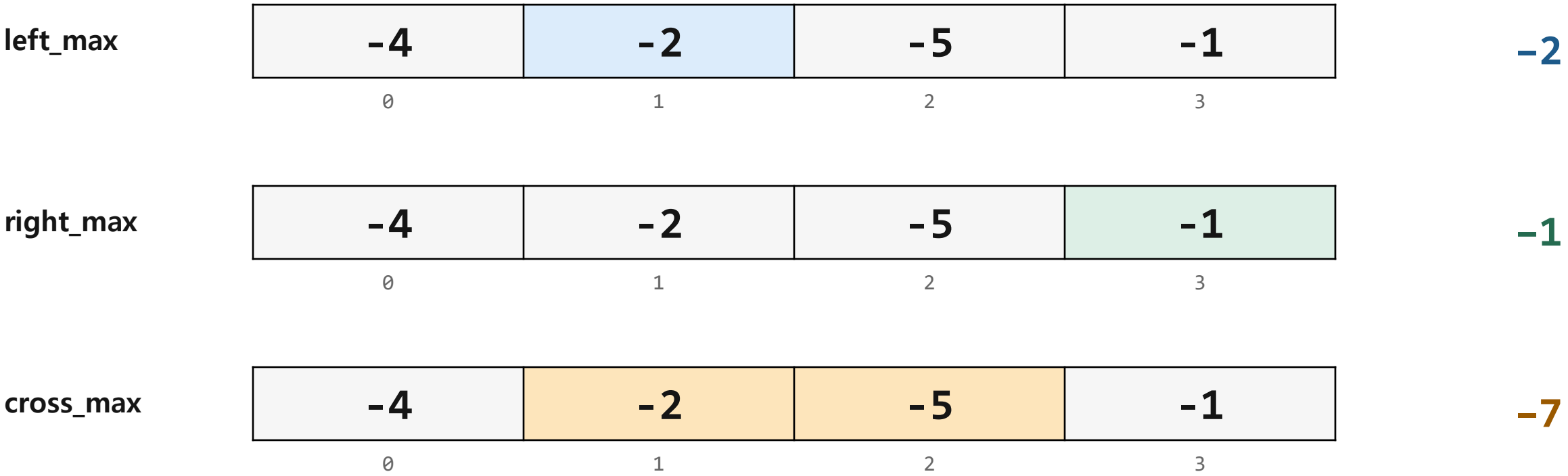
오른쪽 인덱스




range(4, 8) → 4, 5, 6, 7. 현재 호출의 low와 high까지만 읽습니다.

# 모두 음수면 0이 답이 아닙니다

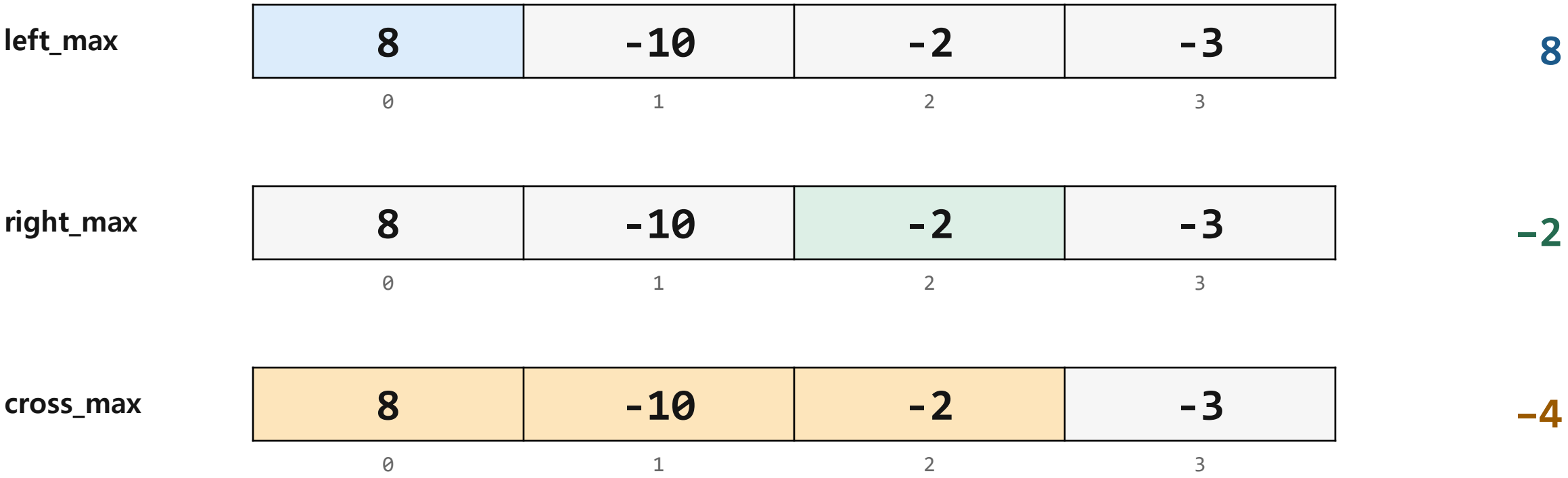
예: a = [−4, −2, −5, −1]. 비어 있지 않은 구간만 고릅니다.
왼쪽 −2, 오른쪽 −1, 걸친 합 −7 중 가장 큰 값은 −1입니다.




왼쪽 스캔: −2 → −6 (기록 −2) / 오른쪽: −5 → −6 (기록 −5)

# cross_max가 항상 이기지는 않습니다

예: a = [8, −10, −2, −3]. 경계를 걸치면 −10을 빼놓을 수 없습니다.
왼쪽 8, 오른쪽 −2, 걸친 합 −4 중 최댓값 8을 반환합니다.




경계 합: (8 − 10) + (−2) = −4. 전체 답은 왼쪽 한 칸의 8입니다.

# cross_max 한 번: 왼쪽 4칸과 오른쪽 4칸

## 왼쪽 스캔: 가운데에서 왼쪽으로

| 3 | -5 | 12 | -2 |
|---|---|---|---|

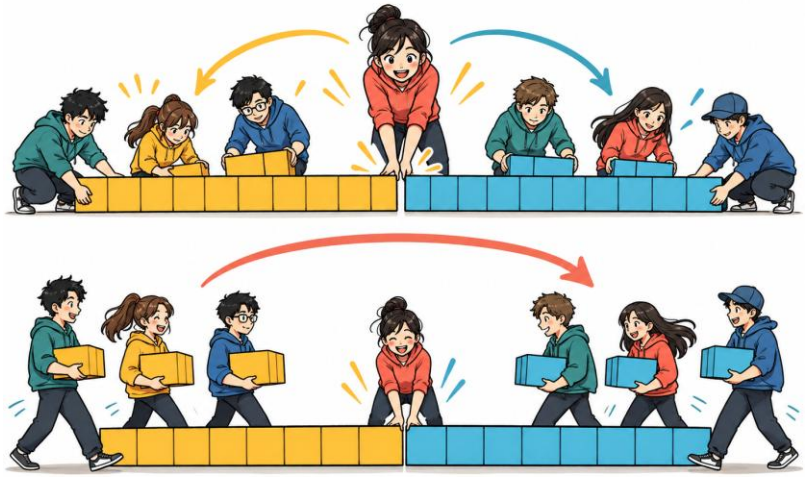
4칸

## 오른쪽 스캔: 가운데 다음부터 오른쪽으로

| 7 | -9 | 5 | 1 |
|---|---|---|---|

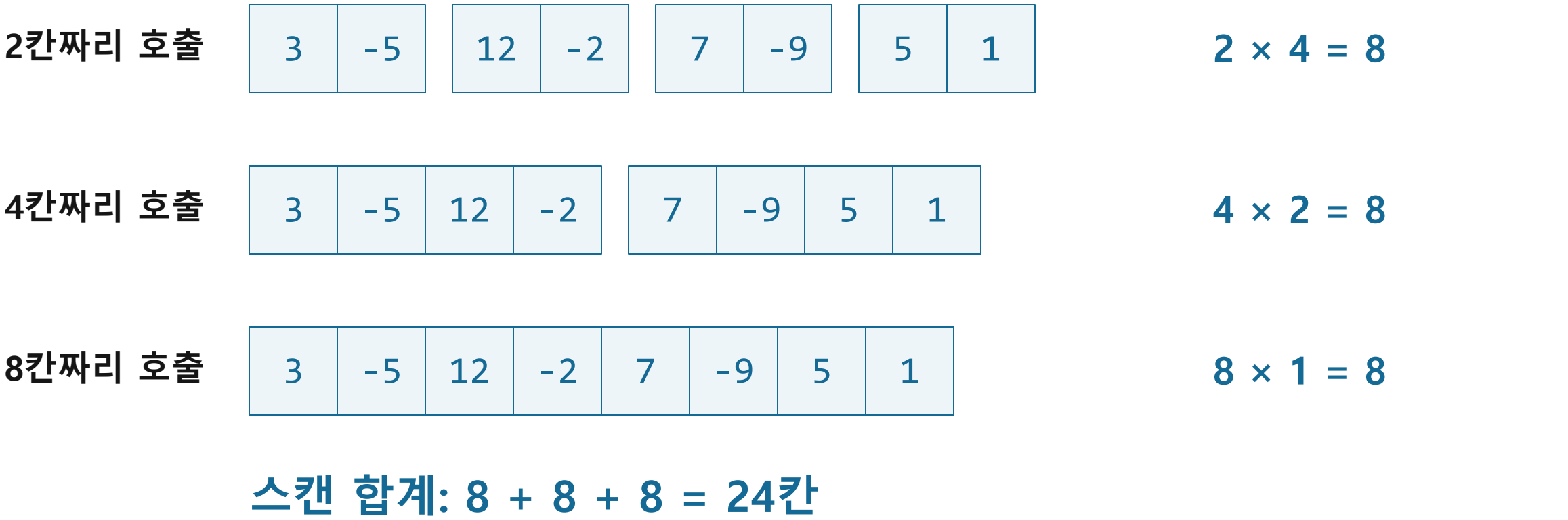
4칸

**4 + 4 = 8칸**



두 반복문은 차례대로 실행합니다. 현재 구간이 n칸이면 cross_max 한 번은 O(n)입니다.

# 작은 호출을 모아도, 한 층에서 8칸을 읽습니다




한 칸짜리에서는 스캔하지 않고 반환합니다. 호출·반환에도 비용이 있지만 전체 시간 차수는 같습니다.

# 8칸을 3층, n칸을 약 $\log_2 n$층

| 입력 길이 | 구간 크기가 줄어드는 과정 | 스캔하는 층 |
|---|---|---|
| 8 | 8, 4, 2, 1 | 3층 |
| 16 | 16, 8, 4, 2, 1 | 4층 |
| 32 | 32, 16, 8, 4, 2, 1 | 5층 |

## 한 층의 읽기 n칸 × 약 $\log_2 n$층

최대 구간 합의 전체 시간은 O(n log n). 마지막 한 칸에서는 cross_max를 계산하지 않습니다.

# 점화식은 코드의 세 부분을 적은 것입니다

$$T(n) = T(n/2) + T(n/2) + O(n)$$

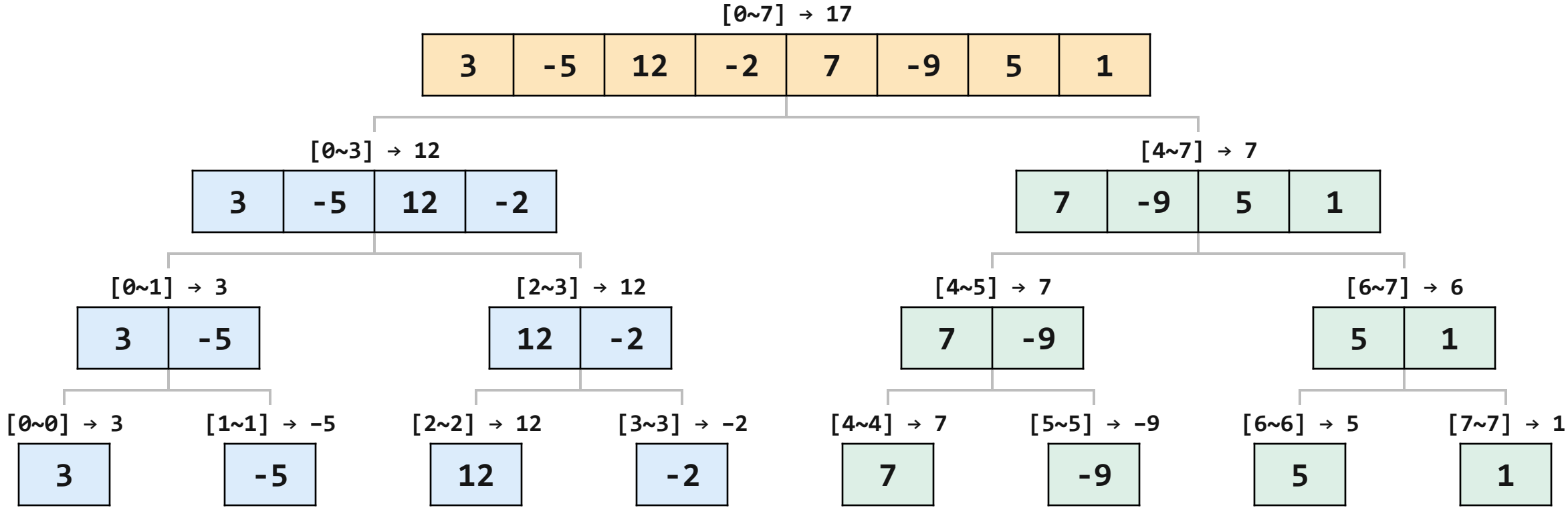
| $T(n)$ | $T(n/2)$ | $T(n/2)$ | $O(n)$ |
|---|---|---|---|
| 전체 시간 | 왼쪽 절반의 시간 | 오른쪽 절반의 시간 | 현재 구간의 작업 |
| | **왼쪽 답을 구함** | **오른쪽 답을 구함** | **중앙에서 스캔** |

$$T(n) = 2T(n/2) + O(n)$$

전체 시간은 앞의 층 그림으로 계산했습니다. n칸씩 약 $\log_2 n$층을 처리하므로 $O(n \log n)$입니다.

# 작은 반환값이 모여 전체 답 17

원소 하나 → 두 칸 → 네 칸 → 여덟 칸 순서로 답이 돌아옵니다.
전체 호출은 left_max = 12, right_max = 7, cross_max = 17을 비교합니다.




최종 구간: a[2~4] = [12, −2, 7], 합 = 17. 중간 칸을 빠뜨리지 않았습니다.

4.5

# 쿼드 트리

Quadtree
2차원 공간을 재귀적으로 4등분하는 자료 구조
이미지 압축부터 게임 충돌 감지까지

# 쿼드 트리의 기본 아이디어

쿼드 트리(quadtree)는 2차원 공간을 재귀적으로 4개의 구역(사분면)으로 분할하여 데이터를 효율적으로 표현하는 자료 구조다. 이진 탐색이 탐색 범위를 1/2씩 줄여나갔다면, 쿼드 트리는 평면을 1/4씩 줄여나간다.

**① 확인**

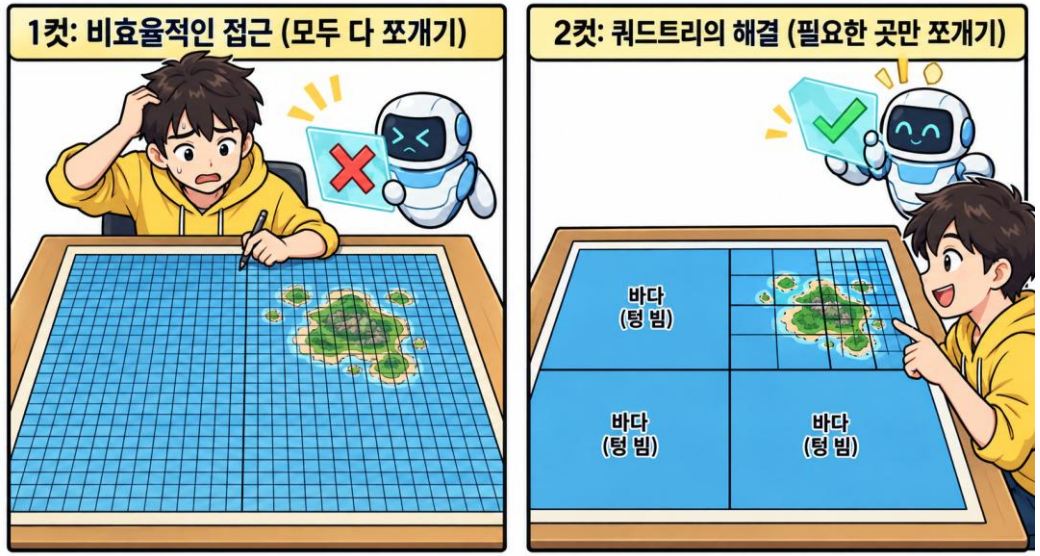
전체 영역을 확인한다.

**② 기저 사례**

데이터가 '단일 속성'(모두 같은 색이거나 텅 비어 있음)이면 더 이상 나누지 않고 하나의 노드로 표현한다.

**③ 재귀 단계**

'섞여 있다면' 영역을 정확히 4등분(좌상, 우상, 좌하, 우하)하여 각 부분에 대해 ①을 다시 수행한다.




**이론에 그치지 않는다**

게임의 물리 엔진(충돌 처리), 포토샵 같은 이미지 소프트웨어(압축)의 핵심 원리다.

# 응용 ① 흑백 이미지 압축

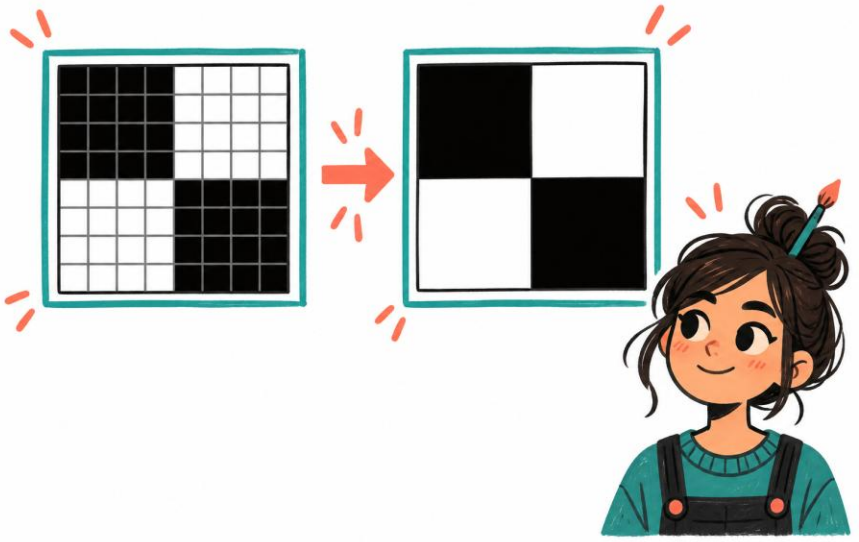
0과 1로 이루어진 영상에서 균일한 영역은 픽셀마다 저장할 필요가 없다.

① 검사: 현재 정사각형이 모두 같은 값인지 확인한다.

② 기저 사례: 모두 0이면 0, 모두 1이면 1을 기록하고 끝낸다.

③ 분할: 값이 섞여 있으면 네 영역으로 나눈다.
좌상 → 우상 → 좌하 → 우하 순서로 재귀 호출하고 결과를 괄호로 묶는다.

균일한 영역이 클수록 짧게 표현된다. 색이 복잡하게 섞이면 압축 효과는 줄어든다.



# 같은 색이면 하나로, 섞이면 네 조각

균일한 영역은 0 또는 1. 혼합 영역만 네 부분으로 나눕니다.




좌상 → 우상 → 좌하 → 우하 순서를 저장과 복원에서 똑같이 사용합니다.

# 입력 영상: 검은 칸의 위치부터 확인합니다

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

행 / 열은 0부터 셉니다.

첫 호출: (행 0, 열 0, 크기 8)

오른쪽 위는 전부 1입니다.

추가 검은 칸: 행 5, 열 1

전체는 0과 1이 섞였습니다.

**전체는 혼합 → 네 부분으로 분할**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# 첫 분할: 네 호출의 좌표를 정합니다

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

행 / 열은 0부터 셉니다.

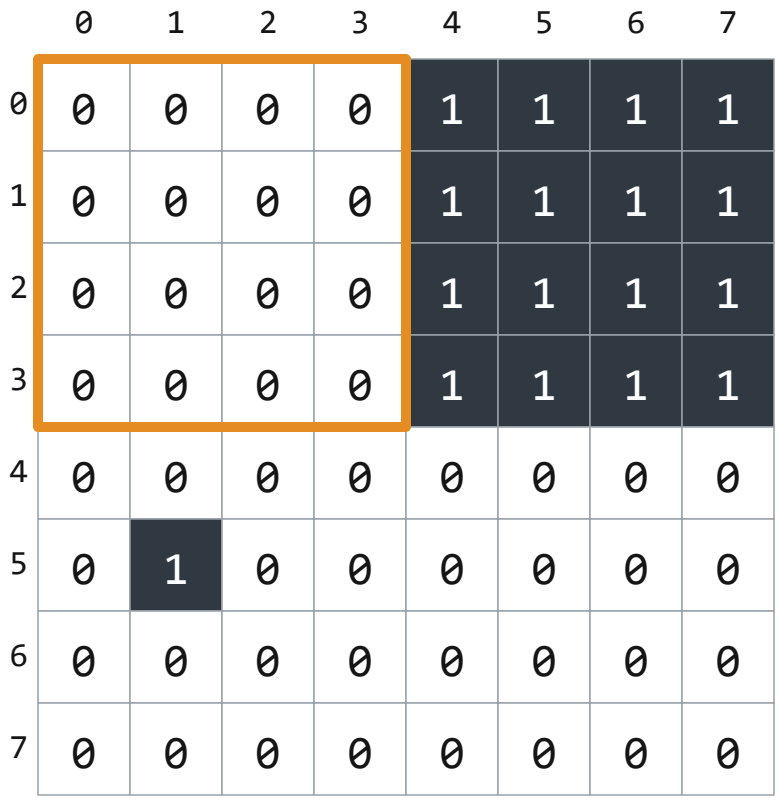
half = 8 // 2 = 4

좌상: (0, 0, 4)
우상: (0, 4, 4)
좌하: (4, 0, 4)
우하: (4, 4, 4)

**부모 결과의 틀: ( ? ? ? ? )**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# ① 좌상: 16칸을 0 하나로 반환



| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

행 / 열은 0부터 셉니다.

현재 호출: (0, 0, 4)

16칸이 모두 0입니다.
더 나누지 않습니다.

반환: "0"

**부모가 모은 결과: ( 0 ? ? ? )**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# ② 우상: 16칸을 1 하나로 반환

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

행 / 열은 0부터 셉니다.

현재 호출: (0, 4, 4)

16칸이 모두 1입니다.
더 나누지 않습니다.

반환: "1"

**부모가 모은 결과: ( 0 1 ? ? )**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# ③ 좌하: 검은 칸이 있어 다시 나눕니다

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

행 / 열은 0부터 셉니다.

현재 호출: (4, 0, 4)

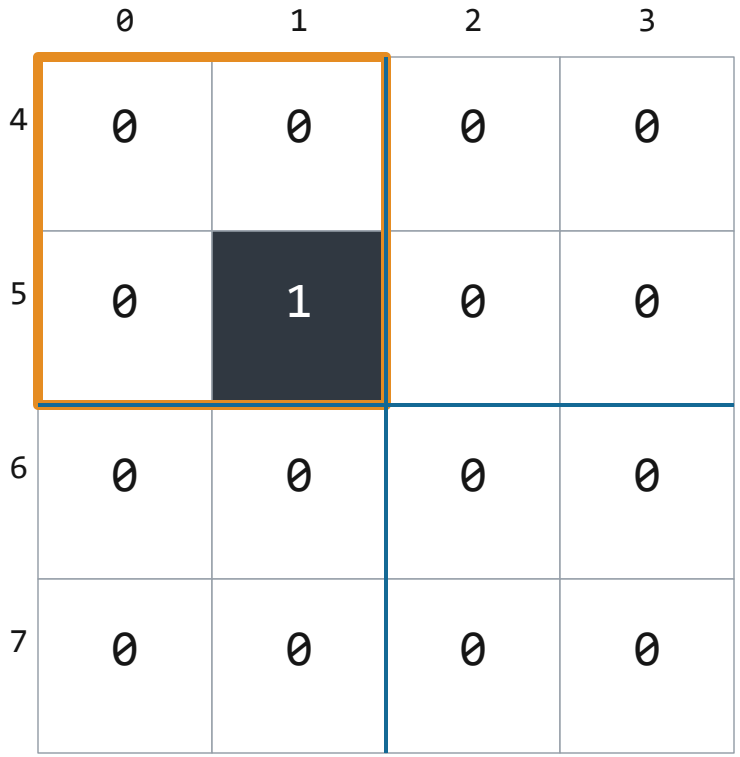
행 5, 열 1의 값이 1입니다.
다른 칸은 0 → 혼합입니다.

다시 크기 2의 네 영역으로 분할

**루트는 좌하의 결과를 기다립니다.**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# 좌하의 네 부분: 첫 2×2만 혼합입니다




행 / 열은 0부터 셉니다.

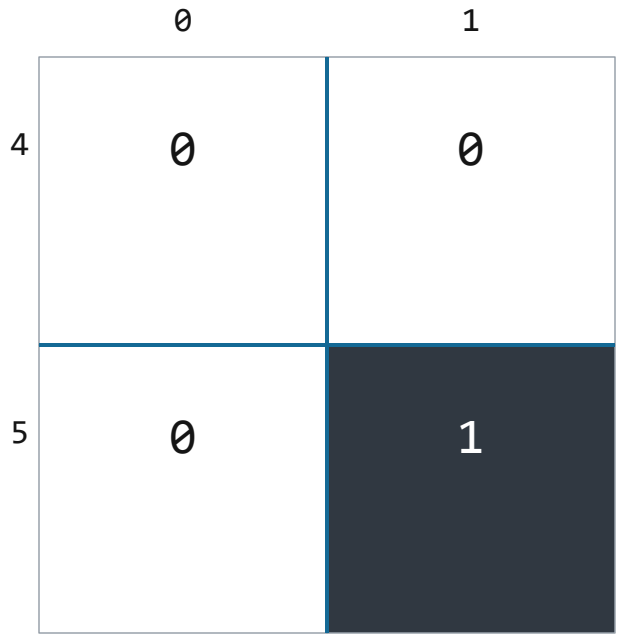
크기 2의 자식들

좌상 (4, 0, 2): 혼합
우상 (4, 2, 2): 전부 0
좌하 (6, 0, 2): 전부 0
우하 (6, 2, 2): 전부 0

**먼저 (4, 0, 2)를 처리합니다.**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# 혼합된 2×2: 네 픽셀을 순서대로 읽습니다




행 / 열은 0부터 셉니다.

현재 호출: (4, 0, 2)

좌상 (4, 0): 0
우상 (4, 1): 0
좌하 (5, 0): 0
우하 (5, 1): 1

**네 자식의 색 순서: 0 → 0 → 0 → 1**

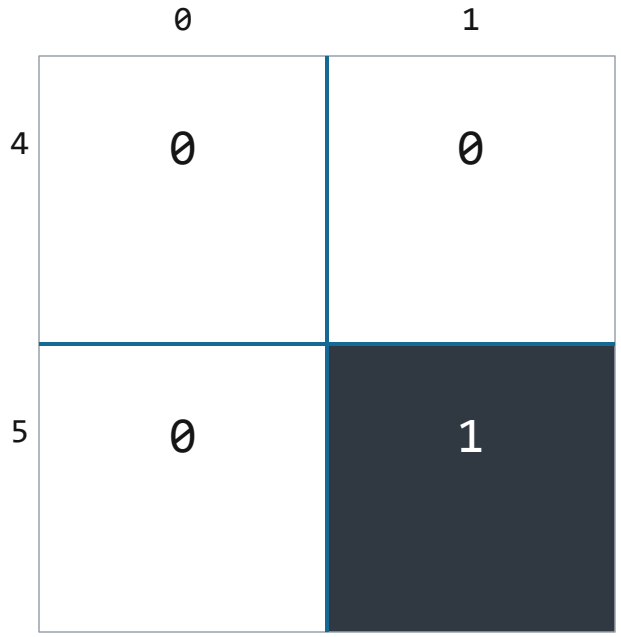
0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# 크기 1의 네 호출: 각각 즉시 끝납니다




한 칸은 반드시 균일합니다. 크기 1 아래로는 더 나누지 않습니다.

# 2×2의 답: (0001)을 반환합니다




행 / 열은 0부터 셉니다.

두 번째 층의 혼합 영역

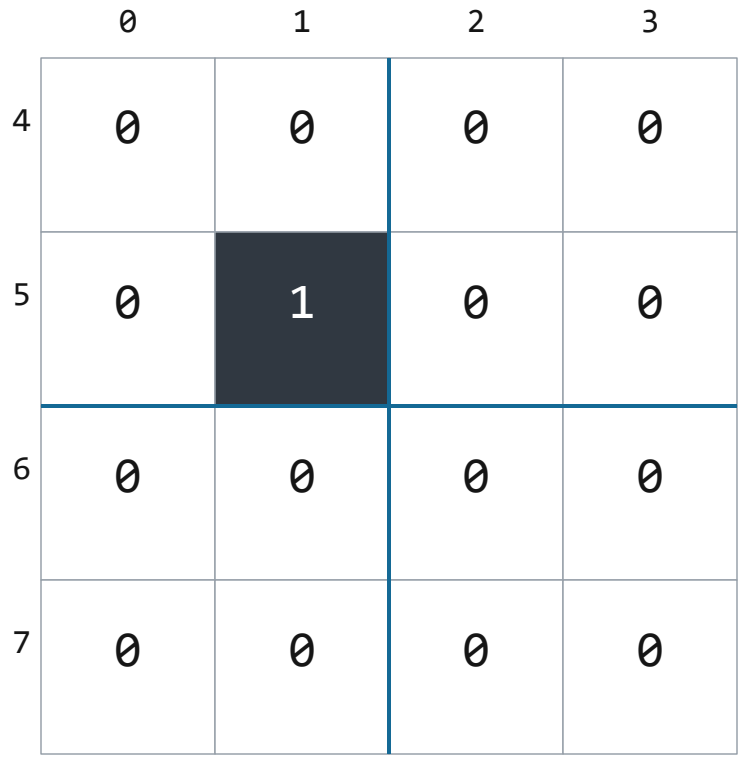
여는 괄호: (
자식 결과: 0 0 0 1
닫는 괄호: )

합치면 "(0001)"

**좌하 4×4의 첫 자식 답: (0001)**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# 좌하의 나머지 세 영역은 모두 0입니다




행 / 열은 0부터 셉니다.

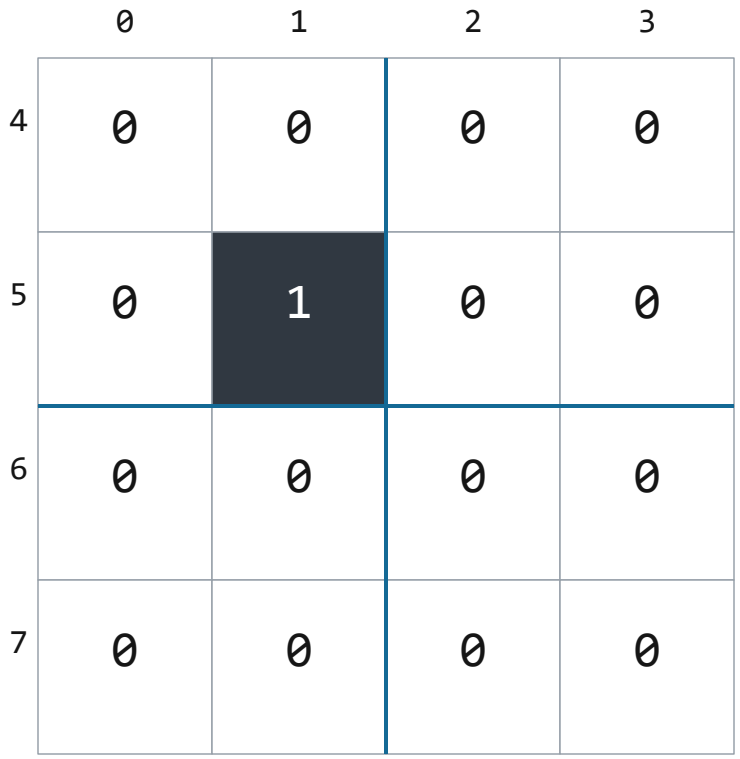
첫 자식: (0001)

우상 (4, 2, 2) → "0"
좌하 (6, 0, 2) → "0"
우하 (6, 2, 2) → "0"

**네 답을 모두 받았습니다.**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# 좌하 4×4의 답: ((0001)000)




행 / 열은 0부터 셉니다.

좌하 4×4가 받은 네 결과

(0001) / 0 / 0 / 0

자기 괄호로 한 번 더 묶습니다.

반환: "((0001)000)"

**루트: ( 0 1 ((0001)000) ? )**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# ④ 우하: 마지막 16칸도 0 하나

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

행 / 열은 0부터 셉니다.

현재 호출: (4, 4, 4)

16칸이 모두 0입니다.
더 나누지 않습니다.

반환: "0"

**루트의 네 자식 결과가 완성되었습니다.**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# 전체 결과: (01((0001)000)0)

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

행 / 열은 0부터 셉니다.

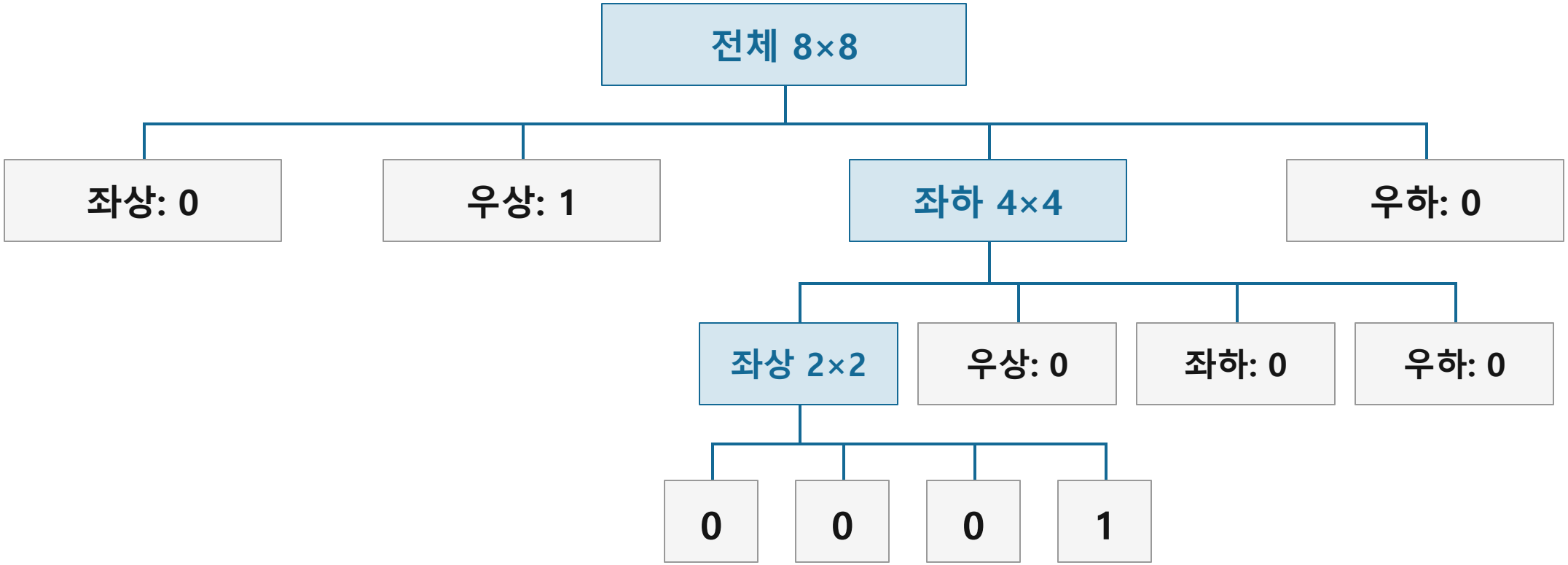
좌상: 0
우상: 1
좌하: ((0001)000)
우하: 0

전체를 괄호로 묶습니다.

**완성: (01((0001)000)0)**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# 재귀 트리: 나눈 곳만 자식 네 개




## 혼합 노드 3개 + 단색 리프 10개 = 총 13회 호출

선이 이어진 부모만 네 자식으로 나눕니다. 파란 상자 3개는 각각 괄호 1쌍입니다.

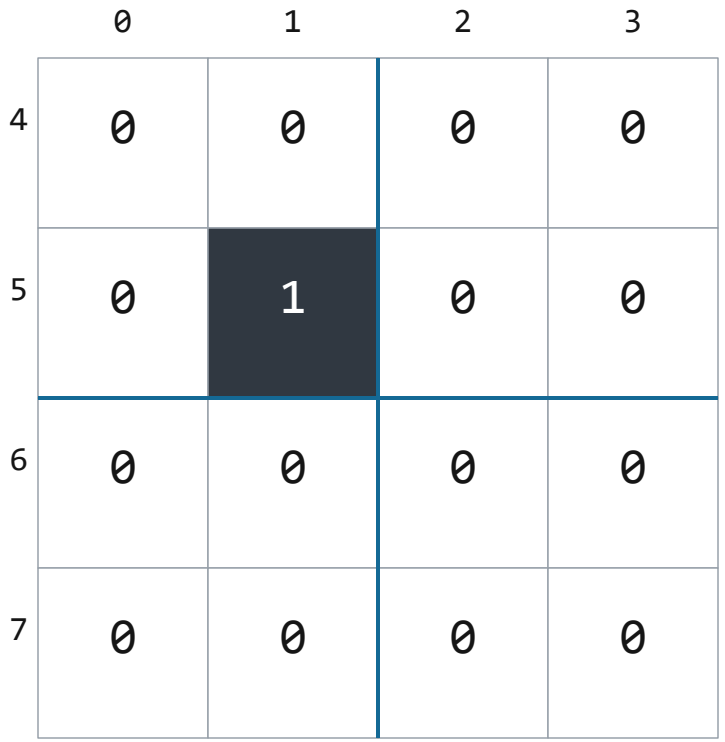
# 복원 ①: 괄호를 만나면 네 영역으로

**`( 0 1 ((0001)000) 0 )`**

| 읽은 표현 | 복원할 위치 | 하는 일 |
|---|---|---|
| 첫 ( | 전체 8×8 | 크기 4의 네 자식으로 내려감 |
| 0 | 좌상 4×4 | 16칸을 모두 0으로 채움 |
| 1 | 우상 4×4 | 16칸을 모두 1로 채움 |
| 다음 ( | 좌하 4×4 | 크기 2의 네 자식으로 내려감 |

복원은 위치와 크기를 함께 전달합니다. 색 한 글자는 현재 영역 전체를 채웁니다.

# 복원 ②: 좌하의 겹친 괄호를 읽습니다




행 / 열은 0부터 셉니다.

좌하 표현: ((0001)000)

첫 2×2: (0001)
→ 네 픽셀을 0, 0, 0, 1로

나머지 2×2 세 영역: 0, 0, 0
→ 각 영역 전체를 흰색으로

**행 5, 열 1의 검은 칸도 복원!**

닫는 괄호를 만나면, 그 괄호가 시작한 부모 영역으로 돌아옵니다.

# 복원 ③: 마지막 0을 채우면 원본과 같습니다

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 2 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

행 / 열은 0부터 셉니다.

우하의 "0" → 16칸을 흰색으로

루트의 ")" → 전체 복원 종료

검은 칸 위치가 원본과 같습니다.

저장·복원의 자식 순서는 동일!

**64개 픽셀의 색이 모두 복원됩니다.**

0 = 흰색, 1 = 검은색. 현재 호출의 영역만 처리한 뒤 결과를 반환합니다.

# 모든 그림이 더 작아지는 것은 아닙니다

## 단색 4×4

| | | | |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |

**16개 값 → 1글자**

## 체커보드 4×4

| | | | |
|---|---|---|---|
| 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 |

**16개 값 → 26글자**

균일 영역은 유리하고, 끝까지 섞인 무늬는 괄호 때문에 표현이 더 길어질 수 있습니다.

# 쿼드 트리 알고리즘 (의사코드)

quadtree_1.pseudo

```
ConstructQuadTree(I, x, y, size):
    initialValue <- I[x][y]
    isUniform <- true
    for i <- x to x + size - 1 do
        for j <- y to y + size - 1 do
            if I[i][j] != initialValue then
                isUniform <- false
    if isUniform then
        return CreateLeafNode(initialValue)
```

quadtree_2.pseudo

```
  else
     half <- size / 2
     root <- CreateInternalNode()
     root.topLeft     <- ConstructQuadTree(I, x,      y,      half)
     root.topRight    <- ConstructQuadTree(I, x,      y+half, half)
     root.bottomLeft  <- ConstructQuadTree(I, x+half, y,      half)
     root.bottomRight <- ConstructQuadTree(I, x+half, y+half, half)
     return root
```

**트리 깊이 ≤ log_2(size)**

### 노드 하나가 곧 한 영역

리프 노드는 균일한 영역을, 내부 노드는 4개의 자식으로 쪼개진 영역을 뜻한다. 트리의 깊이는 최대 $\log_2(\text{size})$이다.

# 파이썬 구현 — 8×8 이미지 압축

quadtree_compress.py

```
def quadtree_compress(x, y, n, image):
    color = image[x][y]
    is_uniform = True
    for i in range(x, x + n):
        for j in range(y, y + n):
            if image[i][j] != color:
                is_uniform = False
                break
        if not is_uniform:
            break
    if is_uniform:
        return str(color)
    half = n // 2
    result = "("
    result += quadtree_compress(x, y, half, image)
    result += quadtree_compress(x, y + half, half, image)
    result += quadtree_compress(x + half, y, half, image)
    result += quadtree_compress(x + half, y + half, half, image)
    result += ")"
    return result
```

example.py

```
example_image = [
    [0,0,0,0, 1,1,1,1],
    [0,0,0,0, 1,1,1,1],
    [0,0,0,0, 1,1,1,1],
    [0,0,0,0, 1,1,1,1],
    [0,0,0,0, 0,0,0,0],
    [0,1,0,0, 0,0,0,0],
    [0,0,0,0, 0,0,0,0],
    [0,0,0,0, 0,0,0,0],
]
print(quadtree_compress(0, 0, 8, example_image))
```

OUTPUT

**(01((0001)000)0)**

픽셀 64개 → 문자 16개

# 트리 깊이와 픽셀 검사 횟수는 다릅니다

| 호출 영역 | 그 영역의 검사 횟수 |
|---|---|
| 전체 8×8 | 다른 색까지 5 |
| 균일한 4×4 세 영역 | 16 × 3 = 48 |
| 혼합 4×4 / 혼합 2×2 | 6 + 4 = 10 |
| 균일한 2×2 세 영역 / 픽셀 네 개 | 4 × 3 + 1 × 4 = 16 |

## 실제 예제: 5 + 48 + 10 + 16 = 79회 검사

현재 Python의 조기 종료를 포함한 값입니다. 출력 문자열을 붙이는 비용은 별도로 있습니다.

# 영상 전체를 매번 검사할 때의 시간 상한

## 한 변 S = 8 / 전체 픽셀 N = $S^2$ = 64

크기 8: 1개



**64칸**

크기 4: 4개



**4 × 16 = 64칸**

크기 2: 16개



**16 × 4 = 64칸**

크기 1: 64개



**64 × 1 = 64칸**

전체 검사 모델: O(N log S) 상한. 조기 종료를 하는 실제 코드는 더 적게 검사할 수 있습니다.

# 응용 ② 게임 개발과 충돌 감지

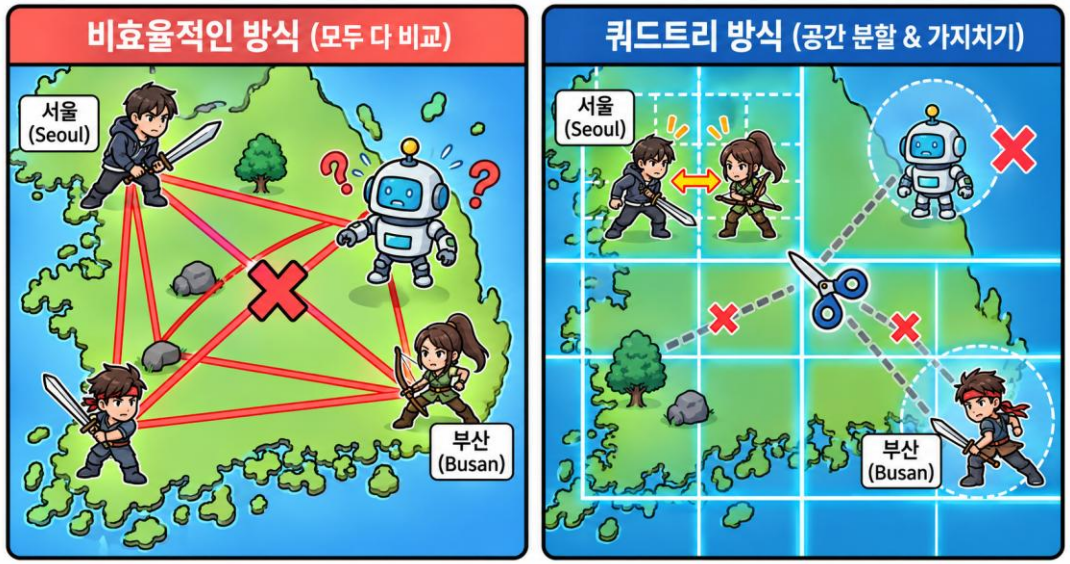
화면 속에는 수십 명의 캐릭터, 수백 발의 총알, 수천 개의 지형지물이 움직인다. 게임 엔진은 이 객체들이 서로 부딪혔는지 판별하는 충돌 감지(collision detection)를 매 프레임(약 0.016초)마다 수행해야 한다.

**무식한 접근 — 브루트 포스**

$$(n-1) + (n-2) + \cdots + 1 \approx n^2/2$$

객체 1,000개 → 매 프레임 약 50만 번 검사

1. **공간 분할** — 전체 맵을 4개의 사분면으로 나눈다. 한 구역에 오브젝트가 너무 많으면 그 구역을 다시 4개로 쪼갠다.
2. **객체 등록** — 모든 오브젝트는 자신의 위치에 따라 트리의 특정 노드(구역)에 등록된다.
3. **지역적 검사** — 현재 객체와 겹칠 수 있는 영역만 조회한다. 경계를 걸친 객체와 이웃 영역도 포함해 충돌 후보를 빠짐없이 검사한다.

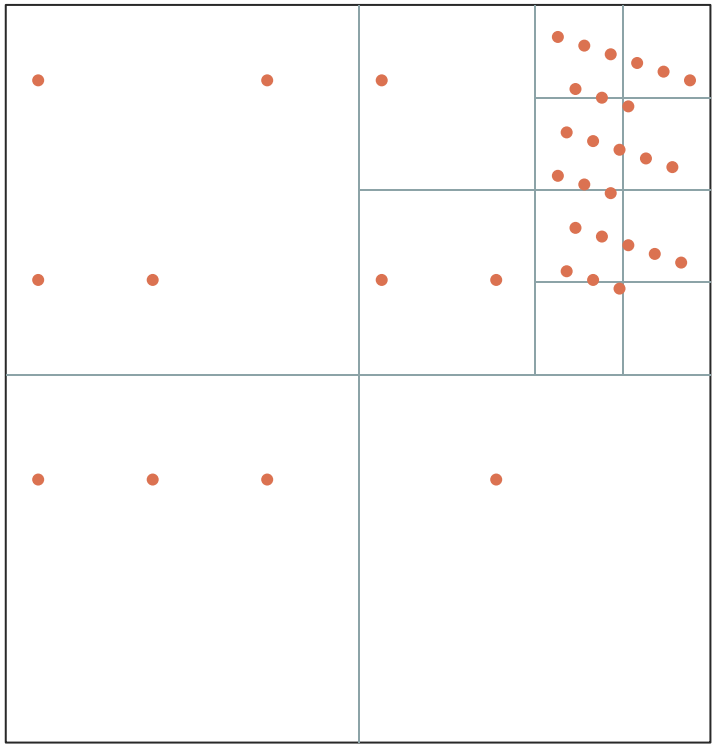



"서로 멀리 떨어져 있는 물체끼리는 충돌 검사를 할 필요가 없다"

# 성능의 혁신과 LOD (Level of Detail)

쿼드 트리는 지형의 LOD(Level of Detail)를 관리하는 데 활용할 수 있다. 가까운 영역은 작은 격자로, 먼 영역은 큰 격자로 표현한다. 화면에 필요한 세밀함을 판단하는 로직과 함께 사용한다.

| | |
|---|---|
| **가까운 영역** | 깊게 분할 → 고해상도 렌더링 |
| **먼 영역** | 얕게 분할 → 저해상도 렌더링 |
| **결과** | 같은 프레임 예산으로 훨씬 넓은 세계를 그린다 |



점이 밀집된 영역을 더 세밀하게 분할

# 마스터 정리: 세어 본 작업을 식과 연결

**먼저 그림에서 센 내용**

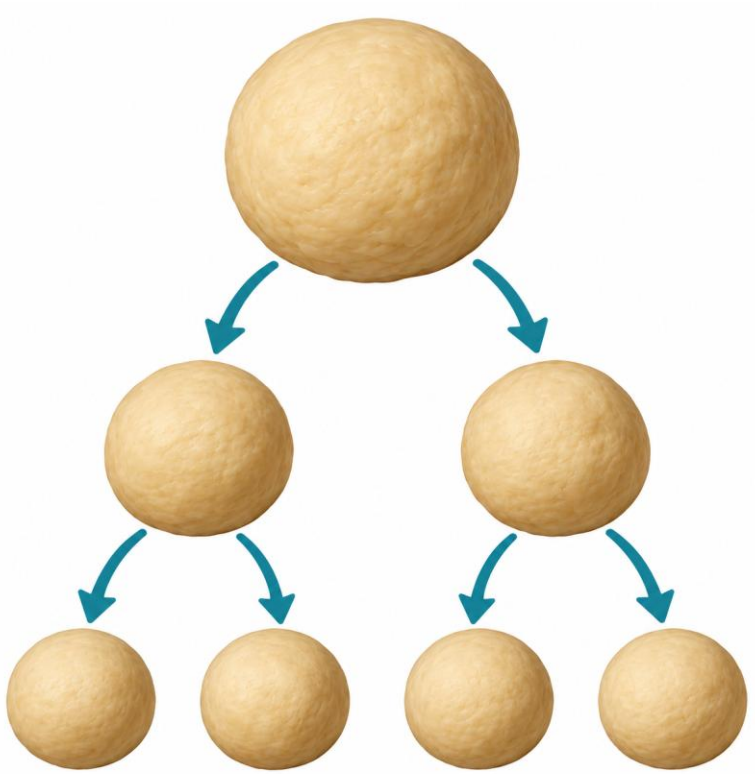
## 한 층에서 하는 일

병합·중앙 스캔: n에 비례

## 그런 층의 개수

반씩 줄이면: $\log_2 n$에 비례

**전체: $n \times \log_2 n$에 비례**



오른쪽은 절반씩 나뉘는 호출 구조입니다. 각 층의 작업량을 함께 세어 전체 시간을 구합니다.

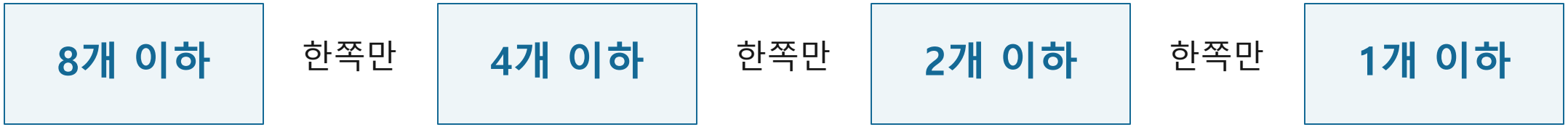
# 시간식의 T, 2, n/2, O(n)을 하나씩 읽습니다

$$T(n) = 2T(n/2) + O(n)$$

| 식의 부분 | 우리말로 읽기 |
|---|---|
| T(n) | n개 입력을 끝까지 푸는 전체 시간 |
| T(n/2) | 절반 크기의 작은 문제 하나를 푸는 시간 |
| 앞의 2 | 왼쪽과 오른쪽, 작은 문제를 두 개 풂 |
| + O(n) | 현재 구간의 n개 값에 비례하는 추가 작업 |

2T(n/2)와 O(n)을 더한 뒤, 이 구조가 몇 층 반복되는지 계산해야 합니다.

# 이진 탐색: 절반 두 개 중 한쪽만 계속 봅니다




## 각 단계에서 가운데 값 하나를 비교

여러 갈래 전체를 함께 탐색하지 않습니다.

일찍 찾으면 끝납니다. 최대 단계 수는 $\log_2 n$에 비례하고, 전체 시간은 $O(\log n)$입니다.

# 이진 탐색은 한 단계에 n개를 훑지 않습니다

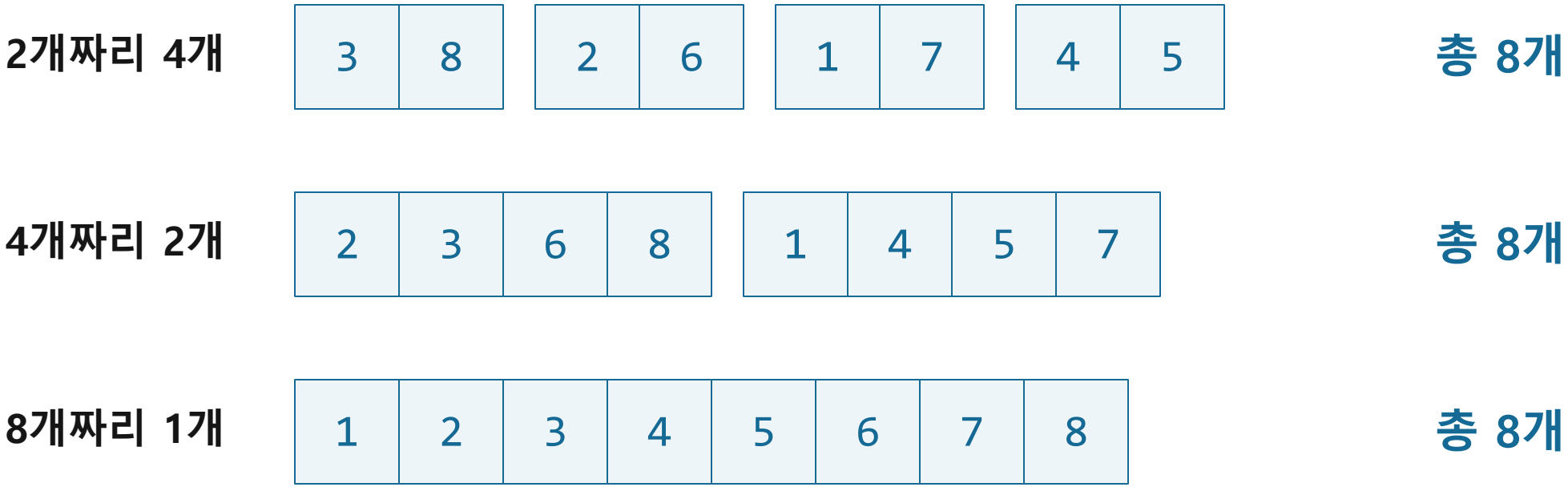
$$T(n) = T(n/2) + O(1)$$

**절반 크기의 문제 한 개** **가운데 값 비교와 경계 변경**

**한 단계의 일정한 작업 × 약 $\log_2 n$단계**

O(1)은 입력 길이에 따라 늘어나지 않는 작업량입니다. 전체 시간은 O(log n)입니다.

# 병합 정렬은 양쪽 절반을 모두 풉니다




모든 갈래를 풉니다. 한 층은 O(n), 병합 층 수는 O(log n), 전체는 O(n log n)입니다.

# 마스터 정리의 a, b, f(n)은 코드에서 찾습니다

$$T(n) = a \times T(n/b) + f(n)$$

| 기호 | 병합 정렬 코드에서의 뜻 |
|---|---|
| a = 2 | 왼쪽과 오른쪽, 작은 문제 두 개 |
| b = 2 | 각 작은 문제의 길이는 원래 길이의 절반 |
| f(n) | 정렬된 두 결과를 합치는 작업: n에 비례 |

일반 정리의 적용 조건은 뒤의 참고 페이지에 있습니다. 호출 개수와 크기 감소율을 구분합니다.

# 전체 시간은 작업량과 호출 구조를 함께 봅니다

| 알고리즘 | 현재 호출과 다음 호출 | 전체 시간 |
|---|---|---|
| 이진 탐색 | 한 값 비교 + 한쪽 절반만 탐색 | O(log n) |
| 병합 정렬 | 현재 구간 병합 + 양쪽 절반 정렬 | O(n log n) |
| 최대 구간 합 | 중앙 스캔 + 양쪽 절반의 답 계산 | O(n log n) |
| 빠른 거듭제곱 | 일정한 곱셈 + 절반 지수 호출 한 번 | O(log(b+1)) |

퀵 정렬: 균형 있는 분할은 O(n log n), 최악의 치우친 분할은 $O(n^2)$입니다.

# 참고: 마스터 정리의 일반 조건

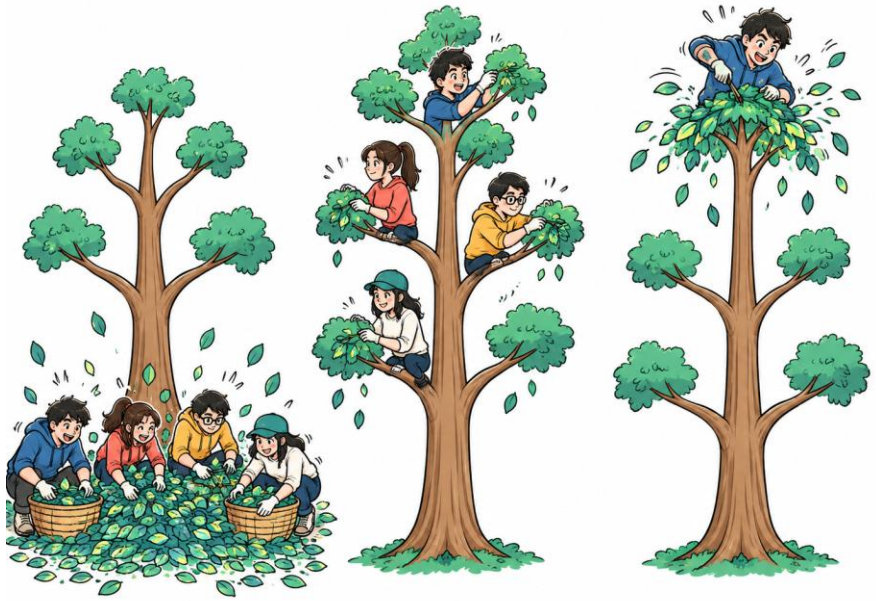
기준 지수 $p = \log_b a$, $\varepsilon > 0$으로 두고 비교한다.

① $f(n) = O(n^{(p-\varepsilon)})$
하위 문제의 비용이 지배 → $T(n) = \Theta(n^p)$

② $f(n) = \Theta(n^p)$
각 층의 비용이 비슷 → $T(n) = \Theta(n^p \log n)$

③ $f(n) = \Omega(n^{(p+\varepsilon)})$
현재 단계의 비용이 지배 → $T(n) = \Theta(f(n))$

③은 $a \cdot f(n/b) \le c \cdot f(n)$, $c < 1$인 정규성 조건도 필요하다. 조건에 맞지 않으면 다른 분석을 사용한다.



4.7

# 코딩 테스트

분할 정복으로 실전 문제 두 개를 직접 풀어본다

# 색종이 만들기

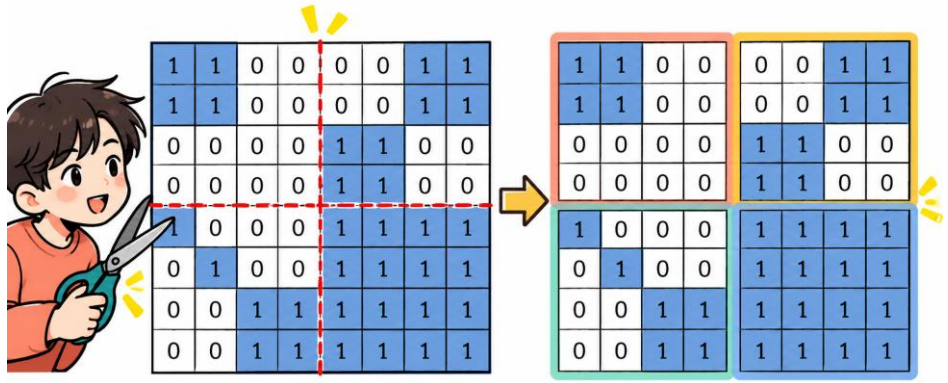
여러 개의 정사각형 칸(N × N)으로 이루어진 종이가 있다. 각 칸은 하얀색 아니면 파란색이다. 전체 종이가 모두 같은 색이 아니면, 가로와 세로의 중앙을 잘라 똑같은 크기의 네 개의 N/2 × N/2 색종이로 나눈다. 이 과정을 잘라진 종이가 모두 같은 색이 되거나 한 칸이 될 때까지 반복한다.

최종적으로 하얀색 색종이와 파란색 색종이가 각각 몇 장인지 구하라.

쿼드 트리와 완전히 동일한 구조

검사(check) → 균일하면 카운트, 아니면 4등분

한 칸(n = 1)은 언제나 균일하므로 재귀는 반드시 끝난다



**입력 (N = 8)**

`1 1 0 0 0 0 1 1  …  0 0 1 1 1 1 1 1`

**출력**

**하얀색 9장, 파란색 7장**

# 풀이 — 재귀적으로 4등분하며 세기

solve.py

```
import sys
sys.setrecursionlimit(10 ** 6)
def solve(x, y, n):
    global white_cnt, blue_cnt
    color = paper[x][y]
    for i in range(x, x + n):
        for j in range(y, y + n):
            if paper[i][j] != color:
                nn = n // 2
                solve(x,      y,      nn)
                solve(x,      y + nn, nn)
                solve(x + nn, y,      nn)
                solve(x + nn, y + nn, nn)
                return
    if color == 0:
        white_cnt += 1
    else:
        blue_cnt += 1
```

run.py

```
N = 8
white_cnt = blue_cnt = 0
solve(0, 0, N)
print(f"하얀색 색종이: {white_cnt}")
print(f"파란색 색종이: {blue_cnt}")
```

**OUTPUT**

```
하얀색 색종이: 9
파란색 색종이: 7
```

### return 위치가 핵심

다른 색을 만나는 즉시 4등분 후 return 해야 한다. 그러지 않으면 아래의 카운트 코드까지 실행되어 잘못된 답이 나온다.

# 색종이 만들기: 넓이와 조각 수를 구분

| | | | |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 |

단색 4×4

1인 픽셀: 16개
최종 종이 조각: 1개

단색 확인은 16칸 검사
분할 재귀는 필요 없습니다.

**한 변과 픽셀 수를 구분하세요.**

검사 비용: 단색 $\Theta(N)$. 반복 전체 검사 모델 $O(N \log S)$ 상한, $N = S^2$.

# 곱셈 — A^B mod C 를 눈 깜짝할 사이에

세 개의 자연수 A, B, C가 주어진다. A를 B번 곱한 수, 즉 A^B를 C로 나눈 나머지를 구하라. 문제는 A와 B가 최대 2,147,483,647(약 21억)에 달하는 매우 큰 수라는 점이다. 단순한 반복으로는 21억 번의 곱셈이 필요하다.

**지수를 절반으로**

A^B = A^(B/2) × A^(B/2). 절반만 계산하면 나머지 절반은 공짜다.

**B가 홀수라면**

A^B = A^(B/2) × A^(B/2) × A. 남는 A 하나를 곱해준다.

**모듈러 성질 활용**

(a × b) mod c = ((a mod c) × (b mod c)) mod c. 매 단계 나머지를 취해 수가 커지지 않게 한다.

$$T(n) = T(n/2) + O(1) \quad \Rightarrow \quad \Theta(\log B)$$

**예제**

입력

```
10 11 12
```

출력

```
4
```

21억 번의 곱셈이
약 31단계로 줄어든다.
($\log_2$ 2,147,483,647 ≈ 31)

# 풀이 — 분할 정복 거듭제곱 (fast power)

power.py

```
def power(a, b, c):
    if b == 0:
        return 1 % c
    half = power(a, b // 2, c)
    result = (half * half) % c
    if b % 2 == 1:
        result = (result * a) % c
    return result
```

run.py

```
A, B, C = map(int, "10 11 12".split())
print(power(A, B, C))
```

**OUTPUT** 4

### 재귀 호출 추적

| | |
|---|---|
| `power(10, 11, 12)` | b = 11 (홀수) |
| `power(10, 5, 12)` | b = 5 (홀수) |
| `power(10, 2, 12)` | b = 2 (짝수) |
| `power(10, 1, 12)` | b = 1 (홀수) |
| `power(10, 0, 12)` | return 1 % c |

### 깊이는 $\log_2 B$

B = 21억이어도 재귀 깊이는 31에 불과하다.

# 빠른 거듭제곱: $3^{11} \bmod 7$을 끝까지

| 11 | 5 | 2 | 1 | 0 |
|---|---|---|---|---|

| 돌아온 지수 b | half | 계산 후 나머지 |
|---|---|---|
| 0 | 기저 사례 | 1 |
| 1 | 1 | $1 \times 1 \times 3 \bmod 7 = 3$ |
| 2 | 3 | $3 \times 3 \bmod 7 = 2$ |
| 5 | 2 | $2 \times 2 \times 3 \bmod 7 = 5$ |
| 11 | 5 | $5 \times 5 \times 3 \bmod 7 = 5$ |

각 단계는 나머지를 저장합니다. 결과: $3^{11} \bmod 7 = 5$.

# 지수 절반 호출 한 번: O(log b)

```
half = power(a, b // 2, c)  ← 한 번만 호출
```

```
result = (half * half) % c
```

홀수 b일 때만 a를 한 번 더 곱합니다.

**T(b) = T(⌊b/2⌋) + O(1) → O(log b)**

정해진 크기의 정수 연산을 상수로 세는 기준입니다. b = 0은 바로 반환합니다.

# 4장 정리

### 분할 정복의 3단계

분할(divide) → 정복(conquer) → 결합(combine). 기저 사례에 도달할 때까지 재귀적으로 나눈다.

### 결합이 알고리즘의 핵심

병합 정렬은 나누기가 쉽고 합치기가 어렵다. 반대로 퀵 정렬은 나누기가 어렵고 합치기가 쉽다.

### 연속 구간의 경계를 확인합니다

최대 구간 합은 왼쪽·오른쪽 내부와 중앙을 걸친 구간을 모두 비교합니다.

### 공간도 분할한다

쿼드 트리는 평면을 1/4씩 재귀 분할한다. 이미지 압축, 충돌 감지, LOD의 기반이다.

### 마스터 정리

T(n) = aT(n/b) + f(n) 형태와 각 경우의 적용 조건을 확인한 뒤 비용의 크기를 비교합니다.

### 종료 조건과 재귀 구간

빈 구간과 한 원소에서 종료합니다. 퀵 정렬은 확정된 피벗을 제외한 두 구간만 다시 정렬합니다.

# Q & A

질문을 나누어 봅시다.
큰 질문도 작게 쪼개면 답할 수 있습니다.